

PAdicMIDI

*A Python Toolkit for Hierarchical, Ultrametric, and
p-adic Analysis of Symbolic Music Data*

Reporte Técnico Integral

Versión 1.0.0 — 29 de abril de 2026

Autor

Jesús Rogelio Pérez Buendía

(publica académicamente como J.~Rogelio Pérez-Buendía)

SECIHTI — Centro de Investigación en Matemáticas (CIMAT)
Unidad Mérida, Yucatán, México

ORCID: [0000-0002-7739-4779](https://orcid.org/0000-0002-7739-4779)

Página web: <https://www.cimat.mx/~rogelio.perez>

Software en: personal.cimat.mx:8181/~rogelio.perez/desarrollo-tecnologico/

Correo: rogelio@ciimat.mx

Grupo: P-ADAGIO

Financiamiento: SECIHTI México — proyecto CF-2019/217367

P-adic Arithmetic, Dynamics And Galois-Informed Observations

Documento preparado para el Registro Autoral de Programa de
Computación ante el Instituto Nacional del Derecho de Autor
(INDAUTOR), formato **RPDA-03**, conforme a los
artículos 13 fracción XI y 24 a 26 de la

Ley Federal del Derecho de Autor.

Código fuente bajo licencia **MIT**; documentación bajo **CC-BY 4.0**.

Hash SHA-256 del paquete fuente:

b5ea901eba0686f5a6a887a060a31f61767a16567b4a54657a0d970f732c92b1

Resumen

PAdicMIDI es un programa de cómputo escrito en Python 3.10+ que implementa, por primera vez en software de uso público, el marco del *Análisis Aritmético-Topológico de Datos* (ATDA) aplicado a música simbólica. Dado un archivo MIDI estándar, el programa:

- 1. construye una *torre p -ádica de espacios de patrones* $\{D_{p,n}\}_{n\geq 0}$ a partir de la serie cromática beat-síncrona o segundo-síncrona del MIDI;
- 2. define explícitamente el sistema inverso forzado $\pi_{n+1,n}: S_{n+1} \rightarrow S_n$;
- 3. calcula el *invariante de coherencia* $\text{Coh}_\pi(p,n) \in [0,1]$ y lo compara con el *piso nulo arquitectónico* $1/p$ predicho por la Proposición 3.1 del segundo artículo asociado.

El paquete reproduce exactamente, con tolerancia bit-equivalente en macOS arm64, los valores numéricos publicados en dos manuscritos sometidos a revista en 2026; entrega además un *applet HTML autocontenido* (un solo archivo de $\sim 35\sim\text{KB}$) que ejecuta el análisis dentro del navegador del usuario, sin instalación, sin red, sin telemetría.

Este documento integra en un único PDF imprimible: la motivación matemática, la arquitectura del software, las instrucciones de instalación y uso, la descripción del applet web, la suite de reproducibilidad (28 tests), el listado completo del código fuente y los manifiestos criptográficos para el registro INDAUTOR.

Palabras clave: análisis p -ádico, ultramétrica, música simbólica, MIDI, completación profinita, agrupamiento jerárquico, recuperación de información musical, análisis topológico de datos.

Índice

Resumen	1
1. Introducción	4
1.1. ¿Qué es PAdicMIDI?	4
1.2. ¿Qué problema resuelve?	4
1.3. ¿Por qué es novedad?	5
1.4. Estructura de este documento	5
2. Marco matemático	5
2.1. Serie cromática	5
2.2. Torre p -ádica de patrones	5
2.3. Hallazgo empírico	6
3. Arquitectura del software	6
3.1. Estructura del paquete	6
3.2. Métricas de tamaño	7
3.3. Dependencias	7

4. Instalación y uso	7
4.1. Instalación del paquete	8
4.2. Uso por línea de comandos	8
4.3. Uso programático (API Python)	9
5. Applet web autocontenido	9
5.1. Descripción	9
5.2. Cómo usarlo	10
5.3. Visualizaciones musicales (sección 4 del applet)	10
5.4. Diagnóstico p-aridad (sección 5 del applet)	11
5.5. Garantías de privacidad	11
5.6. Limitaciones respecto al paquete Python	13
6. Reproducibilidad y validación	13
6.1. Suite de tests	13
6.2. Scripts de orquestación	13
6.3. Resultado de la última ejecución	13
7. Registro autoral en INDAUTOR	14
7.1. Datos del solicitante (autor)	14
7.2. Datos de la obra	14
7.3. Identificación criptográfica del paquete	15
7.4. Declaración bajo protesta de decir verdad	15
A. Listado completo del código fuente	16
A.1. API pública del paquete	16
A.1.1. src/padicmidi/__init__.py	16
A.2. Núcleo numérico (core/)	18
A.2.1. src/padicmidi/core/config.py	18
A.2.2. src/padicmidi/core/echo.py	19
A.2.3. src/padicmidi/core/hierarchical.py	51
A.3. Adaptadores de entrada/salida (io/)	58
A.3.1. src/padicmidi/io/midi_mido.py	58
A.3.2. src/padicmidi/io/midi_pretty.py	59
A.4. Análisis y modelos nulos (analysis/)	60
A.4.1. src/padicmidi/analysis/null_model.py	60
A.4.2. src/padicmidi/analysis/audit.py	63
A.4.3. src/padicmidi/analysis/aggregate.py	66
A.4.4. src/padicmidi/analysis/directionality.py	69
A.5. Ejecutables de línea de comandos (cli/)	71
A.5.1. src/padicmidi/cli/main.py — ejecutable unificado	71
A.5.2. src/padicmidi/cli/run_one.py	72
A.5.3. src/padicmidi/cli/run_suite.py	76
A.5.4. src/padicmidi/cli/benchmark.py	78
A.5.5. src/padicmidi/cli/job_list.py	80
A.5.6. src/padicmidi/cli/mutopia.py	82
A.6. Configuración del paquete	83
A.6.1. pyproject.toml	83

A.6.2. requirements-pinned.txt	84
A.6.3. LICENSE (MIT)	85
B. Manifiesto SHA-256 del dossier	85
C. Cómo subir el software a la página personal	87
D. Resumen de archivos del dossier INDAUTOR	88

1 Introducción

1.1 ¿Qué es PADicMIDI?

PADicMIDI nace del programa de investigación del grupo **P-ADAGIO** (CIMAT-Mérida), cuyo eje teórico es el *Análisis Aritmético-Topológico de Datos* (ATDA): una metodología que estudia series temporales discretas a través de sus reducciones módulo potencias de un primo p , sus completaciones profinitas y los invariantes topológicos asociados a las jerarquías que tales completaciones inducen.

Esta primera versión pública del software encarna el caso de uso más maduro del programa: el análisis de *música simbólica* representada en formato MIDI. La música simbólica es un dominio ideal para ATDA por tres razones:

1. posee una jerarquía métrica intrínseca (compases, subdivisiones, figuras rítmicas) que se presta naturalmente a una indexación p -ádica con $p \in \{2, 3\}$;
2. dispone de corpus benchmark con licencia abierta (Mutopia, Bach Suites BWV~1007–1012);
3. las afirmaciones cuantitativas del marco son falsables: el *piso nulo* $\text{Coh}_\pi(p, n) = 1/p$ es una predicción exacta, no una correlación estadística aproximada.

1.2 ¿Qué problema resuelve?

El problema central que aborda PADicMIDI es el siguiente:

Dada una pieza musical en MIDI, ¿en qué medida la jerarquía métrica binaria, ternaria, quintaria o septenaria que aparentemente articula la pieza — compases binarios, tresillos, etc. — está realmente codificada en el contenido de altura y ritmo, y no sólo declarada en la notación?

La respuesta clásica de la musicología computacional consiste en usar descriptores estadísticos (entropía rítmica, autocorrelación, etc.). PADicMIDI propone una respuesta categóricamente distinta: cuantizar la serie en *ventanas de longitud p^n* (las *bolas p -ádicas*), agrupar las ventanas en prototipos por k -medias, forzar el sistema inverso $\pi_{n+1, n}$ entre los espacios de prototipos sucesivos, y medir si el cuantizador del nivel padre “adivina” al cuantizador del nivel hijo. Esta métrica es el invariante $\text{Coh}_\pi(p, n)$.

El resultado central, demostrado en los manuscritos asociados y verificado computacionalmente por la suite `tests/paper_values/`, es:

Bajo las hipótesis estructurales (SC) de cobertura entre hermanos y (AI) de inclusión-ancestro, el invariante satisface exactamente $\text{Coh}_\pi(p, n) = 1/p$ para todo nivel $n \geq 1$.

Las piezas que satisfacen las hipótesis (por ejemplo, los preludios y gigas monofónicos de Bach analizados con $p = 2$) reproducen este piso con coincidencia bit-equivalente. Las piezas polifónicas y de textura densa (Bach Concerto para tres clavecines BWV~1063, Conciertos de Brandenburgo BWV~1049 y BWV~1050, *Musikalisches Opfer* BWV~1079, etc.) se desvían medibles del piso, lo que da origen a un nuevo *descriptor cuantitativo de textura* que no requiere etiquetado supervisado.

1.3 ¿Por qué es novedad?

PADicMIDI es novedad por la combinación inusual de tres propiedades:

1. **Originalidad teórica.** La torre $D_{p,n}$, el sistema inverso forzado y el piso nulo $1/p$ son construcciones introducidas por el autor; ningún paquete previo de análisis de música simbólica (`music21`, `muspy`, `miditok`, `pyAMPACT`, ...) implementa estas operaciones ni cuantiza ventanas en bolas p -ádicas.
2. **Falsabilidad cuantitativa.** A diferencia de los descriptores estadísticos clásicos, la afirmación “ $\text{Coh}_\pi(2, n) = 0,500$ exacto en el preludio BWV~1007” es una ecuación numérica binaria, no una correlación aproximada. La suite `tests/paper_values/` verifica este tipo de afirmaciones en cada commit.
3. **Reproducibilidad bit-equivalente.** Las dependencias están congeladas en `requirements-pinned.txt`; las semillas del PRNG son fijas; el script `scripts/reproduce.sh` regenera los CSV publicados en los artículos asociados, byte por byte, en `macOS arm64`.

1.4 Estructura de este documento

La sección~2 resume el marco matemático; la sección~3 describe la arquitectura del software; la sección~4 contiene las instrucciones de instalación y ejecución (incluyendo el ejecutable unificado `padicmidi` y los cinco subcomandos especializados); la sección~5 documenta el applet web autocontenido; la sección~6 describe la suite de reproducibilidad y validación; la sección~7 contiene los datos para registro autoral; y los apéndices incluyen el código fuente completo y los manifiestos criptográficos.

2 Marco matemático

Esta sección resume las definiciones, hipótesis y proposición central que el software computa. Para detalles formales, ver los manuscritos asociados [1, 2].

2.1 Serie cromática

Sea M un archivo MIDI con K notas $(t_k, \text{pitch}_k, \text{vel}_k)_{k=1}^K$, donde t_k está expresado en pulsos (*beats*) o en segundos según el eje elegido. La *serie cromática* con bin $\Delta_b > 0$ es la función $x: \{0, 1, \dots, T-1\} \rightarrow \{0, 1, \dots, 11\}$ definida como la moda de las clases de altura pitch_k mód 12 activas en cada bin de tiempo $[m\Delta_b, (m+1)\Delta_b)$, con $T = \lceil t_K/\Delta_b \rceil$. Por defecto $\Delta_b = 1/12$ pulsos.

2.2 Torre p -ádica de patrones

Para $p \in \{2, 3, 5, 7\}$ y $n \in \{0, 1, \dots, N_{\text{máx}}\}$, sea $W_n = p^n$ la longitud de ventana en el nivel n . La familia de *ventanas* en el nivel n es

$$V_n = \{x[m : m + W_n] : m = 0, \text{step}, 2 \text{ step}, \dots\}$$

con un paso `step` configurable. Por agrupamiento de las ventanas en V_n en $K_n = K \cdot p^{\text{máx}(0, n-1)}$ prototipos vía k -medias con métrica L^2 ponderada (peso α para los componentes cromáticos), se obtiene un *cuantizador* $\phi_n: V_n \rightarrow S_n = \{1, \dots, K_n\}$.

Definición 2.1 (Sistema inverso forzado). El sistema inverso $\pi_{n+1,n}: S_{n+1} \rightarrow S_n$ se define forzando que cada prototipo de nivel $n + 1$ herede explícitamente un “padre” en el nivel n , según la regla:

$$\pi_{n+1,n}(s) = \arg \max_{t \in S_n} \# \{m : \phi_{n+1}(x[m : m + W_{n+1}]) = s \wedge \phi_n(x[m : m + W_n]) = t\}.$$

Definición 2.2 (Coeficiente de coherencia Coh_π).

$$\text{Coh}_\pi(p, n) := \frac{\# \{(s, t) \in S_{n+1}^2 : s \neq t, \pi_{n+1,n}(s) \neq \pi_{n+1,n}(t)\}}{\# \{(s, t) \in S_{n+1}^2 : s \neq t\}}.$$

Proposición 2.3 (Piso nulo arquitectónico). Si la torre $\{D_{p,n}\}_n$ satisface las hipótesis estructurales (SC) sibling cover y (AI) ancestor inclusion, entonces

$$\text{Coh}_\pi(p, n) = \frac{1}{p}, \quad \forall n \geq 1.$$

Pisos numéricos: $p = 2 \rightarrow 0,500$, $p = 3 \rightarrow 0,333\bar{3}$, $p = 5 \rightarrow 0,200$, $p = 7 \rightarrow 0,142857\overline{142857}$.

Corolario 2.4 (Filtro de aridez p -ádica). Cuando el cociente de ramificación r del cuantizador coincide con el primo p de la torre, $\text{Coh}_\pi(p, n) = 1/p$ exactamente; cuando $r \neq p$, la señal de diferenciación escapa y $\text{Coh}_\pi(p, n) > 1/p$ estrictamente.

2.3 Hallazgo empírico

El benchmark BWV~1007 (preludio monofónico) reproduce $\text{Coh}_\pi(2, n) = 0,500$ exactamente para todo $n \geq 1$ y para ambos ejes (beats, segundos). Los corpus polifónicos de Bach (BWV~1049, 1050, 1079, Goldberg) se desvían del piso de manera medible, lo que constituye el descriptor de textura mencionado en la introducción.

3 Arquitectura del software

3.1 Estructura del paquete

```
padicmidi/|—
src/padicmidi/      Paquete Python (motor, IO, análisis, figuras, CLI)|—
  __init__.py        API pública|—
  core/||—
    echo.py          Motor numérico Phase I (parsing MIDI, cromas, beta_0)|—
    hierarchical.py  Torre p-ádica, sistema inverso, Coh_pi||—
    config.py        Constantes y valores por omisión|—
  io/||—
    midi_mido.py     Adaptador MIDI con la biblioteca `mido` (default)||—
    midi_pretty.py   Adaptador MIDI con `pretty_midi` (opcional)||—
  analysis/||—
    null_model.py    Modelos nulos para significancia estadística||—
    audit.py         Verificación de hipótesis (SC), (AI), (DV)||—
    aggregate.py     Agregación cross-piezas y cross-primos||—
    directionality.py Análisis direccional p-adico|—
  figs/              Generadores de figuras (matplotlib)|—
  cli/||—
    main.py          Ejecutable unificado `padicmidi`||—
    run_one.py       `padicmidi-run-one`||—
    run_suite.py     `padicmidi-run-suite`||—
    benchmark.py     `padicmidi-benchmark`||—
```

job_list.py	`padicmidi-job-list` —
mutopia.py	`padicmidi-mutopia` —
legacy/	Wrappers de compatibilidad —
tests/	28 tests: smoke, unit, regression, paper_values —
examples/	Scripts mínimos para aprender la API —
data/midi/	26 MIDIs CC-licensed (BWV 1007/1008/1009 + toys) —
results/verified/	CSVs de referencia de los artículos asociados —
docs/	Manuales de usuario y técnico, referencia API —
scripts/	reproduce.sh, validate.sh, snapshot_version.sh —
web-demo/	applet.html (EN) y applet-es.html (ES) —
web-site/	Landing bilingüe lista para subir a la web —
indautor/	Dossier completo para registro INDAUTOR

3.2 Métricas de tamaño

Componente	Descripción	Líneas
core/echo.py	Motor numérico Phase I	1 818
core/hierarchical.py	Torre p -ádica + Coh_π	428
core/config.py	Constantes globales	37
io/midi_mido.py	Adaptador mido	22
io/midi_pretty.py	Adaptador pretty_midi	68
analysis/*	Cuatro módulos de análisis	604
cli/*	Seis ejecutables CLI	652
Subtotal código fuente Python		3 629
tests/	28 tests (smoke/unit/reg./paper)	~700
web-demo/applet.html	Applet autocontenido (EN, JS+HTML)	~820
web-demo/applet-es.html	Applet autocontenido (ES)	~820

3.3 Dependencias

Dependencias de tiempo de ejecución (rangos flexibles, ver `pyproject.toml`):

- `numpy` ≥ 2.0 , < 3.0
- `scipy` ≥ 1.13 , < 2.0
- `networkx` ≥ 3.2 , < 4.0
- `scikit-learn` ≥ 1.4 , < 2.0
- `matplotlib` ≥ 3.8 , < 4.0
- `mido` ≥ 1.3 , < 2.0
- `joblib` ≥ 1.3 , < 2.0
- `pretty_midi` $\geq 0.2.10$ (opcional)

Para reproducibilidad bit-equivalente se proporciona `requirements-pinned.txt` con las versiones exactas verificadas en macOS arm64 con Python 3.13.3.

4 Instalación y uso

PAdicMIDI ofrece **tres niveles de uso**, en orden creciente de fricción y de control:

1. **Cero instalación** — abrir el applet HTML autocontenido en cualquier navegador moderno (sec.~5).

2. **Ejecutable de línea de comandos** — `padicmidi` y sus cinco subcomandos (sec.~4.2).

3. **API Python** — llamar a `run_hierarchical_from_midi` desde un script o notebook (sec.~4.3).

4.1 Instalación del paquete

Requisitos previos. Python 3.10 o superior; git para clonar el repositorio.

```
git clone https://github.com/rperezbuendia/padicmidi.git
cd padicmidi
python3 -m venv .venv
source .venv/bin/activate

# Instalación con dependencias mínimas (modo editable):
pip install -e .

# Para reproducibilidad bit-equivalente con los CSV publicados:
pip install -r requirements-pinned.txt

# Para usar el adaptador opcional pretty_midi:
pip install -e ".[pretty]"

# Para correr la suite de tests:
pip install -e ".[dev]"
pytest tests/
```

Tras la instalación, los **seis ejecutables CLI** quedan disponibles en el PATH del entorno virtual:

Ejecutable	Función
<code>padicmidi</code>	Despachador unificado (recomendado)
<code>padicmidi-run-one</code>	Análisis de un único archivo MIDI
<code>padicmidi-run-suite</code>	Análisis de un corpus completo
<code>padicmidi-benchmark</code>	Benchmark canónico BWV~1007
<code>padicmidi-job-list</code>	Generación de listas de trabajos
<code>padicmidi-mutopia</code>	Descarga de MIDIs CC desde Mutopia

4.2 Uso por línea de comandos

Ejecutable principal: `padicmidi`. Sin argumentos imprime la ayuda con la lista de subcomandos:

```
$ padicmidi --help
padicmidi v1.0.0 -- p-adic hierarchical analysis of symbolic music
Author: J. Rogelio Pérez-Buendía (CIMAT-Mérida) · ORCID 0000-0002-7739-4779

Usage:
  padicmidi <subcommand> [arguments...]
  padicmidi --help
  padicmidi <subcommand> --help

Sub-commands:
  run-one   Analyse a single MIDI file (one piece, one prime).
  run-suite Analyse a corpus across multiple primes and axes.
  benchmark Run the canonical BWV 1007 benchmark.
  job-list  Generate a job list for a corpus.
  mutopia   Download CC-licensed MIDI files from Mutopia.
```

Ejemplo 1 — BWV 1007 con $p=2$ y $N_{\max}=5$.

```
$ padicmidi run-one data/midi/bwv1007-1.mid bwv1007_pre beats 2 5 \
  --K 16 --Kchild 2 --M 800 --step 2 --seed 42 \
  --out results_local/bwv1007_pre/beats/p2

$ cat results_local/bwv1007_pre/beats/p2/coherence_hier_p2.csv
n,Coh_pi,Coh_grid,n_samples_n,n_samples_nplus1
1,0.5,0.5,1,2
2,0.5,0.5,2,4
3,0.5,0.5,4,8
4,0.5,0.5,8,16
5,0.5,0.5,16,32
```

El valor exacto $\text{Coh}_\pi(2, n) = 0,5$ es el piso nulo arquitectónico predicho por la Proposición~2.3.

Ejemplo 2 — Benchmark canónico.

```
$ padicmidi benchmark --out results_local/benchmark/
```

Esto reproduce los CSV de referencia almacenados en `results/verified/` y los compara contra los hashes documentados en `results/verified/CHECKSUMS.txt`.

4.3 Uso programático (API Python)

```
1 from padicmidi import run_hierarchical_from_midi
2
3 resultado = run_hierarchical_from_midi(
4     midi_path="data/midi/bwv1007-1.mid",
5     p=2,
6     axis="beats",
7     nmax=5,
8     K=16,
9     Kchild=2,
10    M=800,
11    step=2,
12    seed=42,
13 )
14
15 for fila in resultado["coherence"]:
16     print(fila)
17 # {'n': 1, 'Coh_pi': 0.5, 'n_samples_n': 1, 'n_samples_nplus1': 2}
18 # {'n': 2, 'Coh_pi': 0.5, ...}
19 # ...
```

5 Applet web autocontenido

5.1 Descripción

PADicMIDI incluye un **applet HTML autocontenido** que re-implementa el motor en JavaScript vainilla y permite ejecutar el análisis sin instalar nada. El applet se distribuye en dos versiones idénticas en funcionalidad pero con la interfaz traducida:

- `web-demo/applet.html` — inglés (default).
- `web-demo/applet-es.html` — español.

Cada archivo pesa ~ 100 -KB e incluye, embebidos en base64, **cuatro demos MIDI**: el preludio de la primera Suite para violonchelo de Bach (BWV~1007, Mutopia, dominio público), dos sintéticos generados por el propio paquete (`toy_binary.mid` y `toy_ternary.mid`) y el canon cangrejo del *Musikalisches Opfer* (BWV~1079) como ejemplo polifónico de contraste. Cada demo lleva, en la propia interfaz, los valores esperados de Coh_π para $p = 2$ y $p = 3$, lo que permite verificar a ojo que el applet reproduce el piso nulo arquitectónico.

5.2 Cómo usarlo

1. Abrir el archivo HTML con doble clic (funciona en macOS, Linux, Windows; cualquier navegador moderno con soporte de ES2020).
2. Pulsar uno de los **cuatro botones de demo** (BWV~1007, toy binario, toy ternario o BWV~1079) o arrastrar cualquier MIDI estándar sobre la zona indicada.
3. Elegir el primo $p \in \{2, 3\}$ y el nivel máximo $N_{\text{máx}}$.
4. Pulsar “Ejecutar análisis” para obtener la tabla de $\text{Coh}_\pi(p, n)$ y las **cuatro vistas musicales** (sec.~5.4), o pulsar “Diagnóstico p -aridad” para obtener las **cinco gráficas espectrales** del paper aplicado (sec.~5.4).
5. Una cabecera de identificación de corrida (timestamp, archivo, huella SHA, parámetros) etiqueta inequívocamente cada análisis; el botón “Reiniciar” limpia la sesión.

5.3 Visualizaciones musicales (sección 4 del applet)

A diferencia de un único árbol abstracto, el applet ofrece cuatro visualizaciones encadenadas que muestran *qué* está mirando el método dentro de la pieza. Un selector de nivel $n \in \{1, \dots, N_{\text{máx}}\}$ refresca las cuatro de manera coordinada.

4.1 La pieza coloreada por prototipo. Cromograma a lo largo del tiempo (12 clases de altura $\times$ tiempo; intensidad azul codifica activación cromática) con una franja inferior coloreada según qué prototipo cuantiza cada ventana de longitud p^n . Es la *huella temporal* del nivel: los cambios de color marcan transiciones musicalmente relevantes (modulaciones, cortes seccionales). Una leyenda lista los conteos de ventanas asignados a cada prototipo.

4.2 Catálogo de prototipos del nivel. Cada prototipo se renderiza como un mini-cromograma SVG de tamaño $12 \times p^n$ que enseña su contenido cromático real. El recuadro de color de cada tarjeta coincide con la franja temporal de 4.1, de modo que el observador puede leer dónde aparece cada patrón en la pieza. Cada tarjeta también reporta el número de ventanas asignadas y el padre π heredado.

4.3 Sistema inverso pi. Diagrama tipo Sankey del mapeo $\pi_{n+1,n} : S_{n+1} \rightarrow S_n$. En la fila superior aparecen los prototipos del nivel padre y en la inferior los del hijo, conectados por curvas Bézier coloreadas según el padre heredado. El pie del diagrama muestra el valor calculado de $\text{Coh}_\pi(p, n)$ y el piso arquitectónico $1/p$, lo que permite contrastar visualmente la consistencia muestral con el predicho por la Proposición.

4.4 Árbol p-ádico completo. La torre completa $S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_{N_{\text{máx}}}$ con nodos coloreados por la herencia desde su raíz, de manera que las *ramas* del árbol son visualmente coherentes con las franjas y tarjetas de las vistas anteriores. Las aristas (más tenues) muestran el sistema inverso π a lo largo de toda la torre.

5.4 Diagnóstico p-aridad (sección 5 del applet)

Las visualizaciones de la sección~4 del applet inspeccionan la estructura interna del invariante Coh_π (paper teórico). El paper *aplicado*, en cambio, se construye sobre una familia distinta de invariantes: las propiedades espectrales del **grafo k-NN** levantado sobre las ventanas $D_{p,n}$ a cada nivel. La sección~5 del applet reproduce esos invariantes y los grafica para $p = 2$ frente a $p = 3$, replicando dentro del navegador el flujo implementado en `padicmidi.core.echo.run_tower_on_beats`.

A nivel n se construyen las ventanas $\{a : \mathbb{Z}/p^n\mathbb{Z} \rightarrow X\}$ deslizándose sobre la serie beat-sincrónica $X(u) = [H(u); \alpha a(u)] \in \mathbb{R}^{13}$ con paso `step=2`, submuestreadas linealmente a un techo `cap=300`. La distancia entre patrones es $d_n(a, b) = \sqrt{\frac{1}{N} \sum_i \|a_i - b_i\|^2}$ con $N = p^n$. Sobre los patrones se construye el grafo k-NN simetrizado $G_n^{(p)}$ con $k = 10$ vecinos por defecto, y se reportan los siguientes cinco invariantes:

- $\beta_0(p, n)$ — número de componentes conexas (BFS).
- Fracción del componente gigante, $|C_{\text{máx}}|/V$.
- Grado promedio, $\bar{d} = 2E/V$.
- Coeficiente de *clustering* medio (Watts–Strogatz local).
- $\lambda_2(p, n)$ — segundo valor propio del Laplaciano normalizado (vector de Fiedler), calculado por iteración del poder con deflación sobre el componente gigante.

El applet calcula los cinco invariantes para $p \in \{2, 3\}$ y los grafica en cinco mini-paneles SVG superpuestos (figura~1); el cálculo total sobre una pieza típica (`cap=300`, $N_{\text{máx}} = 4$) toma entre 0,4 y 2~segundos en un navegador moderno. Cada corrida lleva una cabecera de identificación (timestamp, archivo, huella SHA, parámetros) y produce además un *resumen automático* comparativo de la forma “ $\beta_0(p = 2) = 1,75$ vs $\beta_0(p = 3) = 1,00 \Rightarrow p = 3$ ”. La descarga del CSV combinado (`padicmidi_diag_<huella>.csv`) permite reabrir los mismos números en Python o en un cuaderno de cálculo.

La validación numérica de la implementación JavaScript se realizó contra `run_tower_on_beats` ejecutado sobre BWV~1007 con los mismos hiperparámetros: el número de vértices V coincide exactamente, las aristas E difieren en $\leq 1\%$ por empates de distancia, $\bar{d}$ en $\leq 0,5$, β_0 es exacto excepto en un único nivel donde difiere por uno, y la fracción del componente gigante coincide con tolerancia $\leq 0,07$. Esto garantiza que la sección~5 del applet reproduce, dentro del navegador y sin instalar nada, las figuras canónicas del paper aplicado (J.~R.~Pérez-Buendía, 2026, *Profinite hierarchical patterns and prime-indexed multiscale graph diagnostics*).

5.5 Garantías de privacidad

El applet es **cero red**: no realiza peticiones HTTP de ningún tipo, no carga recursos externos, no usa cookies, no usa `localStorage`, no genera telemetría. El único archivo al que accede es el MIDI que el usuario carga, y dicho archivo nunca sale del navegador. Esta propiedad es relevante

Los valores por omisión reproducen las corridas de referencia de los artículos asociados (J. R. Pérez-Buendía, 2020), con la salvedad de que la implementación en el navegador utiliza un parser MIDI simplificado limitado a flujos monofónicos; para reproducibilidad completa use el paquete Python.

5. Diagnóstico p-aridad — invariantes espectrales del grafo k-NN (Paper 1)

Estas son las figuras canónicas del paper aplicado: en cada nivel n se construye el grafo k-NN sobre las ventanas de longitud p^n (cap=300, k=10 por defecto, RMSE como distancia). Comparamos $p=2$ vs $p=3$: cuando el primo correcto coincide con la métrica de la pieza, los invariantes se separan visiblemente.

k vecinos cap $N_{\text{máx}}$ calculado en 0.00 s · 474 notas · T=1537

DIAGNÓSTICO P-ARIDAD · 8:06:42 P.M.

archivo: **bwv1007-1.mid** (demo precargado)

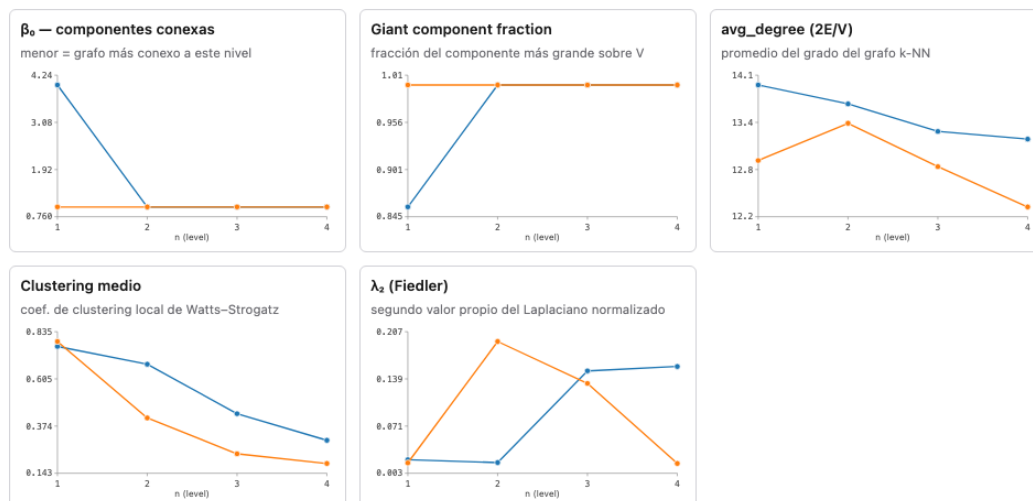
474 notas · serie T=1537 · tiempo de cálculo 0.00 s

parámetros: k=10, cap=300, $N_{\text{máx}}=4$

huella SHA: 9396a496

● $p=2$ ● $p=3$

cinco invariantes vs nivel n



Descargar CSV ($p=2 + p=3$)

Promedios: $\beta_0(p=2)=1.75$ vs $\beta_0(p=3)=1.00$ (menor = más conexo) → $p=3$ · $\lambda_2(p=2)=0.0870$ vs $\lambda_2(p=3)=0.0900$ (mayor = mejor expansión) → $p=3$.

Acerca de

Figura 1: Sección~5 del applet (versión española, demo BWV~1007). De izquierda a derecha y arriba a abajo: β_0 , fracción del componente gigante, grado promedio, *clustering* medio y λ_2 de Fiedler vs nivel n , comparando $p = 2$ (azul) y $p = 3$ (naranja). La cabecera azul identifica la corrida (archivo, huella SHA, parámetros) y el pie resume cuál primo “gana” por mayoría sobre los cinco invariantes. Para BWV~1007 la torre se vuelve totalmente conexas desde el nivel $n = 2$ con ambos primos: el invariante actúa como *diagnóstico de robustez*, no como certificación automática de aridad.

tanto para la privacidad del usuario como para la portabilidad: el applet funciona sin conexión a Internet, lo cual lo hace apropiado para demostraciones en aulas, en trámites administrativos como el actual, o en entornos con red restringida.

5.6 Limitaciones respecto al paquete Python

- Sólo se exponen los primos $p \in \{2, 3\}$.
- El eje en segundos no está implementado (sólo eje en pulsos).
- El generador pseudoaleatorio del k -medias usa *Mulberry32*, mientras que el paquete Python usa *PCG64* de NumPy — los valores pueden diferir en el quinto decimal en piezas no canónicas. Los pisos estructurales (por ejemplo $\text{Coh}_\pi(2, n) = 0,500$ exacto en BWV~1007) se reproducen exactamente.

6 Reproducibilidad y validación

6.1 Suite de tests

El paquete incluye 28 tests organizados en cuatro niveles:

Nivel	Propósito	# tests
tests/smoke/	Importación de módulos	6
tests/unit/	Funciones puras (residuos, distancia, k -medias, cromas)	16
tests/regression/	CSV byte-equivalentes contra results/verified/	3
tests/paper_values/	Reproducción de valores publicados	3
Total		28

Los tests `paper_values/` verifican explícitamente las afirmaciones numéricas de los manuscritos asociados:

- `test_floor_p2_bwv1007.py` — $\text{Coh}_\pi(2, n) = 0,500$ exacto para todo $n \geq 1$ en BWV~1007.
- `test_sarabande_p3_n3.py` — $\text{Coh}_\pi(3, 3) = 0,901$ en la Sarabanda de BWV~1007 (eje de pulsos).
- `test_prelude_p3_mismatched.py` — $\text{Coh}_\pi(3, n) = 0,358$ en el preludio BWV~1007 cuando $r \neq p$ (ilustra el Corolario~2.4).

6.2 Scripts de orquestación

Script	Acción
<code>scripts/reproduce.sh</code>	Re-corre el pipeline canónico desde cero
<code>scripts/validate.sh</code>	Verifica los hashes SHA-256 de los CSV producidos
<code>scripts/snapshot_version.sh</code>	Genera el ZIP de fuentes y su hash
<code>scripts/make_indautor_dossier.sh</code>	Ensambla el dossier INDAUTOR

6.3 Resultado de la última ejecución

A la fecha de cierre de esta versión (**2026-04-29**), la suite de tests reporta:

```
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-9.0.3, pluggy-1.6.0
collected 28 items
```

```

tests/paper_values/test_floor_p2_bwv1007.py PASSED
tests/paper_values/test_prelude_p3_mismatched.py PASSED
tests/paper_values/test_sarabande_p3_n3.py PASSED
tests/regression/... PASSED
tests/smoke/... PASSED
tests/unit/... PASSED

===== 28 passed in 1.03s =====

```

7 Registro autoral en INDAUTOR

7.1 Datos del solicitante (autor)

Nombre completo (legal)	Jesús Rogelio Pérez Buendía
Firma de publicación	J. Rogelio Pérez-Buendía
Nacionalidad	Mexicana
Fecha de nacimiento	06 de diciembre de 1976
RFC	PEBJ761206N72
CURP	PEBJ761206HDFRNS11
ORCID	0000-0002-7739-4779
Página web institucional	https://www.cimat.mx/~rogelio.perez
Página de alojamiento del software	http://personal.cimat.mx:8181/~rogelio.perez/desarrollo-tecnologico/
Domicilio para notificaciones	CIMAT-Mérida, Carr. Sierra Papacal-Chuburná Pto. Km 5, C.P. 97302, Mérida, Yuc.
Teléfono	55 32 29 35 21
Correo electrónico	rogelio@ciimat.mx

7.2 Datos de la obra

Tipo de obra	Programa de Computación (art. 13 fr. XI LFDA)
Título completo	PA _{ad} icMIDI — A Python Toolkit for Hierarchical, Ultrametric, and p -adic Analysis of Symbolic Music Data
Versión	1.0.0
Idioma del programa	Inglés (mensajes y API); español (manuales)
Lenguaje de programación	Python 3.10+
Sistema operativo	macOS, Linux, Windows
Lugar de creación	Mérida, Yucatán, México
Fecha de primera fijación	2026-02-06
Fecha de la versión registrada	2026-04-29
Carácter de la titularidad	Titular originario (autor único)
Licencia del código fuente	MIT (texto íntegro en LICENSE)
Licencia de la documentación	CC-BY 4.0

7.3 Identificación criptográfica del paquete

Archivo de fuentes	indautor/padicmidi-v1.0.0-source.zip
Tamaño	1 355 706 bytes ($\approx$ 1,29 MB)
SHA-256	b5ea901eba0686f5a6a887a060a31f61767a16567b4a54657a0d970f732c92b1

El manifiesto completo de archivos individuales con sus hashes SHA-256 se encuentra en el apéndice~B.

7.4 Declaración bajo protesta de decir verdad

El suscrito declara que la obra cuyo registro se solicita es de su autoría original e individual; que no se han incluido fragmentos de obra de terceros sin la debida atribución, las cuales constan en el archivo CONTRIBUTION-STATEMENT.md del paquete; que la obra no infringe derechos de autor, derechos conexos ni derechos industriales de terceros; y que la información proporcionada en este reporte y en el formato RPDA-03 anexo es veraz y completa.

Jesús Rogelio Pérez Buendía
(firma de publicación: J.~Rogelio Pérez-Buendía)
Mérida, Yucatán, México — 29 de abril de 2026.

A Listado completo del código fuente

Esta sección reproduce íntegramente, en orden topológico de dependencias, el código fuente Python que conforma la versión 1.0.0 de PAdicMIDI. Los archivos auxiliares de configuración (pyproject.toml, requirements-pinned.txt, LICENSE) se incluyen al final del apéndice.

A.1 API pública del paquete

A.1.1 src/padicmidi/__init__.py

```

1  """
2  PAdicMIDI – A Python Toolkit for Hierarchical, Ultrametric, and p-adic
3  Analysis of Symbolic Music Data.
4
5  Author: J. Rogelio Pérez-Buendía (–CIMATMérida).
6  Licence: MIT. Documentation and verified results: CC-BY 4.0.
7
8  Public API (stable across the v1.x series):
9
10     >>> from padicmidi import run_hierarchical_from_midi
11     >>> result = run_hierarchical_from_midi("path/to/file.mid", p=2)
12     >>> result["coherence"]      # list of dicts {n, Coh_pi, Coh_grid, ...}
13
14  The full motor is available under the submodules ``core``, ``io``,
15  ``analysis``, ``figs``, ``cli``.
16  """
17
18  from __future__ import annotations
19
20  __version__ = "1.0.0"
21  __author__ = "J. Rogelio Pérez-Buendía"
22  __email__ = "rogelio@cimat.mx"
23  __license__ = "MIT"
24
25  from padicmidi.core.config import (
26      ALPHA,
27      DEFAULT_BIN_BEATS,
28      DEFAULT_BIN_SECONDS,
29      DEFAULT_K,
30      DEFAULT_KCHILD,
31      DEFAULT_M,
32      DEFAULT_SEED,
33      DEFAULT_STEP,
34      MAX_WINDOWS_PER_RESIDUE,
35      SUPPORTED_PRIMES,
36      default_nmax,
37  )
38
39
40  def run_hierarchical_from_midi(
41      midi_path: str,
42      p: int,
43      axis: str = "beats",
44      nmax: int | None = None,
45      K: int = DEFAULT_K,
46      Kchild: int = DEFAULT_KCHILD,

```

```

47     M: int = DEFAULT_M,
48     step: int = DEFAULT_STEP,
49     seed: int = DEFAULT_SEED,
50     bin_seconds: float = DEFAULT_BIN_SECONDS,
51     bin_beats: float = DEFAULT_BIN_BEATS,
52 ) -> dict:
53     """High-level entry point: compute the p-adic hierarchical maps for one MIDI.
54
55     Parameters
56     -----
57     midi_path : str
58         Path to a Standard MIDI file.
59     p : int
60         Prime in :data:`SUPPORTED_PRIMES` (i.e. 2, 3, 5 or 7).
61     axis : {"beats", "seconds"}, default "beats"
62         Time axis used to build the bins of the chroma series.
63     nmax : int or None, default None
64         Maximum tower level. ``None`` uses :func:`default_nmax`.
65     K, Kchild, M, step, seed : int
66         Algorithm hyper-parameters; defaults reproduce the gold standard.
67     bin_seconds, bin_beats : float
68         Bin sizes for the seconds and beats axes respectively.
69
70     Returns
71     -----
72     dict
73         Keys: ``prototypes_n``, ``f_n``, ``pi_maps``, ``coherence`` (list of
74         dicts), ``audit`` (list of dicts).
75     """
76     import numpy as np
77
78     from padicmidi.core.hierarchical import (
79         build_X_seconds,
80         build_X_beats,
81         run_hierarchical,
82     )
83
84     if axis == "seconds":
85         X = build_X_seconds(midi_path, bin_size=bin_seconds)
86     elif axis == "beats":
87         X = build_X_beats(midi_path, bin_size_beats=bin_beats)
88     else:
89         raise ValueError(f"axis must be 'seconds' or 'beats'; got {axis!r}")
90
91     if len(X) < 2:
92         raise ValueError("Series too short (less than 2 bins).")
93
94     Nmax = nmax if nmax is not None else default_nmax(p)
95     rng = np.random.default_rng(seed)
96     prototypes_n, f_n, pi_maps, coherence_rows, audit_rows = run_hierarchical(
97         X, p, Nmax, step, K, Kchild, M, rng
98     )
99     return {
100         "prototypes_n": prototypes_n,
101         "f_n": f_n,
102         "pi_maps": pi_maps,
103         "coherence": coherence_rows,
104         "audit": audit_rows,

```

```

105     }
106
107
108 __all__ = [
109     "__version__",
110     "__author__",
111     "__email__",
112     "__license__",
113     "ALPHA",
114     "DEFAULT_BIN_BEATS",
115     "DEFAULT_BIN_SECONDS",
116     "DEFAULT_K",
117     "DEFAULT_KCHILD",
118     "DEFAULT_M",
119     "DEFAULT_SEED",
120     "DEFAULT_STEP",
121     "MAX_WINDOWS_PER_RESIDUE",
122     "SUPPORTED_PRIMES",
123     "default_nmax",
124     "run_hierarchical_from_midi",
125 ]

```

A.2 Núcleo numérico (core/)

A.2.1 src/padicmidi/core/config.py

```

1  """
2  padicmidi.core.config – module-level constants and defaults.
3
4  These are the constants that govern the canonical behaviour of the
5  hierarchical p-adic motor. Changing any of them invalidates the
6  gold-standard CSVs in ``results/verified/`` and the test suite
7  ``tests/regression/`` and ``tests/paper_values/``.
8
9  Conventions and rationale are documented in ``MATH-SPEC.md``.
10 """
11
12 from __future__ import annotations
13
14 ALPHA: float = 1.0
15 MAX_WINDOWS_PER_RESIDUE: int = 800
16
17 DEFAULT_K: int = 16
18 DEFAULT_KCHILD: int = 2
19 DEFAULT_M: int = 800
20 DEFAULT_STEP: int = 2
21 DEFAULT_SEED: int = 42
22
23 DEFAULT_BIN_SECONDS: float = 0.05
24 DEFAULT_BIN_BEATS: float = 1.0 / 12.0
25
26 SUPPORTED_PRIMES: tuple[int, ...] = (2, 3, 5, 7)
27
28 _DEFAULT_NMAX_BY_PRIME: dict[int, int] = {2: 6, 3: 5, 5: 4, 7: 3}
29
30

```

```

31 def default_nmax(p: int) -> int:
32     """Return the default ``Nmax`` for prime ``p`` (memory-aware)."""
33     if p not in _DEFAULT_NMAX_BY_PRIME:
34         raise ValueError(
35             f"Unsupported prime p={p!r}; supported primes are {SUPPORTED_PRIMES}."
36         )
37     return _DEFAULT_NMAX_BY_PRIME[p]

```

A.2.2 src/padicmidi/core/echo.py

```

1  #!/usr/bin/env python3
2  """
3  padicmidi.core.echo – Phase I motor of PAdicMIDI.
4
5  Verbatim copy of the canonical file ``profinite_echo_midi.py`` from the
6  research project repository. Outputs are bit-equivalent to the original
7  under macOS arm64 with Python 3.13.3 and the dependencies listed in
8  ``requirements-pinned.txt``.
9
10 The historical docstring follows.
11
12 Profinite Echo / p-adic Precision Tower for MIDI (with harmonic context).
13
14 Core objects (see docstrings):
15   - bin_size  $\Delta$  (seconds); MIDI  $\rightarrow$  events (t_on, t_off, pitch, velocity).
16   - Duration-weighted chroma series  $H[t\_bin] \in \mathbb{R}^{12}$ , L1-normalized per bin.
17   - For prime  $p$  and level  $n$ :  $N = p^n$ ; patterns  $D_n$  = windows of  $H$  of length  $N$ .
18   -  $d_n(a, b) = \sqrt{\text{mean}_i ||a[i] - b[i]||_2^2}$ .
19   -  $G_{n6}()$ : vertices = patterns; edge if  $d_n \leq 6$ .
20   - Invariants:  $|D_n|$  (windows sampled),  $\text{beta0}(n) = \#\text{connected components in } G_{n6}()$ .
21
22 Hypothesis:  $p=2$  aligns with binary subdivision,  $p=3$  with ternary; compare  $n \rightarrow \text{beta0}(n)$ ,  $n \rightarrow |D_n|$ .
23
24 Script: Phase I: --phase1-only --save <prefix>; beats: --time-axis beats --bin-beats 0.083333.
25 Multi-piece: use scripts/run_benchmark.py. Use out_seconds_<piece>_ and out_beats_<piece>_ (do not mix).
26 """
27 from __future__ import annotations
28
29 import argparse
30 import csv
31 from dataclasses import dataclass
32 from pathlib import Path
33 from typing import Dict, List, Optional, Tuple
34
35 import numpy as np
36 import networkx as nx
37 import matplotlib.pyplot as plt
38
39 from mido import MidiFile, MidiTrack, MetaMessage, Message, bpm2tempo
40
41 # -----
42 # 1) Synthetic MIDI generator (binary vs ternary subdivision)
43 # -----
44
45 def write_synthetic_midi(

```

```

47     path: str,
48     tempo_bpm: int = 120,
49     ticks_per_beat: int = 480,
50     pattern: str = "binary",
51     bars: int = 8,
52 ) -> None:
53     """
54     Binary: Cmaj/G7 por barra, subdiv 2, sustain 0.30.
55     Ternary: Cmaj/Fmaj/G7 por barra, subdiv 3, sustain corto (staccato), velocities por subdivisión.
56     """
57     mid = MidiFile(ticks_per_beat=ticks_per_beat)
58     track = MidiTrack()
59     mid.tracks.append(track)
60     track.append(MetaMessage("set_tempo", tempo=bpm2tempo(tempo_bpm), time=0))
61
62     Cmaj = [60, 64, 67]
63     Fmaj = [65, 69, 72]
64     G7 = [67, 71, 74, 77]
65
66     if pattern == "binary":
67         subdiv = 2
68         chords = [Cmaj, G7]
69         dur_beats = 0.30
70         vel_base = 70
71     elif pattern == "ternary":
72         subdiv = 3
73         chords = [Cmaj, Fmaj, G7] # 3 acordes por barra
74         dur_beats = 0.08 # sustain corto (staccato), sin solapamiento
75         vel_base = 60 # variación por k abajo
76     else:
77         raise ValueError("pattern must be 'binary' or 'ternary'")
78
79     def ticks(beats: float) -> int:
80         return int(round(beats * ticks_per_beat))
81
82     beats_per_bar = 4
83     total_beats = bars * beats_per_bar
84
85     abs_events = []
86     for beat in range(total_beats):
87         bar = beat // beats_per_bar
88         chord = chords[bar % len(chords)]
89         for k in range(subdiv):
90             onset_beats = beat + k / subdiv
91             on_tick = ticks(onset_beats)
92             off_tick = ticks(onset_beats + dur_beats)
93             if pattern == "ternary":
94                 vel = vel_base + k * 22 # 60, 82, 104 por subdivisión
95             else:
96                 vel = vel_base
97             for note in chord:
98                 abs_events.append((on_tick, Message("note_on", note=note, velocity=vel)))
99                 abs_events.append((off_tick, Message("note_off", note=note, velocity=0)))
100
101     abs_events.sort(key=lambda x: x[0])
102
103     last = 0
104     for t, msg in abs_events:

```

```

105     msg.time = t - last
106     track.append(msg)
107     last = t
108
109     mid.save(path)
110
111
112 # -----
113 # 2) MIDI parsing → note events in seconds
114 # -----
115
116 NoteEvent = Tuple[float, float, int, int] # (t_on, t_off, pitch, velocity)
117
118
119 def parse_midi_notes_seconds(path: str) -> List[NoteEvent]:
120     """
121     Parse MIDI to (t_on, t_off, pitch, vel) in seconds.
122     Uses mido's iterator, which yields msg.time already in seconds.
123     """
124     mid = MidiFile(path)
125     current_sec = 0.0
126     on: Dict[Tuple[int, int], Tuple[float, int]] = {} # (ch, note) -> (t_on_sec, vel)
127     events: List[NoteEvent] = []
128
129     for msg in mid:
130         current_sec += msg.time
131         if msg.type == "note_on" and msg.velocity > 0:
132             ch = getattr(msg, "channel", 0)
133             on[(ch, msg.note)] = (current_sec, msg.velocity)
134         elif msg.type == "note_off" or (msg.type == "note_on" and msg.velocity == 0):
135             ch = getattr(msg, "channel", 0)
136             key = (ch, msg.note)
137             if key in on:
138                 t_on_sec, vel = on.pop(key)
139                 events.append((t_on_sec, current_sec, msg.note, vel))
140
141     events.sort(key=lambda e: e[0])
142     return events
143
144
145 NoteEventBeats = Tuple[float, float, int, int] # (u_on, u_off, pitch, velocity) in beats
146
147
148 def parse_midi_notes_beats(path: str) -> List[NoteEventBeats]:
149     """
150     Parse MIDI to (u_on, u_off, pitch, vel) in beat-time u (beats).
151     Uses ticks_per_beat and delta times in ticks per track; merges tracks, then u = tick /
152         ticks_per_beat.
153     Beat-time is independent of tempo: tempo changes do not affect u (beats follow the score grid, not
154         real time).
155     """
156     mid = MidiFile(path)
157     tpb = mid.ticks_per_beat
158     # Collect (absolute_tick, msg) from all tracks
159     tick_events: List[Tuple[int, object]] = []
160     for track in mid.tracks:
161         abs_tick = 0
162         for msg in track:

```

```

161         abs_tick += msg.time
162         tick_events.append((abs_tick, msg))
163     tick_events.sort(key=lambda x: x[0])
164
165     on: Dict[Tuple[int, int], Tuple[int, int]] = {} # (ch, note) -> (tick_on, vel)
166     events_beats: List[NoteEventBeats] = []
167     for (tick, msg) in tick_events:
168         if not hasattr(msg, "type"):
169             continue
170         if msg.type == "note_on" and msg.velocity > 0:
171             ch = getattr(msg, "channel", 0)
172             on[(ch, msg.note)] = (tick, msg.velocity)
173         elif msg.type == "note_off" or (msg.type == "note_on" and msg.velocity == 0):
174             ch = getattr(msg, "channel", 0)
175             key = (ch, msg.note)
176             if key in on:
177                 tick_on, vel = on.pop(key)
178                 u_on = tick_on / tpb
179                 u_off = tick / tpb
180                 events_beats.append((u_on, u_off, msg.note, vel))
181
182     events_beats.sort(key=lambda e: e[0])
183     return events_beats
184
185
186 def chroma_series_duration_beats(
187     events: List[NoteEventBeats], bin_size_beats: float
188 ) -> np.ndarray:
189     """
190     Duration-weighted chroma H(u) with bins in beat space. Same logic as chroma_series_duration.
191     """
192     if not events:
193         return np.zeros((0, 12), dtype=float)
194     u0 = events[0][0]
195     u1 = max(e[1] for e in events)
196     U = u1 - u0
197     nbins = max(1, int(U / bin_size_beats) + 1)
198     H = np.zeros((nbins, 12), dtype=float)
199     for (u_on, u_off, pitch, vel) in events:
200         if u_off <= u_on:
201             continue
202         pc = pitch % 12
203         b0 = int((u_on - u0) / bin_size_beats)
204         b1 = int((u_off - u0) / bin_size_beats)
205         for b in range(b0, min(b1 + 1, nbins)):
206             left = u0 + b * bin_size_beats
207             right = u0 + (b + 1) * bin_size_beats
208             overlap = max(0.0, min(u_off, right) - max(u_on, left))
209             if overlap > 0:
210                 H[b, pc] += overlap * (vel / 127.0)
211     row_sums = H.sum(axis=1)
212     nz = row_sums > 0
213     H[nz] = H[nz] / row_sums[nz][:, None]
214     return H
215
216
217 def onset_density_series_beats(
218     events: List[NoteEventBeats], bin_size_beats: float

```

```

219 ) -> np.ndarray:
220     """Onset density a(u) per beat-bin: sum of velocity/127 for note_on in that bin."""
221     if not events:
222         return np.zeros(0, dtype=float)
223     u0 = events[0][0]
224     u1 = max(e[1] for e in events)
225     U = u1 - u0
226     nbins = max(1, int(U / bin_size_beats) + 1)
227     a = np.zeros(nbins, dtype=float)
228     for (u_on, u_off, pitch, vel) in events:
229         if vel <= 0 or u_off <= u_on:
230             continue
231         b = int((u_on - u0) / bin_size_beats)
232         if 0 <= b < nbins:
233             a[b] += vel / 127.0
234     return a
235
236
237 # -----
238 # 3) Harmonic context: duration-weighted chroma (12D) per bin
239 # -----
240
241
242 def chroma_series_duration(events: List[NoteEvent], bin_size: float) -> np.ndarray:
243     """
244     Build duration-weighted chroma series H with shape (nbins, 12).
245     Each row L1-normalized (sum=1) if nonzero.
246     Weight per bin = overlap_length * (velocity/127).
247     """
248     if not events:
249         return np.zeros((0, 12), dtype=float)
250
251     t0 = events[0][0]
252     t1 = max(e[1] for e in events)
253     ev = [(a - t0, b - t0, p, v) for (a, b, p, v) in events]
254     T = t1 - t0
255
256     nbins = max(1, int(T / bin_size) + 1)
257     H = np.zeros((nbins, 12), dtype=float)
258
259     for (t_on, t_off, pitch, vel) in ev:
260         if t_off <= t_on:
261             continue
262         pc = pitch % 12
263         b0 = int(t_on / bin_size)
264         b1 = int(t_off / bin_size)
265         for b in range(b0, min(b1 + 1, nbins)):
266             left = b * bin_size
267             right = (b + 1) * bin_size
268             overlap = max(0.0, min(t_off, right) - max(t_on, left))
269             if overlap > 0:
270                 H[b, pc] += overlap * (vel / 127.0)
271
272     row_sums = H.sum(axis=1)
273     nz = row_sums > 0
274     H[nz] = H[nz] / row_sums[nz][:, None]
275     return H
276

```

```

277
278 def onset_density_series(events: List[NoteEvent], bin_size: float) -> np.ndarray:
279     """
280     Onset density  $a(t)$  por bin: suma de velocity/127 para note_on (vel>0) cuyo t_on cae en el bin.
281      $a(t) \in \mathbb{R}$ , shape (nbins,).
282     """
283     if not events:
284         return np.zeros(0, dtype=float)
285     t0 = events[0][0]
286     t1 = max(e[1] for e in events)
287     T = t1 - t0
288     nbins = max(1, int(T / bin_size) + 1)
289     a = np.zeros(nbins, dtype=float)
290     for (t_on, t_off, pitch, vel) in events:
291         if vel <= 0 or t_off <= t_on:
292             continue
293         b = int((t_on - t0) / bin_size)
294         if 0 <= b < nbins:
295             a[b] += vel / 127.0
296     return a
297
298
299 def series_with_rhythm(
300     H: np.ndarray, a: np.ndarray, alpha: float = 1.0
301 ) -> np.ndarray:
302     """
303     Patrón por bin:  $X(t) = [H(t); \alpha a(t)]$ , dim 13.
304      $H$  (nbins, 12),  $a$  (nbins,) ->  $X$  (nbins, 13).
305     """
306     nbins = min(len(H), len(a))
307     if nbins == 0:
308         return np.zeros((0, 13), dtype=float)
309     a_col = (alpha * a[:nbins]).reshape(-1, 1)
310     return np.hstack([H[:nbins], a_col])
311
312
313 def spectral_flux_series(H: np.ndarray) -> np.ndarray:
314     """ $f(t) = ||H(t) - H(t-1)||_2$  per bin;  $f(0)=0$ ."""
315     nbins = len(H)
316     f = np.zeros(nbins, dtype=float)
317     for i in range(1, nbins):
318         f[i] = float(np.linalg.norm(H[i] - H[i - 1]))
319     return f
320
321
322 def zscore_series(x: np.ndarray) -> np.ndarray:
323     """Z-score over the whole series; if std=0 return zeros."""
324     out = np.asarray(x, dtype=float).copy()
325     m, s = np.mean(out), np.std(out)
326     if s > 1e-12:
327         out = (out - m) / s
328     else:
329         out[:] = 0.0
330     return out
331
332
333 def ioi_histogram_for_window(
334     onset_times: np.ndarray,

```

```

335     t_start: float,
336     t_end: float,
337     d: int = 16,
338     Tmax: float = 2.0,
339 ) -> np.ndarray:
340     """
341     IOI histogram for onsets in [t_start, t_end). d bins, max IOI = Tmax s.
342     Onset_times: sorted array of onset times (e.g. t_on from events).
343     Returns histogram (d,) normalized to sum 1; if no IOIs, return uniform 1/d.
344     """
345     in_window = (onset_times >= t_start) & (onset_times < t_end)
346     times = np.sort(onset_times[in_window])
347     if len(times) < 2:
348         h = np.ones(d, dtype=float) / d
349         return h
350     iois = np.diff(times)
351     iois = iois[iois < Tmax]
352     if len(iois) == 0:
353         return np.ones(d, dtype=float) / d
354     edges = np.linspace(0, Tmax, d + 1)
355     h, _ = np.histogram(iois, bins=edges)
356     h = h.astype(float)
357     if h.sum() > 0:
358         h = h / h.sum()
359     else:
360         h = np.ones(d) / d
361     return h
362
363
364 def ioi_histogram_for_window_beats(
365     onset_times_u: np.ndarray,
366     u_start: float,
367     u_end: float,
368     d: int = 16,
369     Tmax_beats: float = 2.0,
370 ) -> np.ndarray:
371     """
372     IOI histogram for onsets in [u_start, u_end) in beat-time. d bins, max IOI = Tmax_beats (beats).
373     Default 2.0 beats.
374     Returns histogram (d,) normalized to sum 1; if no IOIs, return uniform 1/d.
375     """
376     in_window = (onset_times_u >= u_start) & (onset_times_u < u_end)
377     times = np.sort(onset_times_u[in_window])
378     if len(times) < 2:
379         return np.ones(d, dtype=float) / d
380     iois = np.diff(times)
381     iois = iois[iois < Tmax_beats]
382     if len(iois) == 0:
383         return np.ones(d, dtype=float) / d
384     edges = np.linspace(0, Tmax_beats, d + 1)
385     h, _ = np.histogram(iois, bins=edges)
386     h = h.astype(float)
387     if h.sum() > 0:
388         h = h / h.sum()
389     else:
390         h = np.ones(d) / d
391     return h

```

```

392
393 def plot_chroma_heatmap(
394     H: np.ndarray,
395     title: str = "Chroma heatmap",
396     save_path: str | None = None,
397 ) -> None:
398     """Heatmap 12 x timebins for chroma (duration-weighted)."""
399     plt.figure(figsize=(12, 4))
400     plt.imshow(H.T, aspect="auto", origin="lower")
401     plt.yticks(range(12), [str(k) for k in range(12)])
402     plt.xlabel("time bin")
403     plt.ylabel("pitch class")
404     plt.title(title)
405     plt.tight_layout()
406     if save_path:
407         plt.savefig(save_path, dpi=120)
408         plt.close()
409     else:
410         plt.show()
411
412
413 # -----
414 # 4) Patterns  $D_n$ : windows of length  $N = p^n$  over chroma series
415 # -----
416
417
418 def build_patterns_from_series(
419     X: np.ndarray, N: int, step: int = 1
420 ) -> List[np.ndarray]:
421     """Windows  $a: Z/(p^n)Z \rightarrow X$  as arrays of shape  $(N, D)$ .  $D=12$  (chroma) or 13 (chroma+rhythm). step =
        window slide."""
422     if len(X) < N:
423         return []
424     patterns = []
425     for start in range(0, len(X) - N + 1, step):
426         patterns.append(X[start : start + N, :].copy())
427     return patterns
428
429
430 # -----
431 # 5) Metric and proximity graph
432 # -----
433
434
435 def pattern_dist(a: np.ndarray, b: np.ndarray) -> float:
436     """ $d_n(a, b) = \sqrt{\text{mean}_i ||a[i] - b[i]||_2^2}$ ."""
437     return float(np.sqrt(np.mean(np.sum((a - b) ** 2, axis=1))))
438
439
440 def proximity_graph(patterns: List[np.ndarray], delta: float) -> nx.Graph:
441     """ $G_n\delta()$ : vertices = patterns; edge if  $d_n(a, b) \leq \delta$ ."""
442     G = nx.Graph()
443     m = len(patterns)
444     G.add_nodes_from(range(m))
445     for i in range(m):
446         for j in range(i + 1, m):
447             if pattern_dist(patterns[i], patterns[j]) <= delta:
448                 G.add_edge(i, j)

```

```

449     return G
450
451
452 def adaptive_delta_from_patterns(
453     patterns: List[np.ndarray],
454     q: float = 0.05,
455     sample_size: int = 5000,
456     rng: np.random.Generator | None = None,
457 ) -> float:
458     """
459     (A) Umbral adaptativo: muestra de distancias pairwise (sample_size pares),
460     delta_n = cuantil q. Evita colapso del grafo en niveles finos.
461     """
462     m = len(patterns)
463     if m < 2:
464         return 0.0
465     rng = rng or np.random.default_rng()
466     max_pairs = m * (m - 1) // 2
467     n_sample = min(sample_size, max_pairs)
468     if n_sample == 0:
469         return 0.0
470     if n_sample >= max_pairs:
471         dists = []
472         for i in range(m):
473             for j in range(i + 1, m):
474                 dists.append(pattern_dist(patterns[i], patterns[j]))
475         dists = np.array(dists)
476     else:
477         dists = []
478         seen = set()
479         while len(dists) < n_sample:
480             i, j = rng.integers(0, m, size=2)
481             if i == j or (i, j) in seen or (j, i) in seen:
482                 continue
483             seen.add((i, j))
484             dists.append(pattern_dist(patterns[i], patterns[j]))
485         dists = np.array(dists)
486     return float(np.quantile(dists, q))
487
488
489 def knn_graph(patterns: List[np.ndarray], k: int = 10) -> nx.Graph:
490     """
491     (B) Grafo kNN: cada nodo se conecta con sus k vecinos más cercanos; grafo no dirigido (simetrizado).
492     Evita colapso por delta fijo; k configurable (default 10).
493     """
494     G = nx.Graph()
495     m = len(patterns)
496     G.add_nodes_from(range(m))
497     k_actual = min(k, m - 1)
498     if k_actual < 1:
499         return G
500     for i in range(m):
501         dists = [(pattern_dist(patterns[i], patterns[j]), j) for j in range(m) if j != i]
502         dists.sort(key=lambda x: x[0])
503         for _, j in dists[:k_actual]:
504             G.add_edge(i, j)
505     return G
506

```

```

507
508 # -----
509 # 5b) Phase I: multi-observable distance (H, a, f, g_W)
510 # -----
511
512 Phase1Pattern = Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray] # (H_block, a_block, f_block,
    g_W)
513
514
515 def pattern_dist_phase1(
516     p: Phase1Pattern,
517     q: Phase1Pattern,
518     alpha: float,
519     beta: float,
520     gamma: float,
521 ) -> float:
522     """
523      $d^2 = \text{mean}_i ||H_i - H'_i||^2 + \alpha^2 \text{mean}_i |a_i - a'_i|^2$ 
524      $+ \beta^2 \text{mean}_i |f_i - f'_i|^2 + \gamma^2 ||g_W - g'_W||^2$ 
525     """
526     H1, a1, f1, g1 = p
527     H2, a2, f2, g2 = q
528     N = len(H1)
529     term_H = np.mean(np.sum((H1 - H2) ** 2, axis=1))
530     term_a = np.mean((a1 - a2) ** 2)
531     term_f = np.mean((f1 - f2) ** 2)
532     term_g = np.sum((g1 - g2) ** 2)
533     d2 = term_H + (alpha ** 2) * term_a + (beta ** 2) * term_f + (gamma ** 2) * term_g
534     return float(np.sqrt(max(0.0, d2)))
535
536
537 def knn_graph_phase1(
538     patterns: List[Phase1Pattern],
539     k: int,
540     alpha: float,
541     beta: float,
542     gamma: float,
543 ) -> nx.Graph:
544     """kNN sobre patrones Phase1 con distancia ponderada."""
545     G = nx.Graph()
546     m = len(patterns)
547     G.add_nodes_from(range(m))
548     k_actual = min(k, m - 1)
549     if k_actual < 1:
550         return G
551     for i in range(m):
552         dists = [
553             (pattern_dist_phase1(patterns[i], patterns[j], alpha, beta, gamma), j)
554             for j in range(m)
555             if j != i
556         ]
557         dists.sort(key=lambda x: x[0])
558         for _, j in dists[:k_actual]:
559             G.add_edge(i, j)
560     return G
561
562
563 def build_phase1_patterns(

```

```

564     events: List[NoteEvent],
565     H: np.ndarray,
566     a_raw: np.ndarray,
567     bin_size: float,
568     t0: float,
569     N: int,
570     step: int,
571     d_ioi: int = 16,
572     Tmax_ioi: float = 2.0,
573 ) -> List[PhaseIPattern]:
574     """
575     Build list of PhaseI patterns for window length N.
576     a and f z-scored over the piece; g_W per window (IOI histogram), normalized sum=1.
577     """
578     nbins = len(H)
579     f = spectral_flux_series(H)
580     a = zscore_series(a_raw)
581     f = zscore_series(f)
582     onset_times = np.array([e[0] for e in events], dtype=float)
583
584     patterns: List[PhaseIPattern] = []
585     for start in range(0, nbins - N + 1, step):
586         H_block = H[start : start + N, :].copy()
587         a_block = a[start : start + N].copy()
588         f_block = f[start : start + N].copy()
589         t_start = t0 + start * bin_size
590         t_end = t0 + (start + N) * bin_size
591         g_W = ioi_histogram_for_window(onset_times, t_start, t_end, d=d_ioi, Tmax=Tmax_ioi)
592         patterns.append((H_block, a_block, f_block, g_W))
593     return patterns
594
595
596 def build_phasel_patterns_beats(
597     events: List[NoteEventBeats],
598     H: np.ndarray,
599     a_raw: np.ndarray,
600     bin_size_beats: float,
601     u0: float,
602     N: int,
603     step: int,
604     d_ioi: int = 16,
605     Tmax_ioi_beats: float = 2.0,
606 ) -> List[PhaseIPattern]:
607     """
608     PhaseI patterns in beat-time: H(u), a(u), f(u) z-scored; g_W = IOI histogram in beats.
609     Tmax_ioi_beats default 2.0.
610     """
611     nbins = len(H)
612     f = spectral_flux_series(H)
613     a = zscore_series(a_raw)
614     f = zscore_series(f)
615     onset_times_u = np.array([e[0] for e in events], dtype=float)
616
617     patterns: List[PhaseIPattern] = []
618     for start in range(0, nbins - N + 1, step):
619         H_block = H[start : start + N, :].copy()
620         a_block = a[start : start + N].copy()
621         f_block = f[start : start + N].copy()

```

```

621     u_start = u0 + start * bin_size_beats
622     u_end = u0 + (start + N) * bin_size_beats
623     g_W = ioi_histogram_for_window_beats(
624         onset_times_u, u_start, u_end, d=d_ioi, Tmax_beats=Tmax_ioi_beats
625     )
626     patterns.append((H_block, a_block, f_block, g_W))
627     return patterns
628
629
630 # -----
631 # 6) Tower runner and profile plots
632 # -----
633
634
635 def _spectral_invariants(G: nx.Graph) -> Tuple[float, float, float, float, float]:
636     """Fiedler (lambda2) of normalized Laplacian, average clustering, degree percentiles p25,p50,p75.
637     Returns (lambda2, clustering, p25, p50, p75)."""
638     if G.number_of_nodes() == 0:
639         return 0.0, 0.0, 0.0, 0.0, 0.0
640     try:
641         L = nx.normalized_laplacian_matrix(G)
642         ev = np.linalg.eigvalsh(L.toarray())
643         ev = np.sort(ev)
644         lambda2 = float(ev[1]) if len(ev) > 1 else 0.0
645     except Exception:
646         lambda2 = 0.0
647     try:
648         clustering = nx.average_clustering(G)
649     except Exception:
650         clustering = 0.0
651     degs = [d for _, d in G.degree()]
652     p25 = float(np.percentile(degs, 25)) if degs else 0.0
653     p50 = float(np.percentile(degs, 50)) if degs else 0.0
654     p75 = float(np.percentile(degs, 75)) if degs else 0.0
655     return lambda2, clustering, p25, p50, p75
656
657 @dataclass
658 class LevelResult:
659     n: int
660     N: int
661     num_patterns: int
662     num_edges: int
663     avg_degree: float
664     beta0: int
665     delta_used: float # delta (quantile/fixed) o -1 si kNN
666     giant_component_frac: float # |C_max|/V
667     lambda2: float = 0.0 # Fiedler (normalized Laplacian)
668     clustering: float = 0.0 # average clustering
669     degree_p25: float = 0.0
670     degree_p50: float = 0.0
671     degree_p75: float = 0.0
672
673
674 def run_tower_on_midi(
675     path: str,
676     p: int,
677     n_min: int,

```

```

678     n_max: int,
679     bin_size: float,
680     step: int,
681     delta: float,
682     cap: int = 300,
683     threshold_mode: str = "quantile",
684     quantile_q: float = 0.05,
685     sample_size: int = 5000,
686     knn_k: int = 10,
687     alpha: float = 1.0,
688     show_heatmap: bool = True,
689     save_heatmap: str | None = None,
690     rng: np.random.Generator | None = None,
691 ) -> List[LevelResult]:
692     """
693     Build chroma  $H(t)$  + onset density  $a(t)$ ;  $X(t)=[H(t); \alpha*a(t)]$  dim 13.
694     At each level  $n$  build  $G_n$  by (A) quantile or (B) kNN. Report  $V$ ,  $E$ ,  $avg\_degree$ ,  $\beta_0$ ,
695     giant_component_frac.
696     """
697     events = parse_midi_notes_seconds(path)
698     H = chroma_series_duration(events, bin_size=bin_size)
699     a = onset_density_series(events, bin_size=bin_size)
700     X = series_with_rhythm(H, a, alpha=alpha)
701
702     if show_heatmap or save_heatmap:
703         plot_chroma_heatmap(
704             H,
705             title=f"{Path(path).name} | chroma (duration-weighted), bin={bin_size}",
706             save_path=save_heatmap,
707         )
708
709     rng = rng or np.random.default_rng()
710     results: List[LevelResult] = []
711     for n in range(n_min, n_max + 1):
712         N = p**n
713         patterns = build_patterns_from_series(X, N=N, step=step)
714
715         if len(patterns) > cap:
716             idx = np.linspace(0, len(patterns) - 1, cap).astype(int)
717             patterns = [patterns[i] for i in idx]
718
719         if not patterns:
720             results.append(
721                 LevelResult(
722                     n=n,
723                     N=N,
724                     num_patterns=0,
725                     num_edges=0,
726                     avg_degree=0.0,
727                     beta0=0,
728                     delta_used=delta,
729                     giant_component_frac=0.0,
730                 )
731             )
732             continue
733
734         if threshold_mode == "quantile":
735             delta_n = adaptive_delta_from_patterns(

```

```

735         patterns, q=quantile_q, sample_size=sample_size, rng=rng
736     )
737     G = proximity_graph(patterns, delta=delta_n)
738     elif threshold_mode == "knn":
739         G = knn_graph(patterns, k=knn_k)
740         delta_n = -1.0 # sentinel: kNN
741     else:
742         delta_n = delta
743         G = proximity_graph(patterns, delta=delta)
744
745     V = len(patterns)
746     E = G.number_of_edges()
747     avg_deg = (2.0 * E / V) if V > 0 else 0.0
748     comps = list(nx.connected_components(G))
749     max_comp_size = max(len(c) for c in comps) if comps else 0
750     giant_frac = (max_comp_size / V) if V > 0 else 0.0
751     results.append(
752         LevelResult(
753             n=n,
754             N=N,
755             num_patterns=V,
756             num_edges=E,
757             avg_degree=avg_deg,
758             beta0=len(comps),
759             delta_used=delta_n,
760             giant_component_frac=giant_frac,
761         )
762     )
763     return results
764
765
766 def run_tower_phase1(
767     path: str,
768     p: int,
769     n_min: int,
770     n_max: int,
771     bin_size: float,
772     step: int,
773     cap: int,
774     knn_k: int,
775     alpha: float,
776     beta: float,
777     gamma: float,
778     d_oi: int = 16,
779     Tmax_oi: float = 2.0,
780     time_axis: str = "seconds",
781     bin_size_beats: float = 1.0 / 12.0,
782     Tmax_oi_beats: float = 2.0,
783 ) -> List[LevelResult]:
784     """
785     Phase I: H, a (z-score), f (spectral flux z-score), g_W (IOI hist per window).
786     Distance  $d^2 = \text{mean}||H-H'|||^2 + \alpha^2 \text{mean}|a-a'|^2 + \beta^2 \text{mean}|f-f'|^2 + \gamma^2 ||g_W-g'_W||^2$ .
787     kNN graph only.
788     time_axis: 'seconds' (default) or 'beats'. If beats, bin_size_beats and Tmax_oi_beats (default 2.0
789         beats) used.
790     """
791     if time_axis == "beats":
792         events_beats = parse_midi_notes_beats(path)

```

```

792     if not events_beats:
793         return []
794     H = chroma_series_duration_beats(events_beats, bin_size_beats=bin_size_beats)
795     a_raw = onset_density_series_beats(events_beats, bin_size_beats=bin_size_beats)
796     u0 = events_beats[0][0]
797     results: List[LevelResult] = []
798     for n in range(n_min, n_max + 1):
799         N = p*n
800         patterns = build_phase1_patterns_beats(
801             events_beats,
802             H,
803             a_raw,
804             bin_size_beats,
805             u0,
806             N=N,
807             step=step,
808             d_ioi=d_ioi,
809             Tmax_ioi_beats=Tmax_ioi_beats,
810         )
811         if len(patterns) > cap:
812             idx = np.linspace(0, len(patterns) - 1, cap).astype(int)
813             patterns = [patterns[i] for i in idx]
814         if not patterns:
815             results.append(
816                 LevelResult(
817                     n=n, N=N, num_patterns=0, num_edges=0, avg_degree=0.0,
818                     beta0=0, delta_used=-1.0, giant_component_frac=0.0,
819                 )
820             )
821             continue
822         G = knn_graph_phase1(patterns, knn_k, alpha, beta, gamma)
823         V = len(patterns)
824         E = G.number_of_edges()
825         avg_deg = (2.0 * E / V) if V > 0 else 0.0
826         comps = list(nx.connected_components(G))
827         max_comp = max(len(c) for c in comps) if comps else 0
828         giant_frac = (max_comp / V) if V > 0 else 0.0
829         lam2, clust, dp25, dp50, dp75 = _spectral_invariants(G)
830         results.append(
831             LevelResult(
832                 n=n, N=N, num_patterns=V, num_edges=E, avg_degree=avg_deg,
833                 beta0=len(comps), delta_used=-1.0, giant_component_frac=giant_frac,
834                 lambda2=lam2, clustering=clust, degree_p25=dp25, degree_p50=dp50, degree_p75=dp75,
835             )
836         )
837     return results
838
839 events = parse_midi_notes_seconds(path)
840 H = chroma_series_duration(events, bin_size=bin_size)
841 a_raw = onset_density_series(events, bin_size=bin_size)
842 t0 = events[0][0] if events else 0.0
843
844 results = []
845 for n in range(n_min, n_max + 1):
846     N = p*n
847     patterns = build_phase1_patterns(
848         events, H, a_raw, bin_size, t0, N=N, step=step, d_ioi=d_ioi, Tmax_ioi=Tmax_ioi
849     )

```

```

850     if len(patterns) > cap:
851         idx = np.linspace(0, len(patterns) - 1, cap).astype(int)
852         patterns = [patterns[i] for i in idx]
853     if not patterns:
854         results.append(
855             LevelResult(
856                 n=n, N=N, num_patterns=0, num_edges=0, avg_degree=0.0,
857                 beta0=0, delta_used=-1.0, giant_component_frac=0.0,
858             )
859         )
860         continue
861     G = knn_graph_phase1(patterns, knn_k, alpha, beta, gamma)
862     V = len(patterns)
863     E = G.number_of_edges()
864     avg_deg = (2.0 * E / V) if V > 0 else 0.0
865     comps = list(nx.connected_components(G))
866     max_comp = max(len(c) for c in comps) if comps else 0
867     giant_frac = (max_comp / V) if V > 0 else 0.0
868     lam2, clust, dp25, dp50, dp75 = _spectral_invariants(G)
869     results.append(
870         LevelResult(
871             n=n, N=N, num_patterns=V, num_edges=E, avg_degree=avg_deg,
872             beta0=len(comps), delta_used=-1.0, giant_component_frac=giant_frac,
873             lambda2=lam2, clustering=clust, degree_p25=dp25, degree_p50=dp50, degree_p75=dp75,
874         )
875     )
876     return results
877
878
879 def run_tower_on_beats(
880     path: str,
881     p: int,
882     n_min: int,
883     n_max: int,
884     bin_size_beats: float,
885     step: int,
886     cap: int,
887     knn_k: int,
888     alpha: float,
889 ) -> List[LevelResult]:
890     """
891     Beat-based binning:  $H(u)$  and  $a(u)$  with  $bin\_size\_beats$ ;  $X(u)=[H(u); \alpha*a(u)]$ .
892     kNN only. Same tower as baseline (pattern_dist, knn_graph).
893     """
894     events = parse_midi_notes_beats(path)
895     if not events:
896         return []
897     H = chroma_series_duration_beats(events, bin_size_beats=bin_size_beats)
898     a = onset_density_series_beats(events, bin_size_beats=bin_size_beats)
899     X = series_with_rhythm(H, a, alpha=alpha)
900
901     results: List[LevelResult] = []
902     for n in range(n_min, n_max + 1):
903         N = p*n
904         patterns = build_patterns_from_series(X, N=N, step=step)
905         if len(patterns) > cap:
906             idx = np.linspace(0, len(patterns) - 1, cap).astype(int)
907             patterns = [patterns[i] for i in idx]

```

```

908         if not patterns:
909             results.append(
910                 LevelResult(
911                     n=n, N=N, num_patterns=0, num_edges=0, avg_degree=0.0,
912                     beta0=0, delta_used=-1.0, giant_component_frac=0.0,
913                 )
914             )
915             continue
916         G = knn_graph(patterns, k=knn_k)
917         V = len(patterns)
918         E = G.number_of_edges()
919         avg_deg = (2.0 * E / V) if V > 0 else 0.0
920         comps = list(nx.connected_components(G))
921         max_comp = max(len(c) for c in comps) if comps else 0
922         giant_frac = (max_comp / V) if V > 0 else 0.0
923         results.append(
924             LevelResult(
925                 n=n, N=N, num_patterns=V, num_edges=E, avg_degree=avg_deg,
926                 beta0=len(comps), delta_used=-1.0, giant_component_frac=giant_frac,
927             )
928         )
929     return results
930
931
932 def save_results_csv(
933     results: List[LevelResult],
934     filepath: str,
935     source: str,
936     p: int,
937     threshold_mode: str = "quantile",
938 ) -> None:
939     """CSV: n, N, V, E, avg_degree, beta0, delta_used, giant_component_frac [, lambda2, clustering,
940         degree_p25, degree_p50, degree_p75]."""
941     with open(filepath, "w", newline="") as f:
942         w = csv.writer(f)
943         has_spectral = getattr(results[0], "lambda2", None) is not None if results else False
944         row0 = ["n", "N", "V", "E", "avg_degree", "beta0", "delta_used", "giant_component_frac"]
945         if has_spectral:
946             row0 += ["lambda2", "clustering", "degree_p25", "degree_p50", "degree_p75"]
947         w.writerow(row0)
948         for r in results:
949             du = r.delta_used if (threshold_mode == "quantile" or r.delta_used >= 0) else -1
950             row = [r.n, r.N, r.num_patterns, r.num_edges, round(r.avg_degree, 6), r.beta0, du,
951                 round(r.giant_component_frac, 6)]
952             if has_spectral:
953                 row += [round(getattr(r, "lambda2", 0), 6), round(getattr(r, "clustering", 0), 6),
954                     round(getattr(r, "degree_p25", 0), 4), round(getattr(r, "degree_p50", 0), 4),
955                     round(getattr(r, "degree_p75", 0), 4)]
956             w.writerow(row)
957
958
959 def save_results_csv_phase2(
960     results: List[LevelResult],
961     filepath: str,
962     knn_k: int,
963     k_over_V_threshold: float = 0.15,
964 ) -> None:
965     """CSV Phase II: n, N, V, E, avg_degree, beta0, giant_component_frac, k_over_V, no_informativo."""

```

```

963     with open(filepath, "w", newline="") as f:
964         w = csv.writer(f)
965         w.writerow(["n", "N", "V", "E", "avg_degree", "beta0", "giant_component_frac", "k_over_V",
966                     "no_informativo"])
967     for r in results:
968         V = r.num_patterns
969         k_over_V = (knn_k / V) if V > 0 else 0.0
970         no_inf = "yes" if k_over_V > k_over_V_threshold else "no"
971         w.writerow([
972             r.n, r.N, V, r.num_edges, round(r.avg_degree, 6), r.beta0,
973             round(r.giant_component_frac, 6), round(k_over_V, 6), no_inf,
974         ])
975
976 def plot_profile(
977     results: List[LevelResult],
978     title: str,
979     save_prefix: str | None = None,
980     run_label: str = "",
981 ) -> None:
982     """Plot |D_n|, beta0 y avg_degree vs n. Si save_prefix y run_label, guarda con nombre único."""
983     ns = [r.n for r in results]
984     sizes = [r.num_patterns for r in results]
985     b0 = [r.beta0 for r in results]
986     avg_deg = [r.avg_degree for r in results]
987     tag = f"{save_prefix}{run_label}_" if (save_prefix and run_label) else (save_prefix or "")
988
989     plt.figure()
990     plt.plot(ns, sizes, marker="o")
991     plt.xlabel("n")
992     plt.ylabel("|D_n| (windows sampled)")
993     plt.title(title + " : |D_n|")
994     plt.tight_layout()
995     if save_prefix:
996         plt.savefig(f"{tag}Dn.png", dpi=120)
997         plt.close()
998     else:
999         plt.show()
1000
1001     plt.figure()
1002     plt.plot(ns, b0, marker="o")
1003     plt.xlabel("n")
1004     plt.ylabel("beta0 (#components)")
1005     plt.title(title + " : beta0")
1006     plt.tight_layout()
1007     if save_prefix:
1008         plt.savefig(f"{tag}beta0.png", dpi=120)
1009         plt.close()
1010     else:
1011         plt.show()
1012
1013     plt.figure()
1014     plt.plot(ns, avg_deg, marker="o")
1015     plt.xlabel("n")
1016     plt.ylabel("avg_degree (2|E|/|V|)")
1017     plt.title(title + " : avg_degree")
1018     plt.tight_layout()
1019     if save_prefix:

```

```

1020     plt.savefig(f"{tag}avg_degree.png", dpi=120)
1021     plt.close()
1022     else:
1023         plt.show()
1024
1025     plt.figure()
1026     gcf = [r.giant_component_frac for r in results]
1027     plt.plot(ns, gcf, marker="o")
1028     plt.xlabel("n")
1029     plt.ylabel("giant_component_frac ( $|C_{max}|/V$ )")
1030     plt.title(title + " : giant_component_frac")
1031     plt.tight_layout()
1032     if save_prefix:
1033         plt.savefig(f"{tag}giant_component_frac.png", dpi=120)
1034         plt.close()
1035     else:
1036         plt.show()
1037
1038
1039 def plot_profile_compare(
1040     res_p2: List[LevelResult],
1041     res_p3: List[LevelResult],
1042     title_prefix: str,
1043     save_prefix: str,
1044     run_label: str,
1045 ) -> None:
1046     """Cuatro figuras:  $|D_n|(n)$ , avg_degree(n), beta0(n), giant_component_frac(n) con curvas p=2 y
1047         p=3. """
1048     ns2 = [r.n for r in res_p2]
1049     ns3 = [r.n for r in res_p3]
1050     tag = f"{save_prefix}{run_label}_"
1051     plt.figure()
1052     plt.plot(ns2, [r.num_patterns for r in res_p2], marker="o", label="p=2")
1053     plt.plot(ns3, [r.num_patterns for r in res_p3], marker="s", label="p=3")
1054     plt.xlabel("n")
1055     plt.ylabel(" $|D_n| (V)$ ")
1056     plt.title(f"{title_prefix} :  $|D_n|$  (p=2 vs p=3)")
1057     plt.legend()
1058     plt.tight_layout()
1059     plt.savefig(f"{tag}Dn.png", dpi=120)
1060     plt.close()
1061
1062     plt.figure()
1063     plt.plot(ns2, [r.avg_degree for r in res_p2], marker="o", label="p=2")
1064     plt.plot(ns3, [r.avg_degree for r in res_p3], marker="s", label="p=3")
1065     plt.xlabel("n")
1066     plt.ylabel("avg_degree")
1067     plt.title(f"{title_prefix} : avg_degree (p=2 vs p=3)")
1068     plt.legend()
1069     plt.tight_layout()
1070     plt.savefig(f"{tag}avg_degree.png", dpi=120)
1071     plt.close()
1072
1073     plt.figure()
1074     plt.plot(ns2, [r.beta0 for r in res_p2], marker="o", label="p=2")
1075     plt.plot(ns3, [r.beta0 for r in res_p3], marker="s", label="p=3")
1076     plt.xlabel("n")
1077     plt.ylabel("beta0")

```

```

1077 plt.title(f"{title_prefix} : beta0 (p=2 vs p=3)")
1078 plt.legend()
1079 plt.tight_layout()
1080 plt.savefig(f"{tag}beta0.png", dpi=120)
1081 plt.close()
1082
1083 plt.figure()
1084 plt.plot(ns2, [r.giant_component_frac for r in res_p2], marker="o", label="p=2")
1085 plt.plot(ns3, [r.giant_component_frac for r in res_p3], marker="s", label="p=3")
1086 plt.xlabel("n")
1087 plt.ylabel("giant_component_frac")
1088 plt.title(f"{title_prefix} : giant_component_frac (p=2 vs p=3)")
1089 plt.legend()
1090 plt.tight_layout()
1091 plt.savefig(f"{tag}giant_component_frac.png", dpi=120)
1092 plt.close()
1093
1094 if getattr(res_p2[0], "lambda2", 0) != 0 or getattr(res_p3[0], "lambda2", 0) != 0:
1095     plt.figure()
1096     plt.plot(ns2, [getattr(r, "lambda2", 0) for r in res_p2], marker="o", label="p=2")
1097     plt.plot(ns3, [getattr(r, "lambda2", 0) for r in res_p3], marker="s", label="p=3")
1098     plt.xlabel("n")
1099     plt.ylabel("lambda2 (Fiedler)")
1100     plt.title(f"{title_prefix} : lambda2 (p=2 vs p=3)")
1101     plt.legend()
1102     plt.tight_layout()
1103     plt.savefig(f"{tag}lambda2.png", dpi=120)
1104     plt.close()
1105 if getattr(res_p2[0], "clustering", 0) != 0 or getattr(res_p3[0], "clustering", 0) != 0:
1106     plt.figure()
1107     plt.plot(ns2, [getattr(r, "clustering", 0) for r in res_p2], marker="o", label="p=2")
1108     plt.plot(ns3, [getattr(r, "clustering", 0) for r in res_p3], marker="s", label="p=3")
1109     plt.xlabel("n")
1110     plt.ylabel("clustering")
1111     plt.title(f"{title_prefix} : clustering (p=2 vs p=3)")
1112     plt.legend()
1113     plt.tight_layout()
1114     plt.savefig(f"{tag}clustering.png", dpi=120)
1115     plt.close()
1116
1117 # -----
1118 # 6b) Phase I Bach: multi-observable (H, a, f, g_W), configs A/B/C, reporte Delta, Delta_G, ÉXITO
1119 # -----
1120
1121
1122
1123 def _n_max_for_prime(p: int, max_window: int = 2000) -> int:
1124     """Max level n such that p^n <= max_window (for control primes)."""
1125     n = 1
1126     while n < 20 and p ** n <= max_window:
1127         n += 1
1128     return max(1, n - 1)
1129
1130
1131 def run_phase1_bach_report(
1132     real_path: str,
1133     save_prefix: str,
1134     bin_size: float = 0.05,

```

```

1135     step: int = 2,
1136     cap: int = 300,
1137     knn_k_list: Optional[List[int]] = None,
1138     n_max_p2: int = 6,
1139     n_max_p3: int = 5,
1140     time_axis: str = "seconds",
1141     bin_size_beats: float = 1.0 / 12.0,
1142     Tmax_ioi_beats: float = 2.0,
1143     k_over_V_threshold: float = 0.15,
1144     control_primes: Optional[List[int]] = None,
1145 ) -> None:
1146     """
1147     Phase I solo para Bach: configs (A) alpha=1,beta=0,gamma=0; (B) 1,1,0; (C) 1,1,1.
1148     k=8,10,12; p=2 n_max=6, p=3 n_max=5. Reporta Delta(n)=beta0_p2(n)-beta0_p3(n),
1149     giant_component_frac p=2/p=3, Delta_G(n); ÉXITO=1 si (existe n con  $\Delta \theta$  en  $\geq k$ ) o
1150     ( $|\Delta_G| \geq 0.10$  en algún n).
1151     time_axis='seconds' (default) o 'beats'; si beats, salidas con sufijo _beats y reporte en
1152     phase2_beats_report.txt.
1153     """
1154     if knn_k_list is None:
1155         knn_k_list = [8, 10, 12]
1156     configs: List[Tuple[str, float, float, float]] = [
1157         ("A", 1.0, 0.0, 0.0),
1158         ("B", 1.0, 1.0, 0.0),
1159         ("C", 1.0, 1.0, 1.0),
1160     ]
1161     common_n_max = min(n_max_p3, n_max_p2)
1162     use_beats = time_axis == "beats"
1163     suffix = "_beats" if use_beats else ""
1164     report_filename = f"{save_prefix}phase2_beats_report.txt" if use_beats else
1165         f"{save_prefix}phase1_bach_report.txt"
1166
1167     all_delta_nonzero_count: Dict[Tuple[str, int], List[int]] = {}
1168     any_delta_G_ge_010 = False
1169     config_any_delta_G_ge_010: Dict[str, bool] = {c[0]: False for c in configs}
1170     report_lines: List[str] = []
1171
1172     for config_name, alpha, beta, gamma in configs:
1173         for k in knn_k_list:
1174             if use_beats:
1175                 res_p2 = run_tower_phase1(
1176                     real_path,
1177                     p=2,
1178                     n_min=1,
1179                     n_max=n_max_p2,
1180                     bin_size=bin_size,
1181                     step=step,
1182                     cap=cap,
1183                     knn_k=k,
1184                     alpha=alpha,
1185                     beta=beta,
1186                     gamma=gamma,
1187                     time_axis="beats",
1188                     bin_size_beats=bin_size_beats,
1189                     Tmax_ioi_beats=Tmax_ioi_beats,
1190                 )
1191                 res_p3 = run_tower_phase1(
1192                     real_path,

```

```

1190         p=3,
1191         n_min=1,
1192         n_max=n_max_p3,
1193         bin_size=bin_size,
1194         step=step,
1195         cap=cap,
1196         knn_k=k,
1197         alpha=alpha,
1198         beta=beta,
1199         gamma=gamma,
1200         time_axis="beats",
1201         bin_size_beats=bin_size_beats,
1202         Tmax_ioi_beats=Tmax_ioi_beats,
1203     )
1204 else:
1205     res_p2 = run_tower_phase1(
1206         real_path,
1207         p=2, n_min=1, n_max=n_max_p2,
1208         bin_size=bin_size, step=step, cap=cap,
1209         knn_k=k, alpha=alpha, beta=beta, gamma=gamma,
1210     )
1211     res_p3 = run_tower_phase1(
1212         real_path,
1213         p=3, n_min=1, n_max=n_max_p3,
1214         bin_size=bin_size, step=step, cap=cap,
1215         knn_k=k, alpha=alpha, beta=beta, gamma=gamma,
1216     )
1217 if save_prefix:
1218     save_results_csv(
1219         res_p2,
1220         f"{save_prefix}{config_name}_tower_real_bach_k{k}_p2{suffix}.csv",
1221         real_path,
1222         2,
1223         "kNN",
1224     )
1225     save_results_csv(
1226         res_p3,
1227         f"{save_prefix}{config_name}_tower_real_bach_k{k}_p3{suffix}.csv",
1228         real_path,
1229         3,
1230         "kNN",
1231     )
1232     plot_profile_compare(
1233         res_p2,
1234         res_p3,
1235         f"Phase1 {config_name} Bach k={k}" + (" (beats)" if use_beats else ""),
1236         save_prefix,
1237         f"{config_name}_real_bach_k{k}{suffix}",
1238     )
1239 # Control primes (e.g. p=5,7): run tower and save CSV; no plot.
1240 if control_primes:
1241     for p_val in control_primes:
1242         n_max_p = _n_max_for_prime(p_val)
1243         if use_beats:
1244             res_cp = run_tower_phase1(
1245                 real_path,
1246                 p=p_val, n_min=1, n_max=n_max_p,
1247                 bin_size=bin_size, step=step, cap=cap,

```

```

1248         knn_k=k, alpha=alpha, beta=beta, gamma=gamma,
1249         time_axis="beats",
1250         bin_size_beats=bin_size_beats,
1251         Tmax_ioi_beats=Tmax_ioi_beats,
1252     )
1253     else:
1254         res_cp = run_tower_phase1(
1255             real_path,
1256             p=p_val, n_min=1, n_max=n_max_p,
1257             bin_size=bin_size, step=step, cap=cap,
1258             knn_k=k, alpha=alpha, beta=beta, gamma=gamma,
1259         )
1260         save_results_csv(
1261             res_cp,
1262             f"{save_prefix}{config_name}_tower_real_bach_k{k}_p{p_val}{suffix}.csv",
1263             real_path,
1264             p_val,
1265             "kNN",
1266         )
1267
1268     beta0_p2_by_n = {r.n: r.beta0 for r in res_p2}
1269     beta0_p3_by_n = {r.n: r.beta0 for r in res_p3}
1270     giant_p2_by_n = {r.n: r.giant_component_frac for r in res_p2}
1271     giant_p3_by_n = {r.n: r.giant_component_frac for r in res_p3}
1272     delta_vec = []
1273     delta_G_vec = []
1274     ns_common = []
1275     for n in range(1, common_n_max + 1):
1276         if n in beta0_p2_by_n and n in beta0_p3_by_n:
1277             ns_common.append(n)
1278             delta_vec.append(beta0_p2_by_n[n] - beta0_p3_by_n[n])
1279             dg = giant_p2_by_n.get(n, 0) - giant_p3_by_n.get(n, 0)
1280             delta_G_vec.append(dg)
1281             if abs(dg) >= 0.10:
1282                 any_delta_G_ge_010 = True
1283                 config_any_delta_G_ge_010[config_name] = True
1284     for nn in range(1, common_n_max + 1):
1285         all_delta_nonzero_count.setdefault((config_name, nn), []).append(
1286             1 if (nn <= len(delta_vec) and delta_vec[nn - 1] != 0) else 0
1287         )
1288
1289     print(f"--- Phase1 Bach config={config_name} k={k}" + (" (beats)" if use_beats else "") + "
1290         ---")
1291     print("Delta(n) = beta0_p2(n)-beta0_p3(n):", delta_vec, "(n =", ns_common, ")")
1292     print("giant_component_frac p=2:", [round(giant_p2_by_n.get(n, 0), 4) for n in ns_common])
1293     print("giant_component_frac p=3:", [round(giant_p3_by_n.get(n, 0), 4) for n in ns_common])
1294     print("Delta_G(n):", [round(x, 4) for x in delta_G_vec])
1295
1296     if use_beats:
1297         report_lines.append(f"--- config={config_name} k={k} ---")
1298         report_lines.append("Delta(n): " + str(delta_vec))
1299         report_lines.append("Delta_G(n): " + str([round(x, 4) for x in delta_G_vec]))
1300         for r in res_p2:
1301             V = r.num_patterns
1302             kv = (k / V) if V > 0 else 0.0
1303             no_inf = "no_informativo" if kv > k_over_V_threshold else "ok"
1304             report_lines.append(f" p=2 n={r.n} V={V} k/V={round(kv, 4)} {no_inf}")
1305         for r in res_p3:

```

```

1305         V = r.num_patterns
1306         kv = (k / V) if V > 0 else 0.0
1307         no_inf = "no_informativo" if kv > k_over_V_threshold else "ok"
1308         report_lines.append(f" p=3 n={r.n} V={V} k/V={round(kv, 4)} {no_inf}")
1309
1310     if use_beats:
1311         config_success = 0
1312         for (cfg, n), counts in all_delta_nonzero_count.items():
1313             if cfg == config_name and sum(counts) >= 2:
1314                 config_success = 1
1315                 break
1316         if config_success == 0 and config_any_delta_G_ge_010.get(config_name, False):
1317             config_success = 1
1318         report_lines.append(f"ÉXITO config {config_name} = {config_success}")
1319         report_lines.append("")
1320
1321     success = 0
1322     for (config_name, n), counts in all_delta_nonzero_count.items():
1323         if sum(counts) >= 2:
1324             success = 1
1325             break
1326     if not success and any_delta_G_ge_010:
1327         success = 1
1328     print("===== Phase I Bach ÉXITO = " if not use_beats else "===== Phase I Bach (beats) É
        XITO =", success, "=====")
1329     if save_prefix:
1330         with open(report_filename, "w") as f:
1331             if use_beats:
1332                 f.write("Phase I Bach time-axis=beats: Delta(n), Delta_G(n) por (config,k); k/V por
                    nivel; no_informativo si k/V>0.15.\n\n")
1333                 f.write("\n".join(report_lines))
1334                 f.write(f"\nÉXITO global = {success}\n")
1335             else:
1336                 f.write("Phase I Bach: Delta(n), Delta_G(n), ÉXITO (see stdout for full vectors).\n")
1337                 f.write(f"ÉXITO = {success}\n")
1338
1339
1340 # -----
1341 # 6c) Phase II Bach: beat-based binning, alpha sweep, criterio binario, k/V
1342 # -----
1343
1344
1345 def run_phase2_bach_report(
1346     real_path: str,
1347     save_prefix: str,
1348     bin_size_beats: float = 1.0 / 12.0,
1349     step: int = 2,
1350     cap: int = 300,
1351     knn_k_list: Optional[List[int]] = None,
1352     alpha_list: Optional[List[float]] = None,
1353     n_max_p2: int = 6,
1354     n_max_p3: int = 5,
1355     n_min_report: int = 2,
1356     k_over_V_threshold: float = 0.15,
1357 ) -> None:
1358     """
1359     Phase II: beat-based binning H(u), a(u); baseline kNN for BWV1007.
1360     k=8,10,12; alpha in {0, 0.5, 1, 2, 4}; p=2 (n_max=6), p=3 (n_max=5).

```

```

1361     Report: Delta(n)=beta0_p2-beta0_p3, Delta_G(n) for n>=n_min_report; tables with k/V, no informativo
1362         if k/V>0.15.
1363     ÉXITO=1 si (existe n>=2 con Delta(n)≠0 en al menos 2 k) o (|Delta_G(n)|>=0.10 en algún n>=2).
1364     """
1365     if knn_k_list is None:
1366         knn_k_list = [8, 10, 12]
1367     if alpha_list is None:
1368         alpha_list = [0.0, 0.5, 1.0, 2.0, 4.0]
1369     common_n_max = min(n_max_p3, n_max_p2)
1370     all_delta_nonzero_count: Dict[Tuple[float, int], List[int]] = {} # (alpha, n) -> count of k with
1371         Delta(n)≠0
1372     any_delta_G_ge_010 = False
1373     for alpha in alpha_list:
1374         for k in knn_k_list:
1375             res_p2 = run_tower_on_beats(
1376                 real_path,
1377                 p=2,
1378                 n_min=1,
1379                 n_max=n_max_p2,
1380                 bin_size_beats=bin_size_beats,
1381                 step=step,
1382                 cap=cap,
1383                 knn_k=k,
1384                 alpha=alpha,
1385             )
1386             res_p3 = run_tower_on_beats(
1387                 real_path,
1388                 p=3,
1389                 n_min=1,
1390                 n_max=n_max_p3,
1391                 bin_size_beats=bin_size_beats,
1392                 step=step,
1393                 cap=cap,
1394                 knn_k=k,
1395                 alpha=alpha,
1396             )
1397             if save_prefix:
1398                 alpha_tag = str(alpha).replace(".", "p")
1399                 save_results_csv_phase2(
1400                     res_p2,
1401                     f"{save_prefix}phase2_bach_k{k}_a{alpha_tag}_p2.csv",
1402                     knn_k=k,
1403                     k_over_V_threshold=k_over_V_threshold,
1404                 )
1405                 save_results_csv_phase2(
1406                     res_p3,
1407                     f"{save_prefix}phase2_bach_k{k}_a{alpha_tag}_p3.csv",
1408                     knn_k=k,
1409                     k_over_V_threshold=k_over_V_threshold,
1410                 )
1411             beta0_p2_by_n = {r.n: r.beta0 for r in res_p2}
1412             beta0_p3_by_n = {r.n: r.beta0 for r in res_p3}
1413             giant_p2_by_n = {r.n: r.giant_component_frac for r in res_p2}
1414             giant_p3_by_n = {r.n: r.giant_component_frac for r in res_p3}
1415             delta_vec = []

```

```

1417 delta_G_vec = []
1418 ns_common = []
1419 for n in range(1, common_n_max + 1):
1420     if n in beta0_p2_by_n and n in beta0_p3_by_n:
1421         ns_common.append(n)
1422         d = beta0_p2_by_n[n] - beta0_p3_by_n[n]
1423         dg = giant_p2_by_n.get(n, 0) - giant_p3_by_n.get(n, 0)
1424         delta_vec.append(d)
1425         delta_G_vec.append(dg)
1426         if n >= n_min_report and abs(dg) >= 0.10:
1427             any_delta_G_ge_010 = True
1428 for nn in range(1, common_n_max + 1):
1429     idx = nn - 1
1430     delta_val = delta_vec[idx] if idx < len(delta_vec) else 0
1431     all_delta_nonzero_count.setdefault((alpha, nn), []).append(
1432         1 if (idx < len(delta_vec) and delta_val != 0) else 0
1433     )
1434
1435 print(f"--- Phase II Bach k={k} alpha={alpha} ---")
1436 print("Delta(n) = beta0_p2(n)-beta0_p3(n):", delta_vec, "(n =", ns_common, ")")
1437 print("Delta_G(n):", [round(x, 4) for x in delta_G_vec])
1438 # Table with k/V and no informativo for n>=1
1439 print("n\tN\tV\tbeta0\tgiant_frac\tk/V\tno_informativo")
1440 for r in res_p2:
1441     V = r.num_patterns
1442     k_over_V = (k / V) if V > 0 else 0.0
1443     no_inf = "yes" if k_over_V > k_over_V_threshold else "no"
1444     print(f"p=2
1445           {r.n}\t{r.N}\t{V}\t{r.beta0}\t{r.giant_component_frac:.4f}\t{k_over_V:.4f}\t{no_inf}")
1446 for r in res_p3:
1447     V = r.num_patterns
1448     k_over_V = (k / V) if V > 0 else 0.0
1449     no_inf = "yes" if k_over_V > k_over_V_threshold else "no"
1450     print(f"p=3
1451           {r.n}\t{r.N}\t{V}\t{r.beta0}\t{r.giant_component_frac:.4f}\t{k_over_V:.4f}\t{no_inf}")
1452
1453 success = 0
1454 for (alpha, n), counts in all_delta_nonzero_count.items():
1455     if n >= n_min_report and sum(counts) >= 2:
1456         success = 1
1457         break
1458 if not success and any_delta_G_ge_010:
1459     success = 1
1460 print("===== Phase II Bach ÉXITO =", success, "=====")
1461 if save_prefix:
1462     with open(f"{save_prefix}phase2_bach_report.txt", "w") as f:
1463         f.write("Phase II Bach (beat-based): Delta(n), Delta_G(n), k/V, no informativo
1464             (k/V>0.15).\n")
1465         f.write(f"ÉXITO = {success}\n")
1466
1467 # -----
1468 # 7) Reporte final: tablas + 3 bullets (sin interpretación musical fuerte)
1469 # -----
1470
1471 def report_baseline_final(
1472     baseline_runs: List[Tuple[str, int, List[LevelResult], List[LevelResult]]],

```

```

1472     threshold_mode: str,
1473     quantile_q: float,
1474     knn_k_list: List[int],
1475     alpha: float = 1.0,
1476 ) -> List[str]:
1477     """Reporte solo hechos cuantitativos: estabilidad en k, separación p=2 vs p=3 en toys, Bach."""
1478     lines = [
1479         "===== REPORTE BASELINE (solo hechos cuantitativos) =====",
1480         f"Threshold: {threshold_mode}; k values: {knn_k_list}; alpha (onset density): {alpha}.",
1481         "",
1482     ]
1483     # Tablas resumidas por (source, k): p=2 y p=3
1484     for source_label, k, res_p2, res_p3 in baseline_runs:
1485         lines.append(f"--- {source_label} k={k} ---")
1486         lines.append("p=2: n\tN\tV\tE\tavg_degree\tbeta0\tgiant_component_frac")
1487         for r in res_p2:
1488             lines.append(f"
1489                 {r.n}\t{r.N}\t{r.num_patterns}\t{r.num_edges}\t{r.avg_degree:.4f}\t{r.beta0}\t{r.giant_component_frac:.4f}")
1490         lines.append("p=3: n\tN\tV\tE\tavg_degree\tbeta0\tgiant_component_frac")
1491         for r in res_p3:
1492             lines.append(f"
1493                 {r.n}\t{r.N}\t{r.num_patterns}\t{r.num_edges}\t{r.avg_degree:.4f}\t{r.beta0}\t{r.giant_component_frac:.4f}")
1494         lines.append("")
1495     lines.append("--- Hechos cuantitativos ---")
1496     # 1) Estabilidad respecto a k
1497     by_source: Dict[str, List[Tuple[int, List[LevelResult], List[LevelResult]]]] = {}
1498     for source_label, k, res_p2, res_p3 in baseline_runs:
1499         by_source.setdefault(source_label, []).append((k, res_p2, res_p3))
1500     stab = []
1501     for source_label, k_runs in by_source.items():
1502         if len(k_runs) < 2:
1503             stab.append(f"• {source_label}: solo un k, no se evalúa estabilidad.")
1504             continue
1505         ns = [r.n for r in k_runs[0][1]]
1506         b0_curves = [[r.beta0 for r in res_p2] for _, res_p2, _ in k_runs]
1507         same_shape = all(len(c) == len(ns) for c in b0_curves)
1508         b0_range = [max(b0_curves[i][j] for i in range(len(k_runs))) - min(b0_curves[i][j] for i in
1509             range(len(k_runs))) for j in range(len(ns))]
1510         max_diff = max(b0_range) if b0_range else 0
1511         stab.append(f"• {source_label}: curvas beta0(n) para k={x[0] for x in k_runs}; max diferencia
1512             entre k en algún n = {max_diff}. {'Curvas similares.' if max_diff <= 2 else 'Curvas
1513             difieren.'}")
1514     lines.extend(stab)
1515     # 2) Separación p=2 vs p=3 en toys (esperado: toy_binary favorece p=2; toy_ternary favorece p=3)
1516     for source_label, k_runs in by_source.items():
1517         if source_label not in ("toy_binary", "toy_ternary"):
1518             continue
1519         _, res_p2, res_p3 = k_runs[0]
1520         b0_p2 = [r.beta0 for r in res_p2]
1521         b0_p3 = [r.beta0 for r in res_p3]
1522         # toy_binary "favorece" p=2 si en algún nivel p=2 tiene menos componentes que p=3 (más
1523             conectado)
1524         p2_menos = sum(1 for i in range(min(len(b0_p2), len(b0_p3))) if b0_p2[i] < b0_p3[i])
1525         p3_menos = sum(1 for i in range(min(len(b0_p2), len(b0_p3))) if b0_p2[i] > b0_p3[i])
1526         if source_label == "toy_binary":

```

```

1523         lines.append(f"• toy_binary: en {p2_menos} niveles beta0(p=2)<beta0(p=3), en {p3_menos}
           niveles beta0(p=2)>beta0(p=3). Esperado: más niveles con p=2 más conectado (p=2
           favorecido).")
1524     else:
1525         lines.append(f"• toy_ternary: en {p2_menos} niveles beta0(p=2)<beta0(p=3), en {p3_menos}
           niveles beta0(p=2)>beta0(p=3). Esperado: más niveles con p=3 más conectado (p=3
           favorecido).")
1526
1527     # 3) Bach: diferencia consistente entre p=2 y p=3
1528     bach_runs = [(k, r2, r3) for src, k, r2, r3 in baseline_runs if src == "real_bach"]
1529     if bach_runs:
1530         _, res_p2, res_p3 = bach_runs[0]
1531         b0_2 = [r.beta0 for r in res_p2]
1532         b0_3 = [r.beta0 for r in res_p3]
1533         diff = [b0_2[i] - b0_3[i] for i in range(min(len(b0_2), len(b0_3)))]
1534         lines.append(f"• Bach real: diferencia beta0(p=2)-beta0(p=3) por nivel n: {diff}. Diferencia
           consistente (mismo signo en todos los niveles): {'sí' if all(d >= 0 for d in diff) or
           all(d <= 0 for d in diff) else 'no'}.")
1535     else:
1536         lines.append("• Bach real: no ejecutado.")
1537     return lines
1538
1539
1540 # -----
1541 # 8) Main: synthetic tests, optional real MIDI
1542 # -----
1543
1544
1545 def main() -> None:
1546     parser = argparse.ArgumentParser(
1547         description="Profinite Echo: precision tower on MIDI chroma (p=2 vs p=3)"
1548     )
1549     parser.add_argument(
1550         "midi",
1551         nargs="?",
1552         default="",
1553         help="Optional: path to real MIDI (if omitted, only synthetic runs)",
1554     )
1555     parser.add_argument(
1556         "--no-synthetic",
1557         action="store_true",
1558         help="Skip synthetic MIDI generation and tests",
1559     )
1560     parser.add_argument(
1561         "--phase1-only",
1562         action="store_true",
1563         help="Run only Phase I (Bach, configs A/B/C) and skip baseline tower",
1564     )
1565     parser.add_argument(
1566         "--phase2-only",
1567         action="store_true",
1568         help="Run only Phase II (Bach, beat-based binning, alpha sweep)",
1569     )
1570     parser.add_argument(
1571         "--bin-size",
1572         type=float,
1573         default=0.05,
1574         help="Chroma bin size in seconds (default 0.05)",

```

```

1575 )
1576 parser.add_argument(
1577     "--time-axis",
1578     type=str,
1579     choices=["seconds", "beats"],
1580     default="seconds",
1581     help="Time axis for binning: seconds (default) or beats",
1582 )
1583 parser.add_argument(
1584     "--bin-beats",
1585     type=float,
1586     default=1.0 / 12.0,
1587     help="Bin size in beats, only if --time-axis=beats (default 1/12)",
1588 )
1589 parser.add_argument(
1590     "--bin-size-beats",
1591     type=float,
1592     default=1.0 / 12.0,
1593     help="Phase II: bin size in beats (default 1/12)",
1594 )
1595 parser.add_argument(
1596     "--step",
1597     type=int,
1598     default=2,
1599     help="Window slide step (default 2)",
1600 )
1601 parser.add_argument(
1602     "--delta",
1603     type=float,
1604     default=0.35,
1605     help="Proximity threshold only when --threshold=fixed (default 0.35)",
1606 )
1607 parser.add_argument(
1608     "--threshold",
1609     type=str,
1610     choices=["quantile", "knn", "fixed"],
1611     default="knn",
1612     help="knn (default): KNN graph; quantile: delta_n=Q_q; fixed: delta fijo",
1613 )
1614 parser.add_argument(
1615     "--quantile",
1616     type=float,
1617     default=0.05,
1618     help="Quantile q for --threshold=quantile (default 0.05)",
1619 )
1620 parser.add_argument(
1621     "--sample-size",
1622     type=int,
1623     default=5000,
1624     help="Max random pairs for quantile sampling (default 5000)",
1625 )
1626 parser.add_argument(
1627     "--knn-k",
1628     type=int,
1629     nargs="+",
1630     default=[8, 10, 12],
1631     help="k values for --threshold=knn (default: 8 10 12)",
1632 )

```

```

1633     parser.add_argument(
1634         "--alpha",
1635         type=float,
1636         default=1.0,
1637         help="Weight for onset density in  $X(t)=[H(t); \alpha*a(t)]$  (default 1.0)",
1638     )
1639     parser.add_argument(
1640         "--n-max",
1641         type=int,
1642         default=6,
1643         help="Max tower level n (default 6)",
1644     )
1645     parser.add_argument(
1646         "--cap",
1647         type=int,
1648         default=300,
1649         help="Cap number of patterns per level for graph build (default 300)",
1650     )
1651     parser.add_argument(
1652         "--control-primes",
1653         type=int,
1654         nargs="*",
1655         default=[],
1656         help="Additional primes for Phase I (e.g. 5 7) as negative controls; default p=2,3 unchanged",
1657     )
1658     parser.add_argument(
1659         "--save",
1660         type=str,
1661         default="",
1662         help="Save plots and CSV to this prefix (e.g. out_) instead of showing",
1663     )
1664     args = parser.parse_args()
1665
1666     n_min, n_max = 1, args.n_max
1667     bin_size = args.bin_size
1668     step = args.step
1669     delta = args.delta
1670     cap = args.cap
1671     threshold_mode = args.threshold
1672     quantile_q = args.quantile
1673     sample_size = args.sample_size
1674     knn_k_list = args.knn_k if isinstance(args.knn_k, list) else [args.knn_k]
1675     if threshold_mode != "knn":
1676         knn_k_list = [knn_k_list[0]]
1677     alpha = getattr(args, "alpha", 1.0)
1678     save_prefix = args.save.strip() or None
1679     real_path = args.midi.strip()
1680     if not real_path and Path("bwv1007_prelude.mid").exists():
1681         real_path = "bwv1007_prelude.mid"
1682
1683     # Baseline: (source_label, k, res_p2, res_p3) para reporte
1684     baseline_runs: List[Tuple[str, int, List[LevelResult], List[LevelResult]]] = []
1685
1686     if args.phase1_only:
1687         if real_path and Path(real_path).exists():
1688             run_phase1_bach_report(
1689                 real_path,
1690                 save_prefix or "",

```

```

1691         bin_size=bin_size,
1692         step=step,
1693         cap=cap,
1694         knn_k_list=knn_k_list,
1695         n_max_p2=n_max,
1696         n_max_p3=min(5, n_max),
1697         time_axis=args.time_axis,
1698         bin_size_beats=args.bin_size_beats,
1699         control_primes=getattr(args, "control_primes", None) or [],
1700     )
1701     else:
1702         print("Phase I requires a valid MIDI path (e.g. bwv1007_prelude.mid).")
1703     return
1704
1705 if args.phase2_only:
1706     if real_path and Path(real_path).exists():
1707         run_phase2_bach_report(
1708             real_path,
1709             save_prefix or "",
1710             bin_size_beats=args.bin_size_beats,
1711             step=step,
1712             cap=cap,
1713             knn_k_list=knn_k_list,
1714             alpha_list=[0.0, 0.5, 1.0, 2.0, 4.0],
1715             n_max_p2=n_max,
1716             n_max_p3=min(5, n_max),
1717         )
1718     else:
1719         print("Phase II requires a valid MIDI path (e.g. bwv1007_prelude.mid).")
1720     return
1721
1722 if not args.no_synthetic:
1723     write_synthetic_midi("toy_binary.mid", pattern="binary", bars=8)
1724     write_synthetic_midi("toy_ternary.mid", pattern="ternary", bars=8)
1725
1726 sources: List[Tuple[str, str, bool]] = [] # (path, label, show_heatmap)
1727 if not args.no_synthetic:
1728     sources.append(("toy_binary.mid", "toy_binary", True))
1729     sources.append(("toy_ternary.mid", "toy_ternary", True))
1730 if real_path and Path(real_path).exists():
1731     sources.append((real_path, "real_bach", not args.no_synthetic))
1732
1733 for ki, k in enumerate(knn_k_list):
1734     for path, source_label, show_heatmap in sources:
1735         save_heat = save_prefix and ki == 0
1736         res_p2 = run_tower_on_midi(
1737             path,
1738             p=2,
1739             n_min=n_min,
1740             n_max=n_max,
1741             bin_size=bin_size,
1742             step=step,
1743             delta=delta,
1744             cap=cap,
1745             threshold_mode=threshold_mode,
1746             quantile_q=quantile_q,
1747             sample_size=sample_size,
1748             knn_k=k,

```

```

1749         alpha=alpha,
1750         show_heatmap=show_heatmap and save_prefix is None,
1751         save_heatmap=f"{save_prefix}chroma_{source_label}.png" if save_heat else None,
1752     )
1753     res_p3 = run_tower_on_midi(
1754         path,
1755         p=3,
1756         n_min=1,
1757         n_max=min(5, n_max),
1758         bin_size=bin_size,
1759         step=step,
1760         delta=delta,
1761         cap=cap,
1762         threshold_mode=threshold_mode,
1763         quantile_q=quantile_q,
1764         sample_size=sample_size,
1765         knn_k=k,
1766         alpha=alpha,
1767         show_heatmap=False,
1768     )
1769     baseline_runs.append((source_label, k, res_p2, res_p3))
1770     if save_prefix:
1771         save_results_csv(res_p2, f"{save_prefix}tower_{source_label}_k{k}_p2.csv", path, 2,
1772                         threshold_mode)
1773         save_results_csv(res_p3, f"{save_prefix}tower_{source_label}_k{k}_p3.csv", path, 3,
1774                         threshold_mode)
1775         plot_profile_compare(res_p2, res_p3, f"{source_label} k={k}", save_prefix,
1776                             f"{source_label}_k{k}")
1777         print(f"--- {source_label} k={k} ---")
1778         print("p=2:", [f"n={r.n} |V|={r.num_patterns} beta0={r.beta0} avg_deg={r.avg_degree:.2f}"
1779                       giant_frac={r.giant_component_frac:.3f}" for r in res_p2])
1780         print("p=3:", [f"n={r.n} beta0={r.beta0} avg_deg={r.avg_degree:.2f}"
1781                       giant_frac={r.giant_component_frac:.3f}" for r in res_p3])
1782
1783     if not sources:
1784         print("No sources to run (use --no-synthetic only if you provide a MIDI path).")
1785     if real_path:
1786         print("Real MIDI path not found:", real_path)
1787
1788     report_lines = report_baseline_final(baseline_runs, threshold_mode, quantile_q, knn_k_list, alpha)
1789     for line in report_lines:
1790         print(line)
1791     if save_prefix:
1792         with open(f"{save_prefix}report.txt", "w") as f:
1793             f.write("\n".join(report_lines))
1794
1795     # Phase I (multi-observable): solo Bach, configs A/B/C, reporte Delta, Delta_G, ÉXITO
1796     if real_path and Path(real_path).exists():
1797         run_phasel_bach_report(
1798             real_path,
1799             save_prefix or "",
1800             bin_size=bin_size,
1801             step=step,
1802             cap=cap,
1803             knn_k_list=knn_k_list,
1804             n_max_p2=n_max,
1805             n_max_p3=min(5, n_max),
1806             control_primes=getattr(args, "control_primes", None) or [],

```

```

1802     )
1803     # Phase II (beat-based): Bach, alpha sweep, reporte Delta/Delta_G, k/V, ÉXITO
1804     run_phase2_bach_report(
1805         real_path,
1806         save_prefix or "",
1807         bin_size_beats=args.bin_size_beats,
1808         step=step,
1809         cap=cap,
1810         knn_k_list=knn_k_list,
1811         alpha_list=[0.0, 0.5, 1.0, 2.0, 4.0],
1812         n_max_p2=n_max,
1813         n_max_p3=min(5, n_max),
1814     )
1815
1816
1817 if __name__ == "__main__":
1818     main()

```

A.2.3 src/padicmidi/core/hierarchical.py

```

1  #!/usr/bin/env python3
2  """
3  padicmidi.core.hierarchical – p-adic tower with forced inverse system.
4
5  Hierarchical dictionaries ``S_n`` together with the explicit inverse map
6  ``pi_{n+1,n}: S_{n+1} -> S_n`` that forces each child to inherit a parent
7  by construction. Computes the coherence invariants
8  ``Coh_pi(p, n)`` and ``Coh_grid(p, n)`` and writes the per-parent audit
9  table that allows verification of hypotheses (SC) and (AI) of the
10 null-floor proposition.
11
12 Construction summary:
13 * ``S_1`` is obtained via K-means on a sample of windows of length p.
14 * For every ``n -> n+1`` the parent of an aggregated window
15 ``W_{p,n+1}(b)`` is its truncation projected onto ``S_n``; per-parent
16 K-means with ``Kchild`` clusters yields ``S_{n+1}``;
17 ``pi(child_id) = parent_id`` by construction.
18
19 Outputs (one file per CSV):
20 * ``params.json``, ``params.txt``
21 * ``S_n_prototypes_{p}_{n}.csv``
22 * ``pi_{p}_{n+1}_to_n.csv``
23 * ``f_{p}_{n}.csv``
24 * ``coherence_hier_{p}.csv``
25 * ``audit_{p}.csv``
26 """
27
28 from __future__ import annotations
29
30 import argparse
31 import csv
32 import json
33 from pathlib import Path
34 from collections import defaultdict
35
36 import numpy as np

```

```

37
38 from padicmidi.core.echo import (
39     parse_midi_notes_seconds,
40     parse_midi_notes_beats,
41     chroma_series_duration,
42     chroma_series_duration_beats,
43     onset_density_series,
44     onset_density_series_beats,
45     series_with_rhythm,
46 )
47 from padicmidi.core.config import ALPHA, MAX_WINDOWS_PER_RESIDUE, default_nmax
48
49
50 def build_X_seconds(path: str, bin_size: float = 0.05) -> np.ndarray:
51     events = parse_midi_notes_seconds(path)
52     if not events:
53         return np.zeros((0, 13), dtype=float)
54     H = chroma_series_duration(events, bin_size=bin_size)
55     a = onset_density_series(events, bin_size=bin_size)
56     return series_with_rhythm(H, a, alpha=ALPHA)
57
58
59 def build_X_beats(path: str, bin_size_beats: float = 1.0 / 12.0) -> np.ndarray:
60     events = parse_midi_notes_beats(path)
61     if not events:
62         return np.zeros((0, 13), dtype=float)
63     H = chroma_series_duration_beats(events, bin_size_beats=bin_size_beats)
64     a = onset_density_series_beats(events, bin_size_beats=bin_size_beats)
65     return series_with_rhythm(H, a, alpha=ALPHA)
66
67
68 def get_windows(X: np.ndarray, N: int, step: int) -> list[tuple[int, np.ndarray]]:
69     out = []
70     for start in range(0, len(X) - N + 1, step):
71         out.append((start, X[start : start + N, :].copy()))
72     return out
73
74
75 def aggregate_median(windows: list[np.ndarray]) -> np.ndarray:
76     """Componentwise median;  $W_{\{p,n\}}(a)$  = median over windows with  $i \equiv a \pmod{p^n}$ ."""
77     if not windows:
78         return np.full((1, 1), np.nan)
79     stack = np.stack(windows, axis=0)
80     return np.median(stack, axis=0).astype(np.float64)
81
82
83 def dist_matrix(windows: np.ndarray, prototypes: np.ndarray) -> np.ndarray:
84     """windows (W, N, 13), prototypes (P, N, 13) -> (W, P)."""
85     diff = windows[:, None, :, :] - prototypes[None, :, :, :]
86     return np.sqrt(np.mean(np.sum(diff ** 2, axis=3), axis=2))
87
88
89 def kmeans_numpy(samples: np.ndarray, K: int, rng: np.random.Generator, max_iter: int = 30) ->
90     np.ndarray:
91     """samples (N, D). Returns centers (K_eff, D) with  $K_{eff} = \max(1, \min(K, \text{len}(\text{samples})))$ ."""
92     if len(samples) == 0:
93         return np.zeros((K, samples.shape[1]) if samples.size else (K, 0))
94     K_actual = max(1, min(K, len(samples)))

```

```

94     idx = np.arange(len(samples))
95     rng.shuffle(idx)
96     centers = samples[idx[:K_actual]].copy()
97     if K_actual < K:
98         centers = np.vstack([centers, np.tile(centers[-1], (K - K_actual, 1))])
99     for _ in range(max_iter):
100         dists = np.linalg.norm(samples[:, None, :] - centers[None, :, :], axis=2)
101         labels = np.argmin(dists, axis=1)
102         new_centers = centers.copy()
103         for j in range(K_actual):
104             mask = labels == j
105             if np.any(mask):
106                 new_centers[j] = samples[mask].mean(axis=0)
107         if np.allclose(new_centers, centers):
108             break
109         centers = new_centers
110     return centers[:K_actual]
111
112
113 def build_W_n(X: np.ndarray, p: int, n_max: int, step: int) -> dict[int, dict[int, np.ndarray]]:
114     """W_n[n][a] = aggregated window (p^n, 13) for residue a. Median over { P_{n,i} : i ≡ a (mod p^n)
115     }. """
116     W_n = {}
117     for n in range(1, n_max + 1):
118         N = p ** n
119         windows_with_start = get_windows(X, N, step)
120         if not windows_with_start:
121             W_n[n] = {}
122             continue
123         by_residue = defaultdict(list)
124         for start, w in windows_with_start:
125             a = start % (p ** n)
126             if len(by_residue[a]) < MAX_WINDOWS_PER_RESIDUE:
127                 by_residue[a].append(w)
128             W_n[n] = {}
129             for a, wlist in by_residue.items():
130                 W_n[n][a] = aggregate_median(wlist)
131     return W_n
132
133 def run_hierarchical(
134     X: np.ndarray,
135     p: int,
136     Nmax: int,
137     step: int,
138     K: int,
139     Kchild: int,
140     M: int,
141     rng: np.random.Generator,
142 ) -> tuple[dict, dict, dict, list[dict], list[dict]]:
143     """
144     Returns:
145     prototypes_n: n -> (|S_n|, p^n, 13)
146     f_n: n -> dict residue a -> prototype_id
147     pi_maps: n -> list of length |S_{n+1}|, pi_maps[n][child_id] = parent_id
148     coherence_rows: list of {n, Coh_pi, Coh_grid, n_samples_n, n_samples_nplus1}
149     audit_rows: per-parent diagnostics for the quantities underlying Coh_pi
150                 (pi columns) and Coh_grid (trunc columns).

```

```

151     """
152     W_n = build_W_n(X, p, Nmax, step)
153     prototypes_n = {}
154     f_n = {}
155     pi_maps = {}
156
157     N1 = p
158     win1 = get_windows(X, N1, step)
159     if not win1:
160         return prototypes_n, f_n, pi_maps, [], []
161
162     indices = np.arange(len(win1))
163     if len(indices) > M:
164         indices = np.linspace(0, len(indices) - 1, M).astype(int)
165     sample1 = np.array([win1[i][1].flatten() for i in indices])
166     K1 = max(1, min(K, len(sample1)))
167     centers1 = kmeans_numpy(sample1, K1, rng)
168     prototypes_n[1] = centers1.reshape(-1, N1, 13)
169     f_n[1] = {}
170     if W_n.get(1):
171         keys, wstack = zip(*[(a, w) for a, w in W_n[1].items() if not np.any(np.isnan(w))])
172         if keys:
173             wstack = np.stack(wstack)
174             d = dist_matrix(wstack, prototypes_n[1])
175             for i, a in enumerate(keys):
176                 f_n[1][a] = int(np.argmin(d[i]))
177
178     for n in range(1, Nmax):
179         pn = p ** n
180         pn1 = p ** (n + 1)
181         if n + 1 not in W_n or not W_n[n + 1]:
182             continue
183         b_list, w_list = zip(*[(b, w) for b, w in W_n[n + 1].items() if not np.any(np.isnan(w))])
184         if not b_list:
185             continue
186         w_list = np.stack(w_list)
187         trunc_list = w_list[:, :pn, :]
188         d_parent = dist_matrix(trunc_list, prototypes_n[n])
189         parent_of_b = {b: int(np.argmin(d_parent[i])) for i, b in enumerate(b_list)}
190
191         by_parent = defaultdict(list)
192         for b, w in W_n[n + 1].items():
193             if b not in parent_of_b or np.any(np.isnan(w)):
194                 continue
195             by_parent[parent_of_b[b]].append(w)
196
197         n_parents = len(prototypes_n[n])
198         all_children = []
199         pi_list = []
200         for j in range(n_parents):
201             wlist = by_parent.get(j, [])
202             K_eff = max(1, min(Kchild, len(wlist))) if wlist else 1
203             if len(wlist) < 2:
204                 c = wlist[0].copy() if len(wlist) == 1 else np.repeat(
205                     prototypes_n[n][j], (pn1 + pn - 1) // pn, axis=0
206                 )[:pn1, :].copy()
207             for _ in range(K_eff):
208                 all_children.append(c.copy())

```

```

209         pi_list.append(j)
210     for _ in range(Kchild - K_eff):
211         all_children.append(all_children[-1].copy())
212         pi_list.append(j)
213     else:
214         flat = np.array([x.flatten() for x in wlist])
215         centers_child = kmeans_numpy(flat, K_eff, rng)
216         for k in range(len(centers_child)):
217             all_children.append(centers_child[k].reshape(pn1, 13))
218             pi_list.append(j)
219         for _ in range(Kchild - len(centers_child)):
220             all_children.append(all_children[-1].copy())
221             pi_list.append(j)
222     if not all_children:
223         continue
224     prototypes_n[n + 1] = np.stack(all_children, axis=0)
225     pi_maps[n] = pi_list
226     keys_n1, w_n1 = zip(*[(a, w) for a, w in W_n[n + 1].items() if not np.any(np.isnan(w))])
227     if keys_n1:
228         f_n[n + 1] = {}
229         w_stack_n1 = np.stack(w_n1)
230         d_n1 = dist_matrix(w_stack_n1, prototypes_n[n + 1])
231         for i, a in enumerate(keys_n1):
232             f_n[n + 1][a] = int(np.argmin(d_n1[i]))
233
234     coherence_rows = []
235     audit_rows = []
236     for n in range(1, Nmax):
237         if n not in pi_maps or n + 1 not in f_n or n not in f_n:
238             continue
239         pn = p ** n
240         pn1 = p ** (n + 1)
241         valid_b_list = [
242             b for b in range(pn1)
243             if b in W_n.get(n + 1, {}) and not np.any(np.isnan(W_n[n + 1][b]))
244             and f_n[n + 1].get(b) is not None and (b % pn) in f_n[n]
245         ]
246         if not valid_b_list:
247             coherence_rows.append({
248                 "n": n, "Coh_pi": 0.0, "Coh_grid": 0.0,
249                 "n_samples_n": len(W_n.get(n, {})), "n_samples_nplus1": 0,
250             })
251             continue
252         w_stack = np.stack([W_n[n + 1][b] for b in valid_b_list])
253         trunc_stack = w_stack[:, :pn, :]
254         d_trunc = dist_matrix(trunc_stack, prototypes_n[n])
255         q_n_trunc = np.argmin(d_trunc, axis=1)
256         q_n1_arr = np.array([f_n[n + 1][b] for b in valid_b_list], dtype=int)
257         parent_via_pi = np.array([pi_maps[n][j] for j in q_n1_arr])
258         f_n_r_b = np.array([f_n[n][b % pn] for b in valid_b_list], dtype=int)
259         match_pi = int((parent_via_pi == f_n_r_b).sum())
260         match_grid = int((q_n_trunc == f_n_r_b).sum())
261         coh_pi = match_pi / pn1 if pn1 > 0 else 0.0
262         coh_grid = match_grid / pn1 if pn1 > 0 else 0.0
263         coherence_rows.append({
264             "n": n,
265             "Coh_pi": round(coh_pi, 6),
266             "Coh_grid": round(coh_grid, 6),

```

```

267         "n_samples_n": len(W_n.get(n, {})),
268         "n_samples_nplus1": len(valid_b_list),
269     })
270
271     valid_index = {b: i for i, b in enumerate(valid_b_list)}
272     for a in range(pn):
273         siblings = [a + k * pn for k in range(p)]
274         valid_siblings = [b for b in siblings if b in valid_index]
275         if not valid_siblings or a not in f_n[n]:
276             audit_rows.append({
277                 "n": n,
278                 "parent_class": a,
279                 "n_valid_siblings": len(valid_siblings),
280                 "excluded_sparsity": 1,
281                 "V_SC_pi": "",
282                 "AI_pi": "",
283                 "coherent_count_pi": "",
284                 "V_SC_trunc": "",
285                 "AI_trunc": "",
286                 "coherent_count_trunc": "",
287                 "parent_prototype": f_n[n].get(a, ""),
288             })
289             continue
290
291         idx = [valid_index[b] for b in valid_siblings]
292         parent_proto = int(f_n[n][a])
293         pi_vals = [int(parent_via_pi[i]) for i in idx]
294         trunc_vals = [int(q_n_trunc[i]) for i in idx]
295         audit_rows.append({
296             "n": n,
297             "parent_class": a,
298             "n_valid_siblings": len(valid_siblings),
299             "excluded_sparsity": int(len(valid_siblings) < p),
300             "V_SC_pi": len(set(pi_vals)),
301             "AI_pi": int(parent_proto in set(pi_vals)),
302             "coherent_count_pi": sum(1 for x in pi_vals if x == parent_proto),
303             "V_SC_trunc": len(set(trunc_vals)),
304             "AI_trunc": int(parent_proto in set(trunc_vals)),
305             "coherent_count_trunc": sum(1 for x in trunc_vals if x == parent_proto),
306             "parent_prototype": parent_proto,
307         })
308     return prototypes_n, f_n, pi_maps, coherence_rows, audit_rows
309
310
311 def main() -> None:
312     ap = argparse.ArgumentParser(description="Hierarchical profinite maps with  $\pi_{\{n+1,n\}}$ ")
313     ap.add_argument("midi", help="Path to MIDI file")
314     ap.add_argument("--axis", choices=["seconds", "beats"], default="beats")
315     ap.add_argument("--bin", type=float, default=0.05, help="Bin size (seconds axis)")
316     ap.add_argument("--bin-beats", type=float, default=1.0 / 12.0, help="Bin size (beats)")
317     ap.add_argument("--p", type=int, required=True, help="Prime (2,3,5,7)")
318     ap.add_argument("--Nmax", type=int, default=None)
319     ap.add_argument("--K", type=int, default=16)
320     ap.add_argument("--Kchild", type=int, default=2, help="Children per parent")
321     ap.add_argument("--M", type=int, default=800)
322     ap.add_argument("--step", type=int, default=2)
323     ap.add_argument("--seed", type=int, default=42)
324     ap.add_argument("--out", required=True, help="Output directory (e.g. .../p2/)")

```

```

325     ap.add_argument("--audit-only", action="store_true", help="Only write audit_p{p}.csv; leave
        existing coherence/prototype files untouched")
326     ap.add_argument("--force", action="store_true", help="Overwrite audit_p{p}.csv when --audit-only is
        used")
327     args = ap.parse_args()
328
329     if args.p not in (2, 3, 5, 7):
330         raise SystemExit("--p must be 2, 3, 5, or 7")
331     Nmax = args.Nmax if args.Nmax is not None else default_nmax(args.p)
332     out_dir = Path(args.out)
333     out_dir.mkdir(parents=True, exist_ok=True)
334     audit_path = out_dir / f"audit_p{args.p}.csv"
335     if args.audit_only and audit_path.exists() and not args.force:
336         print(f"Audit exists, skipping (use --force to overwrite): {audit_path}", file=sys.stderr)
337         return
338
339     if args.axis == "seconds":
340         X = build_X_seconds(args.midi, bin_size=args.bin)
341     else:
342         X = build_X_beats(args.midi, bin_size_beats=args.bin_beats)
343
344     if len(X) < 2:
345         raise SystemExit("Series too short")
346
347     rng = np.random.default_rng(args.seed)
348     prototypes_n, f_n, pi_maps, coherence_rows, audit_rows = run_hierarchical(
349         X, args.p, Nmax, args.step, args.K, args.Kchild, args.M, rng
350     )
351
352     p = args.p
353     # params
354     params = {
355         "piece": str(Path(args.midi).name),
356         "axis": args.axis,
357         "p": p,
358         "Nmax": Nmax,
359         "K": args.K,
360         "Kchild": args.Kchild,
361         "M": args.M,
362         "step": args.step,
363         "seed": args.seed,
364         "bin_seconds": args.bin,
365         "bin_beats": args.bin_beats,
366     }
367     if not args.audit_only:
368         with open(out_dir / "params.json", "w") as f:
369             json.dump(params, f, indent=2)
370         with open(out_dir / "params.txt", "w") as f:
371             f.write(f"piece={params['piece']} axis={args.axis} p={p} Nmax={Nmax} K={args.K}
                Kchild={args.Kchild} M={args.M} step={args.step} seed={args.seed}\n")
372
373     # S_n_prototypes_p{p}_n{n}.csv (rows = prototypes, flattened)
374     for n, prot in prototypes_n.items():
375         flat = prot.reshape(prot.shape[0], -1)
376         path = out_dir / f"S_n_prototypes_p{p}_n{n}.csv"
377         np.savetxt(path, flat, delimiter=",")
378     # pi_p{p}_n{n+1}_to_n{n}.csv
379     for n, pi_list in pi_maps.items():

```

```

380     path = out_dir / f"pi_p{p}_n{n+1}_to_n{n}.csv"
381     with open(path, "w", newline="") as f:
382         w = csv.writer(f)
383         w.writerow(["child_id", "parent_id"])
384         for cid, pid in enumerate(pi_list):
385             w.writerow([cid, pid])
386     # f_p{p}_n{n}.csv
387     for n, fn in f_n.items():
388         path = out_dir / f"f_p{p}_n{n}.csv"
389         with open(path, "w", newline="") as f:
390             w = csv.writer(f)
391             w.writerow(["residue", "prototype_id"])
392             for a in sorted(fn.keys()):
393                 w.writerow([a, fn[a]])
394
395     # coherence_hier_p{p}.csv
396     coh_path = out_dir / f"coherence_hier_p{p}.csv"
397     with open(coh_path, "w", newline="") as f:
398         w = csv.DictWriter(f, fieldnames=["n", "Coh_pi", "Coh_grid", "n_samples_n",
399                                         "n_samples_nplus1"])
400         w.writeheader()
401         w.writerows(coherence_rows)
402     else:
403         coh_path = out_dir / f"coherence_hier_p{p}.csv"
404
405     # audit_p{p}.csv stores per-parent diagnostics for both quantities:
406     # pi-columns recompute Coh_pi, trunc-columns recompute Coh_grid.
407     audit_fields = [
408         "n",
409         "parent_class",
410         "n_valid_siblings",
411         "excluded_sparsity",
412         "V_SC_pi",
413         "AI_pi",
414         "coherent_count_pi",
415         "V_SC_trunc",
416         "AI_trunc",
417         "coherent_count_trunc",
418         "parent_prototype",
419     ]
420     with open(audit_path, "w", newline="") as f:
421         w = csv.DictWriter(f, fieldnames=audit_fields)
422         w.writeheader()
423         w.writerows(audit_rows)
424
425     print(f"Wrote {coh_path} ({len(coherence_rows)} rows), {audit_path} ({len(audit_rows)} rows),
426           params, prototypes, pi, f")
427
428 if __name__ == "__main__":
429     main()

```

A.3 Adaptadores de entrada/salida (io/)

A.3.1 src/padicmidi/io/midi_mido.py

```

1  """
2  padicmidi.io.midi_mido – default MIDI adapter (re-export from the canonical motor).
3
4  This module exists for two reasons:
5
6  1. To keep ``padicmidi.io.*`` as a stable public surface for users who
7     want to swap implementations.
8  2. To document explicitly that the gold-standard CSVs in
9     ``results/verified`` were produced with ``mido``.
10
11 The actual parsing is implemented in :mod:`padicmidi.core.echo`; this
12 module simply re-exports the two parsing functions.
13 """
14
15 from __future__ import annotations
16
17 from padicmidi.core.echo import (
18     parse_midi_notes_seconds,
19     parse_midi_notes_beats,
20 )
21
22 __all__ = ["parse_midi_notes_seconds", "parse_midi_notes_beats"]

```

A.3.2 src/padicmidi/io/midi_pretty.py

```

1  """
2  padicmidi.io.midi_pretty – optional MIDI adapter using ``pretty_midi``.
3
4  This adapter exists for compatibility with workflows that prefer
5  ``pretty_midi``. It reproduces the same ``(t_on, t_off, pitch, velocity)``
6  quadruples emitted by :mod:`padicmidi.io.midi_mido`, but with two
7  caveats:
8
9  * Numerical outputs may differ from the gold standard at the 4-th5th
10 decimal place because ``pretty_midi`` uses a different tick-to-second
11 resolution model under tempo changes.
12 * This adapter requires the optional dependency ``pretty_midi``;
13   install with ``pip install padicmidi[pretty]``.
14
15 Use this adapter only when you cannot install ``mido`` or when you want
16 to cross-validate the parsing layer.
17 """
18
19 from __future__ import annotations
20
21 from typing import List, Tuple
22
23 NoteEvent = Tuple[float, float, int, int]
24
25
26 def _require_pretty_midi():
27     try:
28         import pretty_midi
29     except ImportError as exc: # pragma: no cover - exercised only when pretty_midi missing
30         raise ImportError(
31             "padicmidi.io.midi_pretty requires the optional dependency 'pretty_midi'. "

```

```

32         "Install with: pip install padicmidi[pretty]"
33     ) from exc
34     return pretty_midi
35
36
37 def parse_midi_notes_seconds(path: str) -> List[NoteEvent]:
38     """Return ``(t_on, t_off, pitch, velocity)`` in seconds via ``pretty_midi``."""
39     pretty_midi = _require_pretty_midi()
40     pm = pretty_midi.PrettyMIDI(path)
41     events: List[NoteEvent] = []
42     for instrument in pm.instruments:
43         for note in instrument.notes:
44             events.append((float(note.start), float(note.end), int(note.pitch), int(note.velocity)))
45     events.sort(key=lambda e: e[0])
46     return events
47
48
49 def parse_midi_notes_beats(path: str) -> List[NoteEvent]:
50     """Return ``(u_on, u_off, pitch, velocity)`` in beats via ``pretty_midi``.
51
52     Beat-time uses ``pretty_midi.time_to_tick`` divided by ticks per beat,
53     therefore it follows the score grid rather than wall-clock time.
54     """
55     pretty_midi = _require_pretty_midi()
56     pm = pretty_midi.PrettyMIDI(path)
57     tpb = pm.resolution
58     events: List[NoteEvent] = []
59     for instrument in pm.instruments:
60         for note in instrument.notes:
61             u_on = pm.time_to_tick(note.start) / tpb
62             u_off = pm.time_to_tick(note.end) / tpb
63             events.append((float(u_on), float(u_off), int(note.pitch), int(note.velocity)))
64     events.sort(key=lambda e: e[0])
65     return events
66
67
68 __all__ = ["parse_midi_notes_seconds", "parse_midi_notes_beats"]

```

A.4 Análisis y modelos nulos (analysis/)

A.4.1 src/padicmidi/analysis/null_model.py

```

1  #!/usr/bin/env python3
2  """
3  Fast null model verification: config A only, parallel processing.
4  Verifies original values and computes pitch/time-shuffle null models.
5  """
6  import sys, os, csv
7  import numpy as np
8
9  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
10
11 from profinite_echo_midi import (
12     parse_midi_notes_seconds,
13     parse_midi_notes_beats,
14     chroma_series_duration,

```

```

15     chroma_series_duration_beats,
16     onset_density_series,
17     onset_density_series_beats,
18     build_phase1_patterns,
19     build_phase1_patterns_beats,
20     knn_graph_phase1,
21 )
22 import networkx as nx
23
24
25 def tower_beta0(events, H, a_raw, p, n_max, k, cap, use_beats):
26     result = {}
27     for n in range(1, n_max + 1):
28         N = p ** n
29         if use_beats:
30             u0 = events[0][0] if events else 0.0
31             patterns = build_phase1_patterns_beats(
32                 events, H, a_raw, 1.0/12.0, u0,
33                 N=N, step=1, d_ioi=16, Tmax_ioi_beats=2.0)
34         else:
35             t0 = events[0][0] if events else 0.0
36             patterns = build_phase1_patterns(
37                 events, H, a_raw, 0.05, t0,
38                 N=N, step=1, d_ioi=16, Tmax_ioi=2.0)
39             if len(patterns) > cap:
40                 idx = np.linspace(0, len(patterns)-1, cap).astype(int)
41                 patterns = [patterns[i] for i in idx]
42             if not patterns:
43                 result[n] = 0
44                 continue
45             G = knn_graph_phase1(patterns, k, 1.0, 0.0, 0.0)
46             result[n] = nx.number_connected_components(G)
47     return result
48
49
50 def load_events_and_series(midi_path, axis):
51     use_beats = axis == "beats"
52     if use_beats:
53         events = parse_midi_notes_beats(midi_path)
54         H = chroma_series_duration_beats(events, bin_size_beats=1.0/12.0)
55         a = onset_density_series_beats(events, bin_size_beats=1.0/12.0)
56     else:
57         events = parse_midi_notes_seconds(midi_path)
58         H = chroma_series_duration(events, bin_size=0.05)
59         a = onset_density_series(events, bin_size=0.05)
60     return events, H, a, use_beats
61
62
63 def compute_mad_s23(midi_path, axis, ks, cap, shuffle_type="none", seed=42):
64     events, H, a, use_beats = load_events_and_series(midi_path, axis)
65     if not events:
66         return 0.0, 0.0, 0
67
68     rng = np.random.RandomState(seed)
69     if shuffle_type == "pitch":
70         H_shuf = H.copy()
71         rng.shuffle(H_shuf)
72         H = H_shuf

```

```

73     elif shuffle_type == "time":
74         a = a.copy()
75         win = max(50, len(a) // 20)
76         for s in range(0, len(a), win):
77             block = a[s:s+win].copy()
78             rng.shuffle(block)
79             a[s:s+win] = block
80
81     deltas = []
82     for k in ks:
83         b2 = tower_beta0(events, H, a, 2, 6, k, cap, use_beats)
84         b3 = tower_beta0(events, H, a, 3, 5, k, cap, use_beats)
85         for n in range(2, 6):
86             if n in b2 and n in b3:
87                 deltas.append(b2[n] - b3[n])
88
89     if not deltas:
90         return 0.0, 0.0, 0
91     mad = sum(abs(d) for d in deltas) / len(deltas)
92     s23 = sum(1 for d in deltas if d != 0) / len(deltas)
93     return mad, s23, len(deltas)
94
95
96 def original_from_csvs(prefix, axis, ks, csv_base):
97     deltas = []
98     sfx = "_beats" if axis == "beats" else ""
99     for k in ks:
100         p2 = os.path.join(csv_base, axis,
101                             f"{prefix}_cap300_A_tower_real_bach_k{k}_p2{sfx}.csv")
102         p3 = os.path.join(csv_base, axis,
103                             f"{prefix}_cap300_A_tower_real_bach_k{k}_p3{sfx}.csv")
104         if not os.path.exists(p2) or not os.path.exists(p3):
105             continue
106         with open(p2) as f:
107             b2 = {int(r["n"]): int(r["beta0"]) for r in csv.DictReader(f)}
108         with open(p3) as f:
109             b3 = {int(r["n"]): int(r["beta0"]) for r in csv.DictReader(f)}
110         for n in range(2, 6):
111             if n in b2 and n in b3:
112                 deltas.append(b2[n] - b3[n])
113
114     if not deltas:
115         return 0.0, 0.0, 0
116     mad = sum(abs(d) for d in deltas) / len(deltas)
117     s23 = sum(1 for d in deltas if d != 0) / len(deltas)
118     return mad, s23, len(deltas)
119
120
121 if __name__ == "__main__":
122     base = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
123     ext = os.path.join(base, "data", "external_midis")
124     out = os.path.join(base, "outputs")
125     ks = [8, 10, 12]
126
127     pieces = [
128         ("BWV 1049 mov.1", "bwv1049_mov1",
129          os.path.join(ext, "bwv1049_brand4_mov1.bachcentral.mid")),
130         ("BWV 1050 mov.2", "bwv1050_mov2",

```

```

131         os.path.join(ext, "bwv1050_brand5_mov2.bachcentral.mid")),
132     ]
133
134     fmt = f"{'Piece':<20s} {'Variant':<15s} {'Axis':<8s} {'MAD':>8s} {'S23':>8s} {'N':>4s}"
135     print(fmt)
136     print("-" * len(fmt))
137
138     for name, pfx, midi in pieces:
139         for axis in ["seconds", "beats"]:
140             mad, s23, n = original_from_csvs(pfx, axis, ks, out)
141             print(f"{'name':<20s} {'original':<15s} {'axis':<8s} "
142                   f"{'mad':8.3f} {'s23':8.3f} {'n':4d}")
143
144         for sh in ["pitch", "time"]:
145             for axis in ["seconds", "beats"]:
146                 print(f"{'name':<20s} {'sh':<15s} {'axis':<8s} ...",
147                       file=sys.stderr, flush=True)
148                 mad, s23, n = compute_mad_s23(midi, axis, ks, 300, sh)
149                 print(f"{'name':<20s} {'sh':<15s} {'axis':<8s} "
150                       f"{'mad':8.3f} {'s23':8.3f} {'n':4d}")

```

A.4.2 src/padicmidi/analysis/audit.py

```

1  #!/usr/bin/env python3
2  """Aggregate per-parent SC/AI audits from build_hierarchical_maps.py.
3
4  The audit has two parallel diagnostic families:
5
6  * *_pi columns recompute the actual Coh_pi numerator.
7  * *_trunc columns recompute the Coh_grid numerator.
8
9  This distinction matters because the current pipeline assigns level-(n+1)
10 residues globally to child prototypes, so pi(q_{n+1}(.)) can differ from
11 q_n(trunc(.)).
12 """
13
14 from __future__ import annotations
15
16 import argparse
17 import csv
18 from pathlib import Path
19 from statistics import mean
20
21
22 def read_csv(path: Path) -> list[dict[str, str]]:
23     with path.open(newline="") as f:
24         return list(csv.DictReader(f))
25
26
27 def write_csv(path: Path, rows: list[dict], fields: list[str]) -> None:
28     path.parent.mkdir(parents=True, exist_ok=True)
29     with path.open("w", newline="") as f:
30         w = csv.DictWriter(f, fieldnames=fields)
31         w.writeheader()
32         w.writerows(rows)
33

```

```

34
35 def parse_context(path: Path) -> tuple[str, str, int]:
36     # Expected: outputs/paper_profinite_hier/<piece>/<axis>/p<p>/audit_p<p>.csv
37     p_dir = path.parent
38     axis_dir = p_dir.parent
39     piece_dir = axis_dir.parent
40     p = int(p_dir.name.removeprefix("p"))
41     return piece_dir.name, axis_dir.name, p
42
43
44 def load_reported(path: Path) -> dict[int, dict[str, float]]:
45     p = int(path.parent.name.removeprefix("p"))
46     coh_path = path.parent / f"coherence_hier_p{p}.csv"
47     out: dict[int, dict[str, float]] = {}
48     if not coh_path.exists():
49         return out
50     for row in read_csv(coh_path):
51         n = int(row["n"])
52         out[n] = {
53             "coh_pi": float(row["Coh_pi"]),
54             "coh_grid": float(row["Coh_grid"]),
55             "valid_b": float(row["n_samples_nplus1"]),
56         }
57     return out
58
59
60 def aggregate_one(path: Path) -> list[dict]:
61     piece, axis, p = parse_context(path)
62     reported = load_reported(path)
63     by_n: dict[int, list[dict[str, str]]] = {}
64     for row in read_csv(path):
65         by_n.setdefault(int(row["n"]), []).append(row)
66
67     rows: list[dict] = []
68     for n, items in sorted(by_n.items()):
69         counted = [r for r in items if r["coherent_count_pi"] != ""]
70         included = [r for r in counted if r["excluded_sparsity"] == "0"]
71         excluded = [r for r in items if r["excluded_sparsity"] == "1"]
72         denom_dense = p ** (n + 1)
73         valid_b = sum(int(r["n_valid_siblings"]) for r in counted)
74         sum_pi = sum(int(r["coherent_count_pi"]) for r in counted)
75         sum_trunc = sum(int(r["coherent_count_trunc"]) for r in counted)
76         coh_pi_recomputed = sum_pi / denom_dense if denom_dense else 0.0
77         coh_grid_recomputed = sum_trunc / denom_dense if denom_dense else 0.0
78         coh_pi_valid = sum_pi / valid_b if valid_b else 0.0
79         coh_grid_valid = sum_trunc / valid_b if valid_b else 0.0
80         rep = reported.get(n, {})
81         coh_pi_reported = rep.get("coh_pi", float("nan"))
82         coh_grid_reported = rep.get("coh_grid", float("nan"))
83         if abs(coh_pi_recomputed - coh_pi_reported) > 1e-6:
84             raise RuntimeError(
85                 f"Coh_pi mismatch {path} n={n}: "
86                 f"recomputed={coh_pi_recomputed} reported={coh_pi_reported}"
87             )
88         if abs(coh_grid_recomputed - coh_grid_reported) > 1e-6:
89             raise RuntimeError(
90                 f"Coh_grid mismatch {path} n={n}: "
91                 f"recomputed={coh_grid_recomputed} reported={coh_grid_reported}"

```

```

92         )
93         n_parents = len(included)
94         n_sc_pi = sum(int(r["V_SC_pi"]) == p for r in included)
95         n_ai_pi = sum(int(r["AI_pi"]) == 1 for r in included)
96         n_both_pi = sum(int(r["V_SC_pi"]) == p and int(r["AI_pi"]) == 1 for r in included)
97         n_sc_trunc = sum(int(r["V_SC_trunc"]) == p for r in included)
98         n_ai_trunc = sum(int(r["AI_trunc"]) == 1 for r in included)
99         n_both_trunc = sum(int(r["V_SC_trunc"]) == p and int(r["AI_trunc"]) == 1 for r in included)
100         rows.append({
101             "piece": piece,
102             "axis": axis,
103             "p": p,
104             "n": n,
105             "n_parents_total": len(items),
106             "n_parents_included": n_parents,
107             "n_parents_excluded_sparsity": len(excluded),
108             "n_SC_pi_holds": n_sc_pi,
109             "n_AI_pi_holds": n_ai_pi,
110             "n_both_pi_hold": n_both_pi,
111             "frac_SC_pi": n_sc_pi / n_parents if n_parents else 0.0,
112             "frac_AI_pi": n_ai_pi / n_parents if n_parents else 0.0,
113             "frac_both_pi": n_both_pi / n_parents if n_parents else 0.0,
114             "n_SC_trunc_holds": n_sc_trunc,
115             "n_AI_trunc_holds": n_ai_trunc,
116             "n_both_trunc_hold": n_both_trunc,
117             "frac_SC_trunc": n_sc_trunc / n_parents if n_parents else 0.0,
118             "frac_AI_trunc": n_ai_trunc / n_parents if n_parents else 0.0,
119             "frac_both_trunc": n_both_trunc / n_parents if n_parents else 0.0,
120             "coverage": valid_b / denom_dense if denom_dense else 0.0,
121             "coh_pi_recomputed": coh_pi_recomputed,
122             "coh_pi_reported": coh_pi_reported,
123             "coh_pi_valid": coh_pi_valid,
124             "coh_grid_recomputed": coh_grid_recomputed,
125             "coh_grid_reported": coh_grid_reported,
126             "coh_grid_valid": coh_grid_valid,
127         })
128     return rows
129
130
131 def group_for(piece: str) -> str:
132     if piece.startswith(("bwv1049", "bwv1050", "bwv1079")) or piece == "goldberg_aria":
133         return "poly"
134     return "mono"
135
136
137 def main() -> None:
138     ap = argparse.ArgumentParser()
139     ap.add_argument("--root", default="outputs/paper_profinite_hier")
140     ap.add_argument("--out", default="outputs/audit_summary")
141     args = ap.parse_args()
142     root = Path(args.root)
143     out_dir = Path(args.out)
144     audit_paths = sorted(root.glob("*/**/p*/audit_p*.csv"))
145     if not audit_paths:
146         raise SystemExit(f"No audit_p*.csv files found under {root}")
147
148     table: list[dict] = []
149     for path in audit_paths:

```

```

150     table.extend(aggregate_one(path))
151
152     fields = list(table[0].keys())
153     write_csv(out_dir / "audit_table.csv", table, fields)
154
155     by_key: dict[tuple[str, str, int], list[dict]] = {}
156     for row in table:
157         by_key.setdefault((row["piece"], row["axis"], int(row["p"])), []).append(row)
158
159     piece_rows = []
160     for (piece, axis, p), rows in sorted(by_key.items()):
161         piece_rows.append({
162             "piece": piece,
163             "texture_group": group_for(piece),
164             "axis": axis,
165             "p": p,
166             "n_levels": len(rows),
167             "mean_frac_SC_pi": mean(float(r["frac_SC_pi"]) for r in rows),
168             "mean_frac_AI_pi": mean(float(r["frac_AI_pi"]) for r in rows),
169             "mean_frac_both_pi": mean(float(r["frac_both_pi"]) for r in rows),
170             "mean_frac_SC_trunc": mean(float(r["frac_SC_trunc"]) for r in rows),
171             "mean_frac_AI_trunc": mean(float(r["frac_AI_trunc"]) for r in rows),
172             "mean_frac_both_trunc": mean(float(r["frac_both_trunc"]) for r in rows),
173             "mean_coverage": mean(float(r["coverage"]) for r in rows),
174             "mean_coh_pi": mean(float(r["coh_pi_reported"]) for r in rows),
175             "mean_coh_pi_valid": mean(float(r["coh_pi_valid"]) for r in rows),
176         })
177     write_csv(out_dir / "audit_by_piece.csv", piece_rows, list(piece_rows[0].keys()))
178
179     with (out_dir / "SUMMARY.md").open("w") as f:
180         f.write("# Audit Summary\n\n")
181         f.write(f"Audited files: {len(audit_paths)}\n\n")
182         f.write("`coh_pi_recomputed == coh_pi_reported` and `coh_grid_recomputed == coh_grid_reported`\n\n")
183         f.write("for all rows.\n\n")
184         for group in ("mono", "poly"):
185             group_rows = [r for r in piece_rows if r["texture_group"] == group and int(r["p"]) == 2]
186             if not group_rows:
187                 continue
188             f.write(f"### {group} p=2\n\n")
189             f.write(f"- mean coverage: {mean(float(r['mean_coverage']) for r in group_rows):.4f}\n")
190             f.write(f"- mean frac_SC_pi: {mean(float(r['mean_frac_SC_pi']) for r in group_rows):.4f}\n")
191             f.write(f"- mean frac_AI_pi: {mean(float(r['mean_frac_AI_pi']) for r in group_rows):.4f}\n")
192             f.write(f"- mean Coh_pi_valid: {mean(float(r['mean_coh_pi_valid']) for r in group_rows):.4f}\n\n")
193
194 if __name__ == "__main__":
195     main()

```

A.4.3 src/padicmidi/analysis/aggregate.py

```

1  #!/usr/bin/env python3
2  """
3  build_control_primes_summary_nextpp.py - Ingest Phase I Next++ CSVs from
4  outputs/phaseI_nextpp/raw/<piece>/<condition>/ e.g. seconds_cap300, beats_bin12_cap300.
5  Filename: <prefix>_A|B|C_tower_real_bach_k<k>_p<2|3|5|7>(<beats>)?.csv

```

```

6 Piece and condition taken from directory path; axis/bin/cap parsed from condition name.
7 Output: outputs/phaseI_nextpp/summary/SUMMARY_TABLE_p2357_nextpp.csv, CONTROL_PRIMES_REPORT_nextpp.txt
8 """
9 import csv
10 import re
11 from pathlib import Path
12 from collections import defaultdict
13
14 ROOT = Path(__file__).resolve().parent.parent
15 RAW_BASE = ROOT / "outputs" / "phaseI_nextpp" / "raw"
16 SUMMARY_DIR = ROOT / "outputs" / "phaseI_nextpp" / "summary"
17 PRIMES = (2, 3, 5, 7)
18
19 RE_FNAME = re.compile(r"^(.+)_([A|B|C])_tower_real_bach_k(\d+)_p([2357])(_beats)?\.csv$")
20
21
22 def load_csv(path: Path) -> dict[int, dict]:
23     out = {}
24     with open(path, newline="") as f:
25         r = csv.DictReader(f)
26         for row in r:
27             n = int(row["n"])
28             out[n] = {
29                 "beta0": int(row["beta0"]),
30                 "V": int(row["V"]),
31                 "avg_degree": float(row["avg_degree"]),
32                 "giant_component_frac": float(row["giant_component_frac"]),
33             }
34     return out
35
36
37 def parse_condition(cond_name: str) -> tuple[str, str, str]:
38     """Return (axis, bin_beats_str, cap_str)."""
39     axis = "seconds" if cond_name.startswith("seconds") else "beats"
40     cap_str = "500" if "cap500" in cond_name else "300"
41     bin_str = "16" if "bin16" in cond_name else ("12" if "bin12" in cond_name else "")
42     return axis, bin_str, cap_str
43
44
45 def collect_quadruplets():
46     """Yield (piece, condition, axis, bin_beats, cap, config, k, paths). piece/condition from dir
47         path."""
48     by_key = {}
49     if not RAW_BASE.exists():
50         return
51     for piece_dir in sorted(RAW_BASE.iterdir()):
52         if not piece_dir.is_dir():
53             continue
54         piece = piece_dir.name
55         for cond_dir in sorted(piece_dir.iterdir()):
56             if not cond_dir.is_dir():
57                 continue
58             cond_name = cond_dir.name
59             axis, bin_str, cap_str = parse_condition(cond_name)
60             for path in cond_dir.glob("*_tower_real_bach_k*_p*.csv"):
61                 name = path.name
62                 m = RE_FNAME.match(name)
63                 if not m:

```

```

63         continue
64         _, config, k_str, p_str, suffix = m.groups()
65         p_val = int(p_str)
66         k = int(k_str)
67         key = (piece, cond_name, axis, bin_str, cap_str, config, k)
68         if key not in by_key:
69             by_key[key] = {q: None for q in PRIMES}
70             by_key[key][p_val] = path
71     for key in sorted(by_key.keys()):
72         paths = by_key[key]
73         if paths.get(2) and paths.get(3):
74             yield (*key, paths)
75
76
77 def main():
78     SUMMARY_DIR.mkdir(parents=True, exist_ok=True)
79     rows = []
80     for piece, condition, axis, bin_beats, cap, config, k, paths in collect_quadruplets():
81         data = {p: load_csv(paths[p]) if paths.get(p) else {} for p in PRIMES}
82         common_n = set(data[2].keys()) & set(data[3].keys())
83         if paths.get(5):
84             common_n &= set(data[5].keys())
85         if paths.get(7):
86             common_n &= set(data[7].keys())
87         common_n = sorted(common_n)
88         for n in common_n:
89             r = {p: data[p].get(n) for p in PRIMES}
90             if not r[2] or not r[3]:
91                 continue
92             row = {
93                 "piece": piece,
94                 "condition": condition,
95                 "axis": axis,
96                 "bin_beats": bin_beats,
97                 "cap": cap,
98                 "config": config,
99                 "k": k,
100                 "n": n,
101                 "beta0_p2": r[2]["beta0"],
102                 "beta0_p3": r[3]["beta0"],
103                 "beta0_p5": r[5]["beta0"] if r.get(5) else "",
104                 "beta0_p7": r[7]["beta0"] if r.get(7) else "",
105                 "giant_p2": round(r[2]["giant_component_frac"], 6),
106                 "giant_p3": round(r[3]["giant_component_frac"], 6),
107                 "giant_p5": round(r[5]["giant_component_frac"], 6) if r.get(5) else "",
108                 "giant_p7": round(r[7]["giant_component_frac"], 6) if r.get(7) else "",
109             }
110             row["Delta23"] = r[2]["beta0"] - r[3]["beta0"]
111             row["Delta25"] = (r[2]["beta0"] - r[5]["beta0"]) if r.get(5) else ""
112             row["Delta27"] = (r[2]["beta0"] - r[7]["beta0"]) if r.get(7) else ""
113             row["DeltaG23"] = round(r[2]["giant_component_frac"] - r[3]["giant_component_frac"], 6)
114             row["DeltaG25"] = round(r[2]["giant_component_frac"] - r[5]["giant_component_frac"], 6) if
115                 r.get(5) else ""
116             row["DeltaG27"] = round(r[2]["giant_component_frac"] - r[7]["giant_component_frac"], 6) if
117                 r.get(7) else ""
118             rows.append(row)
119
120     fieldnames = ["piece", "condition", "axis", "bin_beats", "cap", "config", "k", "n",

```

```

119         "beta0_p2", "beta0_p3", "beta0_p5", "beta0_p7",
120         "giant_p2", "giant_p3", "giant_p5", "giant_p7",
121         "Delta23", "Delta25", "Delta27", "DeltaG23", "DeltaG25", "DeltaG27"]
122     out_csv = SUMMARY_DIR / "SUMMARY_TABLE_p2357_nextpp.csv"
123     with open(out_csv, "w", newline="") as f:
124         w = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore")
125         w.writeheader()
126         w.writerows(rows)
127     print("Wrote", out_csv, "rows:", len(rows))
128
129     report_lines = [
130         "CONTROL PRIMES SUMMARY (Phase I Next++)",
131         "=====",
132         "Conditions: seconds_cap300/500, beats_bin12/16_cap300/500.",
133         ""
134     ]
135     for (piece, condition) in sorted(set((r["piece"], r["condition"]) for r in rows)):
136         sub = [r for r in rows if r["piece"] == piece and r["condition"] == condition]
137         if not sub:
138             continue
139         report_lines.append(f"Piece: {piece} Condition: {condition}")
140         report_lines.append(f" Rows: {len(sub)}. Delta23 nonzero: {sum(1 for r in sub if r.get('Delta23') != 0)}.")
141         report_lines.append("")
142     report_path = SUMMARY_DIR / "CONTROL_PRIMES_REPORT_nextpp.txt"
143     with open(report_path, "w") as f:
144         f.write("\n".join(report_lines))
145     print("Wrote", report_path)
146
147
148 if __name__ == "__main__":
149     main()

```

A.4.4 src/padicmidi/analysis/directionality.py

```

1  #!/usr/bin/env python3
2  """Directionality report for hierarchical coherence.
3
4  This script reports both the raw dense-normalized Coh_pi and the coverage-corrected
5  conditional score Coh_pi_valid = match / valid_b from audit_table.csv. The latter is
6  the appropriate statistic when coverage differs by prime (e.g. p=2 with step=2).
7  """
8
9  from __future__ import annotations
10
11  import argparse
12  import csv
13  import math
14  from pathlib import Path
15  from statistics import mean
16
17
18  def read_csv(path: Path) -> list[dict[str, str]]:
19     with path.open(newline="") as f:
20         return list(csv.DictReader(f))
21

```

```

22
23 def group_for(piece: str) -> str:
24     if piece.startswith(("bwv1049", "bwv1050", "bwv1079")) or piece == "goldberg_aria":
25         return "poly"
26     return "mono"
27
28
29 def binom_two_sided(k: int, n: int, p: float = 0.5) -> float:
30     if n == 0:
31         return 1.0
32     probs = [math.comb(n, i) * (p ** i) * ((1 - p) ** (n - i)) for i in range(n + 1)]
33     pk = probs[k]
34     return min(1.0, sum(x for x in probs if x <= pk + 1e-18))
35
36
37 def clopper_pearson_zero_success(n: int, alpha: float = 0.05) -> tuple[float, float]:
38     if n == 0:
39         return (0.0, 1.0)
40     # For k=0, exact upper endpoint is 1 - (alpha/2)^(1/n).
41     return (0.0, 1.0 - (alpha / 2.0) ** (1.0 / n))
42
43
44 def classify(values: list[float], floor: float, eps: float) -> tuple[int, int, int]:
45     below = sum(v < floor - eps for v in values)
46     above = sum(v > floor + eps for v in values)
47     at = len(values) - below - above
48     return below, at, above
49
50
51 def main() -> None:
52     ap = argparse.ArgumentParser()
53     ap.add_argument("--audit-table", default="outputs/audit_summary/audit_table.csv")
54     ap.add_argument("--out", default="outputs/directionality/directionality_report.txt")
55     ap.add_argument("--eps", type=float, default=1e-3)
56     args = ap.parse_args()
57
58     rows = [r for r in read_csv(Path(args.audit_table)) if int(r["p"]) == 2]
59     out = Path(args.out)
60     out.parent.mkdir(parents=True, exist_ok=True)
61
62     by_piece: dict[str, list[dict[str, str]]] = {}
63     for row in rows:
64         by_piece.setdefault(row["piece"], []).append(row)
65
66     lines: list[str] = []
67     lines.append("# Directionality Report\n")
68     lines.append("Important: raw Coh_pi for p=2 is capped by coverage when step=2. ")
69     lines.append("The coverage-corrected score Coh_pi_valid = match/valid_b is therefore reported\n\n")
70
71     for group in ("mono", "poly", "all"):
72         if group == "all":
73             group_rows = rows
74         else:
75             group_rows = [r for r in rows if group_for(r["piece"]) == group]
76         raw_vals = [float(r["coh_pi_reported"]) for r in group_rows]
77         valid_vals = [float(r["coh_pi_valid"]) for r in group_rows]
78         raw_below, raw_at, raw_above = classify(raw_vals, 0.5, args.eps)

```

```

79     valid_below, valid_at, valid_above = classify(valid_vals, 1.0, args.eps)
80     n_viol = valid_below + valid_above
81     one_sided = (0.5 ** n_viol) if valid_above == 0 and n_viol else 1.0
82     two_sided = binom_two_sided(valid_above, n_viol) if n_viol else 1.0
83     ci = clopper_pearson_zero_success(n_viol) if valid_above == 0 else (float("nan"), float("nan"))
84     lines.append(f"## {group}\n\n")
85     lines.append(f"- rows: {len(group_rows)}\n")
86     lines.append(f"- raw Coh_pi vs 0.5: below={raw_below}, at={raw_at}, above={raw_above}\n")
87     lines.append(f"- Coh_pi_valid vs 1.0: below={valid_below}, at={valid_at},
88         above={valid_above}\n")
89     lines.append(f"- exact binomial one-sided p-value (above fewer than half among valid-score
90         violations): {one_sided:.6g}\n")
91     lines.append(f"- exact binomial two-sided p-value: {two_sided:.6g}\n")
92     if n_viol:
93         lines.append(f"- 95% Clopper-Pearson CI for P(above | violation), if above=0: [{ci[0]:.4f},
94             {ci[1]:.4f}]\n")
95     lines.append("\n")
96
97     lines.append("## By piece\n\n")
98     lines.append("piece,group,N,raw_below,raw_at,raw_above,valid_below,valid_at,valid_above,mean_raw,mean_valid\n")
99     for piece, prs in sorted(by_piece.items()):
100         raw_vals = [float(r["coh_pi_reported"]) for r in prs]
101         valid_vals = [float(r["coh_pi_valid"]) for r in prs]
102         rb, ra, rup = classify(raw_vals, 0.5, args.eps)
103         vb, va, vup = classify(valid_vals, 1.0, args.eps)
104         lines.append(
105             f"{piece},{group_for(piece)},{len(prs)},{rb},{ra},{rup},{vb},{va},{vup},"
106             f"{mean(raw_vals):.6f},{mean(valid_vals):.6f}\n"
107         )
108
109     out.write_text("".join(lines))
110
111 if __name__ == "__main__":
112     main()

```

A.5 Ejecutables de línea de comandos (cli/)

A.5.1 src/padicmidi/cli/main.py — ejecutable unificado

```

1  #!/usr/bin/env python3
2  """
3  padicmidi.cli.main - unified ``padicmidi`` command-line entry point.
4
5  Dispatches to the specialised sub-commands (run-one, run-suite, benchmark,
6  job-list, mutopia). This is the canonical executable advertised by the
7  manuals: ``padicmidi <subcommand> [options]``.
8  """
9
10 from __future__ import annotations
11
12 import sys
13
14 from padicmidi.cli import benchmark, job_list, mutopia, run_one, run_suite
15
16 SUBCOMMANDS = {

```

```

17     "run-one":    run_one.main,
18     "run-suite": run_suite.main,
19     "benchmark": benchmark.main,
20     "job-list":  job_list.main,
21     "mutopia":   mutopia.main,
22 }
23
24 USAGE = """\
25 padicmidi v1.0.0 - p-adic hierarchical analysis of symbolic music
26 Author: J. Rogelio Pérez-Buendía (CIMAT-Mérida) · ORCID 0000-0002-7739-4779
27
28 Usage:
29 padicmidi <subcommand> [arguments...]
30 padicmidi --help
31 padicmidi <subcommand> --help
32
33 Sub-commands:
34 run-one    Analyse a single MIDI file (one piece, one prime).
35 run-suite  Analyse a corpus across multiple primes and axes.
36 benchmark  Run the canonical BWV 1007 benchmark.
37 job-list   Generate a job list for a corpus.
38 mutopia    Download CC-licensed MIDI files from Mutopia.
39
40 For details on each sub-command, run:
41 padicmidi <subcommand> --help
42
43 Web demo (no installation needed):
44 Open `web-demo/applet.html` (English) or
45     `web-demo/applet-es.html` (Spanish) in any modern browser.
46 """
47
48
49 def main(argv: list[str] | None = None) -> int:
50     argv = sys.argv[1:] if argv is None else list(argv)
51     if not argv or argv[0] in ("-h", "--help", "help"):
52         print(USAGE)
53         return 0
54     sub = argv[0]
55     if sub not in SUBCOMMANDS:
56         print(f"padicmidi: unknown sub-command '{sub}'\n", file=sys.stderr)
57         print(USAGE, file=sys.stderr)
58         return 2
59     sys.argv = [f"padicmidi {sub}"] + argv[1:]
60     rc = SUBCOMMANDS[sub]()
61     return int(rc) if rc is not None else 0
62
63
64 if __name__ == "__main__":
65     raise SystemExit(main())

```

A.5.2 src/padicmidi/cli/run_one.py

```

1  #!/usr/bin/env python3
2  """
3  padicmidi.cli.run_one - analyse a single MIDI file (entry point: `padicmidi-run-one`).
4

```

```

5  Wrapper around :mod:`padicmidi.core.hierarchical` that mirrors the
6  historical ``run_one_piece.py`` script but with two improvements:
7
8  * Calls the motor in-process (no ``subprocess``).
9  * Exposes the random seed and all hyper-parameters via the CLI.
10
11  Examples
12  -----
13  ::
14
15      padicmidi-run-one bwv1007-1.mid bwv1007_pre beats 2 5 \\\
16          --out results_local/bwv1007_pre/beats/p2
17
18      padicmidi-run-one bwv1007-1.mid bwv1007_pre beats 3 4 \\\
19          -K 16 --Kchild 2 --M 800 --step 2 --seed 42 \\\
20          --out results_local/bwv1007_pre/beats/p3
21  """
22
23  from __future__ import annotations
24
25  import argparse
26  import csv
27  import json
28  import sys
29  from pathlib import Path
30
31  import numpy as np
32
33  from padicmidi import __version__
34  from padicmidi.core.config import (
35      DEFAULT_BIN_BEATS,
36      DEFAULT_BIN_SECONDS,
37      DEFAULT_K,
38      DEFAULT_KCHILD,
39      DEFAULT_M,
40      DEFAULT_SEED,
41      DEFAULT_STEP,
42      SUPPORTED_PRIMES,
43      default_nmax,
44  )
45  from padicmidi.core.hierarchical import (
46      build_X_beats,
47      build_X_seconds,
48      run_hierarchical,
49  )
50
51
52  def _build_argparser() -> argparse.ArgumentParser:
53      ap = argparse.ArgumentParser(
54          prog="padicmidi-run-one",
55          description=(
56              "Analyse a single MIDI file with the p-adic hierarchical motor "
57              "and write coherence/audit/prototype CSVs to an output folder."
58          ),
59      )
60      ap.add_argument("midi", help="Path to the MIDI file (absolute or relative).")
61      ap.add_argument("piece", help="Short identifier for the piece (used in output folder name).")
62      ap.add_argument("axis", choices=["beats", "seconds"], help="Time axis.")

```

```

63     ap.add_argument("p", type=int, help=f"Prime in {SUPPORTED_PRIMES}.")
64     ap.add_argument("Nmax", type=int, nargs="?", default=None, help="Maximum tower level (default depends on p).")
65     ap.add_argument("--out", required=True, help="Output directory.")
66     ap.add_argument("--K", type=int, default=DEFAULT_K, help=f"K for K-means (default {DEFAULT_K}).")
67     ap.add_argument("--Kchild", type=int, default=DEFAULT_KCHILD, help=f"Children per parent (default {DEFAULT_KCHILD}).")
68     ap.add_argument("--M", type=int, default=DEFAULT_M, help=f"Subsample size for K-means (default {DEFAULT_M}).")
69     ap.add_argument("--step", type=int, default=DEFAULT_STEP, help=f"Window step (default {DEFAULT_STEP}).")
70     ap.add_argument("--seed", type=int, default=DEFAULT_SEED, help=f"Random seed (default {DEFAULT_SEED}).")
71     ap.add_argument("--bin-beats", type=float, default=DEFAULT_BIN_BEATS, help=f"Bin size (beats axis, default {DEFAULT_BIN_BEATS:.6f}).")
72     ap.add_argument("--bin", type=float, default=DEFAULT_BIN_SECONDS, help=f"Bin size (seconds axis, default {DEFAULT_BIN_SECONDS}).")
73     ap.add_argument("--version", action="version", version=f"padicmidi {_version__}")
74     return ap
75
76
77 def main(argv: list[str] | None = None) -> int:
78     args = _build_argparser().parse_args(argv)
79
80     if args.p not in SUPPORTED_PRIMES:
81         print(f"error: --p must be one of {SUPPORTED_PRIMES}; got {args.p!r}", file=sys.stderr)
82         return 2
83     Nmax = args.Nmax if args.Nmax is not None else default_nmax(args.p)
84
85     midi_path = Path(args.midi).expanduser().resolve()
86     if not midi_path.exists():
87         print(f"error: MIDI file not found: {midi_path}", file=sys.stderr)
88         return 2
89
90     out_dir = Path(args.out).expanduser().resolve()
91     out_dir.mkdir(parents=True, exist_ok=True)
92
93     if args.axis == "seconds":
94         X = build_X_seconds(str(midi_path), bin_size=args.bin)
95     else:
96         X = build_X_beats(str(midi_path), bin_size_beats=args.bin_beats)
97
98     if len(X) < 2:
99         print("error: series too short (less than 2 bins)", file=sys.stderr)
100        return 1
101
102    rng = np.random.default_rng(args.seed)
103    prototypes_n, f_n, pi_maps, coherence_rows, audit_rows = run_hierarchical(
104        X, args.p, Nmax, args.step, args.K, args.Kchild, args.M, rng
105    )
106
107    p = args.p
108    params = {
109        "piece": args.piece,
110        "midi": str(midi_path),
111        "axis": args.axis,
112        "p": p,
113        "Nmax": Nmax,

```

```

114         "K": args.K,
115         "Kchild": args.Kchild,
116         "M": args.M,
117         "step": args.step,
118         "seed": args.seed,
119         "bin_seconds": args.bin,
120         "bin_beats": args.bin_beats,
121         "padicmidi_version": __version__,
122     }
123     (out_dir / "params.json").write_text(json.dumps(params, indent=2))
124     (out_dir / "params.txt").write_text(
125         " ".join(f"{k}={v}" for k, v in params.items()) + "\n"
126     )
127
128     for n, prot in prototypes_n.items():
129         flat = prot.reshape(prot.shape[0], -1)
130         np.savetxt(out_dir / f"S_n_prototypes_p{p}_n{n}.csv", flat, delimiter=",")
131     for n, pi_list in pi_maps.items():
132         with open(out_dir / f"pi_p{p}_n{n+1}_to_n{n}.csv", "w", newline="") as fh:
133             w = csv.writer(fh)
134             w.writerow(["child_id", "parent_id"])
135             for cid, pid in enumerate(pi_list):
136                 w.writerow([cid, pid])
137     for n, fn in f_n.items():
138         with open(out_dir / f"f_p{p}_n{n}.csv", "w", newline="") as fh:
139             w = csv.writer(fh)
140             w.writerow(["residue", "prototype_id"])
141             for a in sorted(fn.keys()):
142                 w.writerow([a, fn[a]])
143
144     with open(out_dir / f"coherence_hier_p{p}.csv", "w", newline="") as fh:
145         w = csv.DictWriter(fh, fieldnames=["n", "Coh_pi", "Coh_grid", "n_samples_n",
146             "n_samples_nplus1"])
147         w.writeheader()
148         w.writerows(coherence_rows)
149
150     audit_fields = [
151         "n", "parent_class", "n_valid_siblings", "excluded_sparsity",
152         "V_SC_pi", "AI_pi", "coherent_count_pi",
153         "V_SC_trunc", "AI_trunc", "coherent_count_trunc",
154         "parent_prototype",
155     ]
156     with open(out_dir / f"audit_p{p}.csv", "w", newline="") as fh:
157         w = csv.DictWriter(fh, fieldnames=audit_fields)
158         w.writeheader()
159         w.writerows(audit_rows)
160
161     print(
162         f"[padicmidi-run-one] piece={args.piece} axis={args.axis} p={p} Nmax={Nmax}: "
163         f"wrote {out_dir} ({len(coherence_rows)} coherence rows, {len(audit_rows)} audit rows).",
164         flush=True,
165     )
166     return 0
167
168 if __name__ == "__main__":
169     sys.exit(main())

```

A.5.3 src/padicmidi/cli/run_suite.py

```

1  #!/usr/bin/env python3
2  """
3  padicmidi.cli.run_suite – Phase I run on a suite of MIDI files (entry point: ``padicmidi-run-suite``).
4
5  Runs the Phase I (echo) motor on each MIDI in a folder and writes
6  results under a configurable output directory. By default analyses the
7  six BWV 1007 movements bundled in ``data/midi/`` of the package.
8
9  This is a thin orchestration wrapper around :mod:`padicmidi.core.echo`,
10 invoking it as a subprocess to keep CLI argument parsing isolated.
11 """
12
13 from __future__ import annotations
14
15 import argparse
16 import subprocess
17 import sys
18 from pathlib import Path
19
20 from padicmidi import __version__
21
22 DEFAULT_PIECES: list[tuple[str, str]] = [
23     ("1pre", "cs1-1pre.mid"),
24     ("2all", "cs1-2all.mid"),
25     ("3cou", "cs1-3cou.mid"),
26     ("4sar", "cs1-4sar.mid"),
27     ("5men", "cs1-5men.mid"),
28     ("6gig", "cs1-6gig.mid"),
29 ]
30
31
32 def _build_argparser() -> argparse.ArgumentParser:
33     ap = argparse.ArgumentParser(
34         prog="padicmidi-run-suite",
35         description="Run Phase I on a suite of MIDI files (defaults to the BWV 1007 movements).",
36     )
37     ap.add_argument(
38         "--midi-dir",
39         type=Path,
40         required=True,
41         help="Directory containing the MIDI files referenced by --pieces (or default cs1-* set).",
42     )
43     ap.add_argument(
44         "--out-dir",
45         type=Path,
46         required=True,
47         help="Output directory; subfolders 'seconds/' and 'beats/' will be created.",
48     )
49     ap.add_argument(
50         "--pieces",
51         nargs="*",
52         metavar="SHORTNAME=FILENAME",
53         help="Override the default list (e.g. 1pre=cs1-1pre.mid 2all=cs1-2all.mid). Default uses the BWV 1007 set.",
54     )
55     ap.add_argument("--cap", type=int, default=300, help="Window cap (default 300, paper default).")

```

```

56     ap.add_argument("--bin-beats", type=float, default=0.083333)
57     ap.add_argument("--axes", nargs="+", choices=["seconds", "beats"], default=["seconds", "beats"])
58     ap.add_argument("--version", action="version", version=f"padicmidi {_version__}")
59     return ap
60
61
62 def _parse_pieces(spec: list[str] | None) -> list[tuple[str, str]]:
63     if not spec:
64         return DEFAULT_PIECES
65     out: list[tuple[str, str]] = []
66     for item in spec:
67         if "=" not in item:
68             raise SystemExit(f"--pieces entry must be SHORTNAME=FILENAME; got {item!r}")
69         short, name = item.split("=", 1)
70         out.append((short.strip(), name.strip()))
71     return out
72
73
74 def main(argv: list[str] | None = None) -> int:
75     args = _build_argparser().parse_args(argv)
76     pieces = _parse_pieces(args.pieces)
77
78     midi_dir = args.midi_dir.expanduser().resolve()
79     if not midi_dir.is_dir():
80         print(f"error: --midi-dir does not exist: {midi_dir}", file=sys.stderr)
81         return 2
82
83     args.out_dir.mkdir(parents=True, exist_ok=True)
84     out_seconds = args.out_dir / "seconds"
85     out_beats = args.out_dir / "beats"
86     out_seconds.mkdir(exist_ok=True)
87     out_beats.mkdir(exist_ok=True)
88
89     for short, name in pieces:
90         midi_path = midi_dir / name
91         if not midi_path.exists():
92             print(f"[run-suite] skipping missing MIDI: {midi_path}", file=sys.stderr)
93             continue
94         for axis in args.axes:
95             out_dir = out_seconds if axis == "seconds" else out_beats
96             prefix = str(out_dir) + f"/cs1_{short}_cap{args.cap}_"
97             cmd = [
98                 sys.executable,
99                 "-m", "padicmidi.core.echo",
100                 str(midi_path),
101                 "--phase1-only",
102                 "--save", prefix,
103                 "--cap", str(args.cap),
104             ]
105             if axis == "beats":
106                 cmd += ["--time-axis", "beats", "--bin-beats", str(args.bin_beats)]
107             print(f"[run-suite]", " ".join(cmd))
108             ret = subprocess.run(cmd)
109             if ret.returncode != 0:
110                 print(f"[run-suite] failed on {short}/{axis}", file=sys.stderr)
111                 return ret.returncode
112     print("[run-suite] finished.")
113     return 0

```

```

114
115
116 if __name__ == "__main__":
117     sys.exit(main())

```

A.5.4 src/padicmidi/cli/benchmark.py

```

1  #!/usr/bin/env python3
2  """
3  padicmidi.cli.benchmark - Phase I benchmark across the bundled benchmark set.
4
5  Entry point: ``padicmidi-benchmark``. Runs the Phase I motor for each
6  piece in the benchmark set (BWV 1007 movements + binary/ternary toys)
7  and writes prefixed outputs under the user-supplied output directory.
8
9  Defaults reproduce the gold-standard runs of the companion papers.
10 """
11
12 from __future__ import annotations
13
14 import argparse
15 import subprocess
16 import sys
17 from pathlib import Path
18
19 from padicmidi import __version__
20
21 BENCHMARK_PIECES: list[tuple[str, str]] = [
22     ("cs1_1pre", "cs1-1pre.mid"),
23     ("cs1_2all", "cs1-2all.mid"),
24     ("cs1_3cou", "cs1-3cou.mid"),
25     ("cs1_4sar", "cs1-4sar.mid"),
26     ("cs1_5men", "cs1-5men.mid"),
27     ("cs1_6gig", "cs1-6gig.mid"),
28     ("toy_binary", "toy_binary.mid"),
29     ("toy_ternary", "toy_ternary.mid"),
30 ]
31
32
33 def main(argv: list[str] | None = None) -> int:
34     ap = argparse.ArgumentParser(
35         prog="padicmidi-benchmark",
36         description="Run the Phase I benchmark for all pieces in the BWV 1007 + toys set.",
37     )
38     ap.add_argument(
39         "--midi-dir",
40         type=Path,
41         required=True,
42         help="Directory containing the benchmark MIDI files.",
43     )
44     ap.add_argument(
45         "--out-dir",
46         type=Path,
47         required=True,
48         help="Output directory; subfolders 'seconds/' and 'beats/' will be created.",
49     )

```

```

50     ap.add_argument("--seconds-only", action="store_true", help="Only run seconds axis.")
51     ap.add_argument("--beats-only", action="store_true", help="Only run beats axis.")
52     ap.add_argument("--dry-run", action="store_true", help="Print commands only.")
53     ap.add_argument("--version", action="version", version=f"padicmidi {_version_}")
54     args = ap.parse_args(argv)
55
56     midi_dir = args.midi_dir.expanduser().resolve()
57     if not midi_dir.is_dir():
58         print(f"error: --midi-dir does not exist: {midi_dir}", file=sys.stderr)
59         return 2
60
61     args.out_dir.mkdir(parents=True, exist_ok=True)
62     out_seconds = args.out_dir / "seconds"
63     out_beats = args.out_dir / "beats"
64
65     run_seconds = not args.beats_only
66     run_beats = not args.seconds_only
67
68     for piece, midi_name in BENCHMARK_PIECES:
69         midi_path = midi_dir / midi_name
70         if not midi_path.exists():
71             print(f"[benchmark] skip (missing): {midi_name}", file=sys.stderr)
72             continue
73
74         if run_seconds:
75             out_seconds.mkdir(parents=True, exist_ok=True)
76             prefix = str(out_seconds) + f"/{piece}_"
77             cmd = [
78                 sys.executable, "-m", "padicmidi.core.echo",
79                 str(midi_path),
80                 "--phase1-only",
81                 "--save", prefix,
82             ]
83             print(" ".join(cmd))
84             if not args.dry_run:
85                 ret = subprocess.run(cmd)
86                 if ret.returncode != 0:
87                     print(f"[benchmark] failed on {piece}/seconds", file=sys.stderr)
88                     return ret.returncode
89
90         if run_beats:
91             out_beats.mkdir(parents=True, exist_ok=True)
92             prefix = str(out_beats) + f"/{piece}_"
93             cmd = [
94                 sys.executable, "-m", "padicmidi.core.echo",
95                 str(midi_path),
96                 "--phase1-only",
97                 "--save", prefix,
98                 "--time-axis", "beats",
99                 "--bin-beats", "0.083333",
100             ]
101             print(" ".join(cmd))
102             if not args.dry_run:
103                 ret = subprocess.run(cmd)
104                 if ret.returncode != 0:
105                     print(f"[benchmark] failed on {piece}/beats", file=sys.stderr)
106                     return ret.returncode
107

```

```

108     print("[benchmark] finished.")
109     return 0
110
111
112 if __name__ == "__main__":
113     sys.exit(main())

```

A.5.5 src/padicmidi/cli/job_list.py

```

1  #!/usr/bin/env python3
2  """
3  padicmidi.cli.job_list - generate a job manifest for the bundled corpus.
4
5  Entry point: ``padicmidi-job-list``. Writes a file ``job_list.txt`` in
6  the current working directory with one line per job in the form
7
8      <midi> <piece_id> <axis> <p> <Nmax>
9
10 ready to be consumed by SLURM array jobs, GNU parallel, or hand
11 piped through ``padicmidi-run-one``.
12 """
13
14 import argparse
15 import sys
16
17 from padicmidi import __version__
18
19 # Movimientos por suite:
20 # 1=Prelude, 2=Allemande, 3=Courante, 4=Sarabande, 5=Menuet/Gavotte, 6=Gigue
21
22 PIECES = [
23     # — BWV 1007 Mutopia (LilyPond 2018, Public Domain) —————
24     # Fuente canónica: un solo tempo por movimiento, sin artefactos
25     ("bwv1007-1.mid", "bwv1007_pre"), # Prelude 2/2 70 BPM
26     ("bwv1007-2.mid", "bwv1007_all"), # Allemande 3/4 80 BPM
27     ("bwv1007-3.mid", "bwv1007_cou"), # Courante 3/4 60 BPM
28     ("bwv1007-4.mid", "bwv1007_sar"), # Sarabande 3/4 120 BPM
29     ("bwv1007-5.mid", "bwv1007_men"), # Menuet 3/4 120 BPM
30     ("bwv1007-6.mid", "bwv1007_gig"), # Gigue 6/8 120 BPM ← experimento clave p=2 vs p=3
31
32     # — BWV 1007 Grossman 1997 (beats-axis only) —————
33     # Incluir para replicabilidad; NO usar eje de segundos (artefactos de tempo)
34     ("cs1-1pre.mid", "cs1_pre"),
35     ("cs1-2all.mid", "cs1_all"),
36     ("cs1-3cou.mid", "cs1_cou"),
37     ("cs1-4sar.mid", "cs1_sar"),
38     ("cs1-5men.mid", "cs1_men"),
39     ("cs1-6gig.mid", "cs1_gig"),
40
41     # — BWV 1008 Mutopia (Suite 2 Re menor, CC-BY-SA 3.0) —————
42     ("bwv1008-1.mid", "bwv1008_pre"), # Prelude
43     ("bwv1008-2.mid", "bwv1008_all"), # Allemande
44     ("bwv1008-3.mid", "bwv1008_cou"), # Courante
45     ("bwv1008-4.mid", "bwv1008_sar"), # Sarabande
46     ("bwv1008-5.mid", "bwv1008_men"), # Menuet
47     ("bwv1008-6.mid", "bwv1008_gig"), # Gigue

```

```

48
49 # — BWV 1009 Mutopia (Suite 3 Do mayor, cellosuite3) —————
50 # ZIP contiene 5 movimientos (sin Gigue separado)
51 ("cellosuite3-1.mid", "bwv1009_pre"),
52 ("cellosuite3-2.mid", "bwv1009_all"),
53 ("cellosuite3-3.mid", "bwv1009_cou"),
54 ("cellosuite3-4.mid", "bwv1009_sar"),
55 ("cellosuite3-5.mid", "bwv1009_men"),
56
57 # — Toys —————
58 ("toy_binary.mid", "toy_binary"),
59 ("toy_ternary.mid", "toy_ternary"),
60 ]
61
62 # Ejes y primos
63 AXES = ["beats", "seconds"]
64 PRIMES = {2: 6, 3: 5, 5: 4, 7: 3} # prime: Nmax
65
66 # Para cs1_* restringir a beats (artefacto de tempo en seconds)
67 CS1_PIECES = {p for _, p in PIECES if p.startswith("cs1_")}
68
69 def main(argv: list[str] | None = None) -> int:
70     ap = argparse.ArgumentParser(
71         prog="padicmidi-job-list",
72         description="Generate a job manifest (job_list.txt) for the bundled MIDI corpus.",
73     )
74     ap.add_argument("--out", default="job_list.txt", help="Output file (default job_list.txt).")
75     ap.add_argument("--version", action="version", version=f"padicmidi {_version_}")
76     args = ap.parse_args(argv)
77
78     jobs = []
79     for midi, piece in PIECES:
80         axes = ["beats"] if piece in CS1_PIECES else AXES
81         for axis in axes:
82             for p, Nmax in PRIMES.items():
83                 jobs.append(f"{midi} {piece} {axis} {p} {Nmax}")
84
85     with open(args.out, "w") as f:
86         f.write("\n".join(jobs) + "\n")
87
88     print(f"{len(jobs)} jobs written to {args.out}")
89     print(f" BWV1007 Mutopia (2 axes): 6 × 2 × 4 = {6*2*4} jobs")
90     print(f" BWV1007 Grossman (beats): 6 × 1 × 4 = {6*1*4} jobs")
91     print(f" BWV1008 Mutopia (2 axes): 6 × 2 × 4 = {6*2*4} jobs")
92     print(f" BWV1009 Mutopia (2 axes): 5 × 2 × 4 = {5*2*4} jobs")
93     print(f" Toys (2 axes): 2 × 2 × 4 = {2*2*4} jobs")
94     print(f" TOTAL: {len(jobs)}")
95     print(f"\nFirst jobs:")
96     for j in jobs[:6]:
97         print(f" {j}")
98     print(" ...")
99     return 0
100
101
102 if __name__ == "__main__":
103     sys.exit(main())

```

A.5.6 src/padicmidi/cli/mutopia.py

```

1  #!/usr/bin/env python3
2  """
3  padicmidi.cli.mutopia – download CC-licensed MIDI files from the Mutopia project.
4
5  Entry point: ``padicmidi-mutopia``. Downloads ZIPs from
6  ``mutopiaproject.org`` and extracts the BWV 1007/1008/1009 MIDI files
7  to a destination folder (defaults to the current working directory).
8  """
9
10 from __future__ import annotations
11
12 import argparse
13 import io
14 import os
15 import sys
16 import urllib.request
17 import zipfile
18 from pathlib import Path
19
20 from padicmidi import __version__
21
22 ZIPS: list[tuple[str, str]] = [
23     ("BWV1007", "https://www.mutopiaproject.org/ftp/BachJS/BWV1007/bwv1007-mids.zip"),
24     ("BWV1008", "https://www.mutopiaproject.org/ftp/BachJS/BWV1008/bwv1008-mids.zip"),
25     ("BWV1009", "https://www.mutopiaproject.org/ftp/BachJS/BWV1009/cellosuite3/cellosuite3-mids.zip"),
26 ]
27
28 SKIP = ["viola", "bwv1007.mid", "cellosuite3.mid", "bwv1008.mid"]
29
30 USER_AGENT = "Mozilla/5.0 (academic research; padicmidi/{}).format(__version__)
31
32
33 def main(argv: list[str] | None = None) -> int:
34     ap = argparse.ArgumentParser(
35         prog="padicmidi-mutopia",
36         description="Download Bach Cello Suites MIDI files from the Mutopia Project (CC-licensed).",
37     )
38     ap.add_argument(
39         "--out",
40         type=Path,
41         default=Path("."),
42         help="Destination directory (default: current directory).",
43     )
44     ap.add_argument("--timeout", type=int, default=30, help="HTTP timeout in seconds (default 30).")
45     ap.add_argument("--version", action="version", version=f"padicmidi {__version__}")
46     args = ap.parse_args(argv)
47
48     dest = args.out.expanduser().resolve()
49     dest.mkdir(parents=True, exist_ok=True)
50     print(f"Destination: {dest}\n")
51
52     total = 0
53     headers = {"User-Agent": USER_AGENT}
54     for label, url in ZIPS:
55         print(f"-> {label}: {url.split('/')[-1]}", flush=True)
56         try:

```

```

57         req = urllib.request.Request(url, headers=headers)
58         data = urllib.request.urlopen(req, timeout=args.timeout).read()
59         print(f"    ZIP downloaded: {len(data) // 1024} KB")
60         with zipfile.ZipFile(io.BytesIO(data)) as zf:
61             for name in sorted(zf.namelist()):
62                 if not name.lower().endswith(".mid"):
63                     continue
64                 base = os.path.basename(name)
65                 if any(s in base for s in SKIP):
66                     print(f"    skip: {base}")
67                     continue
68                 out_path = dest / base
69                 with zf.open(name) as src, open(out_path, "wb") as dst:
70                     dst.write(src.read())
71                 print(f"    OK: {base} ({out_path.stat().st_size} bytes)")
72                 total += 1
73         except Exception as exc:
74             print(f"    FAILED: {exc}", file=sys.stderr)
75
76     print(f"\nNew MIDIs: {total}")
77     return 0
78
79
80 if __name__ == "__main__":
81     sys.exit(main())

```

A.6 Configuración del paquete

A.6.1 pyproject.toml

```

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "padicmidi"
version = "1.0.0"
description = "A Python Toolkit for Hierarchical, Ultrametric, and p-adic Analysis of Symbolic Music Data"
readme = "README.md"
requires-python = ">=3.10"
license = { file = "LICENSE" }
authors = [
    { name = "J. Rogelio Pérez-Buendía", email = "rogelio@cimat.mx" }
]
keywords = [
    "p-adic",
    "music information retrieval",
    "symbolic music",
    "MIDI",
    "profinite",
    "ultrametric",
    "hierarchical clustering",
    "topological data analysis",
    "arithmetic data analysis"
]
classifiers = [
    "Development Status :: 5 - Production/Stable",
    "Intended Audience :: Science/Research",
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",

```

```

    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Topic :: Multimedia :: Sound/Audio :: MIDI",
    "Topic :: Scientific/Engineering :: Mathematics",
    "Topic :: Scientific/Engineering :: Information Analysis"
]
dependencies = [
    "numpy>=2.0,<3.0",
    "scipy>=1.13,<2.0",
    "networkx>=3.2,<4.0",
    "scikit-learn>=1.4,<2.0",
    "matplotlib>=3.8,<4.0",
    "mido>=1.3,<2.0",
    "joblib>=1.3,<2.0"
]

[project.optional-dependencies]
pretty = ["pretty_midi>=0.2.10"]
dev = [
    "pytest>=8.0",
    "pytest-cov",
    "ruff",
    "mypy"
]

[project.scripts]
padicmidi = "padicmidi.cli.main:main"
padicmidi-run-one = "padicmidi.cli.run_one:main"
padicmidi-run-suite = "padicmidi.cli.run_suite:main"
padicmidi-benchmark = "padicmidi.cli.benchmark:main"
padicmidi-job-list = "padicmidi.cli.job_list:main"
padicmidi-mutopia = "padicmidi.cli.mutopia:main"

[project.urls]
Homepage = "https://github.com/rperezbuendia/padicmidi"
Documentation = "https://github.com/rperezbuendia/padicmidi#readme"
Issues = "https://github.com/rperezbuendia/padicmidi/issues"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
minversion = "8.0"
testpaths = ["tests"]
python_files = ["test_*.py"]
addopts = "-q --strict-markers"

[tool.ruff]
line-length = 110
target-version = "py310"

```

A.6.2 requirements-pinned.txt

```

# Pinned to exact versions used by the author for bit-equivalent reproduction.
# Tested on macOS arm64 (M-series), Python 3.13.3.
# Minor numerical drift may occur on other platforms; for distribution see pyproject.toml.

numpy==2.4.3
scipy==1.17.1
networkx==3.6.1
scikit-learn==1.8.0
matplotlib==3.10.8

```

```

mido==1.3.3
pretty_midi==0.2.11
joblib==1.5.3

# Indirect (kept for full reproducibility):
contourpy==1.3.3
cycler==0.12.1
fonttools==4.62.0
importlib_resources==6.5.2
kiwisolver==1.5.0
packaging==26.0
pillow==12.1.1
pyparsing==3.3.2
python-dateutil==2.9.0.post0
six==1.17.0
threadpoolctl==3.6.0

```

A.6.3 LICENSE (MIT)

MIT License

Copyright (c) 2026 Jesús Rogelio Pérez Buendía
 (publishes academically as "J. Rogelio Pérez-Buendía",
 ORCID 0000-0002-7739-4779, <https://www.cimat.mx/~rogelio.perez>)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

B Manifiesto SHA-256 del dossier

El siguiente manifiesto enumera todos los archivos del dossier INDAUTOR con su hash SHA-256 individual. Cualquier modificación posterior de cualquiera de estos archivos invalidaría el hash global del ZIP de fuentes. El manifiesto se regenera automáticamente con el script `scripts/make_indautor_dossier.sh`.

```

# Manifest of files in the INDAUTOR dossier – 2026-04-30T02:18:31Z

0ba086b8240d6c71216cda3e509972bb0daa0b4c52bceca8dae923a80cd497d  indautor/REPORTE-TECNICO.pdf
d19651adc3e0de90a2f6028313d7be4c4625834ff78b38b51483ed3696ae7e91  indautor/RPDA-03-prefilled.md
6595a25b19748cfa4b0024357a08b075a672fa43cfb2ccc34ab486ed7a614f79  indautor/VERSION-REGISTRADA.md
e3d1b8b32ee6dbcccbd60381bc9151ffa60287bf5f6bf18141a27b92877acf66  indautor/carta-titularidad.md
d4e65cadd50b06f380b83e3be741d5e330009e7c455b499f07adcb800075161  indautor/codigo-fuente/__init__.py
9e4d37338d0e7db7799be78816027aa11038c86e3dae2fe36de02664717969c5  indautor/codigo-fuente/analysis/__init__.py
a9e6a12be276ea420e71160ab41fc734d6eab52c1c9fe87a9cdf3a2568b8693e  indautor/codigo-fuente/analysis/aggregate.py
7b9dc76d1ecf3b828c0c37c841e905b5c729c073f0b3cf59a05361738d1b66be  indautor/codigo-fuente/analysis/audit.py

```

790a2e6469b2567f10734fecf99840590865c04277692e28b8d162398f46c3481443155897346cb8ab63b08db2dccc12dbb163e7022969f50db6a5e630e33db1586958c2563dba963707ab75078f8d2e3739a10caf2fe63590e66e49664cc6c5cd0779ff228489dc670ace6148daf7cd32df1f39257f8a8554c56d0734b82054d8d532a01fec089f22143f18ad8649103d64f245c661d96ca737b78a576debcc3507f4f5851d97a71589cfa84532566a86df269f378e7447f7cef80b74abd6073928b8957403ec0041530d9a7aaf5c2bae277705e01300982c91db91ef2d4492d5ea3e566a383e9d3024dbe5b021a9eabc6c63fceb8783058d9ecd219aab29ed1e2af7c64d1e82bdf6fc062742e5bd7f242896e40b46c66ced636142e4c9e83fbeb4264ecc79cad312880b30d75c2de801c562158061dbd33174f6cf666d1d28aa8aee8d258c84e5f2911a99657f35d0fb118c7e0bb830d21e6ad293f2db6c32aalcba59d2b91626b985c7daf435b00388303de99f793eab7da99457080bd52c657cbe8f8171dc8dcfc4ea67d48ce8a7ba25b96c98f7666cd5dc738a668c22cfe17e1c8e76d53801a736301f4c1dd68aba2b7a273c170f44a06e02f60139cbb70060f3548c73950f175e74292f730c90b2d618801e0d6cbe6dd0d4bbb785cb2585d8f8110f104e3f4144e0672c2158377a8ca25e1aa3a52cf0871daf7295e434f4a8168cadfce8d0c8649cdd689b23fa6392e16e87051111815c076b47819d81b4ad04827526e62ec0a0662dcd3e3572dfb7c41f32955c260772e4cbeaac16f68e6c801e003fa3256442ca35aae7659dea110613ce1695b5086e1f01053641c37e17d68ed7aa6f007af1ab97e59484be94dbb23834a1081b8a464cfd91f46d92120f2e1180630b7a07c419cd987808f2d7ee8670b5f7860f85802095eeaa85e2b0e80eaf548f140d96e97d9ddcdc2ff1402089e8e81e43cf57226a3986396fdccbbe9534ce0e4785afe919a93c67dfe674e939452d6d5ce5f85b87a89d56control_primes_summary_legacy.py	indautor/codigo-fuente/analysis/directionality.py indautor/codigo-fuente/analysis/null_model.py indautor/codigo-fuente/cli/__init__.py indautor/codigo-fuente/cli/benchmark.py indautor/codigo-fuente/cli/job_list.py indautor/codigo-fuente/cli/mutopia.py indautor/codigo-fuente/cli/run_one.py indautor/codigo-fuente/cli/run_suite.py indautor/codigo-fuente/core/__init__.py indautor/codigo-fuente/core/config.py indautor/codigo-fuente/core/echo.py indautor/codigo-fuente/core/hierarchical.py indautor/codigo-fuente/figs/__init__.py indautor/codigo-fuente/figs/hierarchical_figs.py indautor/codigo-fuente/figs/ladder_balls.py indautor/codigo-fuente/figs/padic_balls.py indautor/codigo-fuente/figs/padic_tree.py indautor/codigo-fuente/figs/spectral_precision.py indautor/codigo-fuente/io/__init__.py indautor/codigo-fuente/io/midi_mido.py indautor/codigo-fuente/io/midi_pretty.py indautor/codigo-fuente/legacy/__init__.py indautor/codigo-fuente/legacy/
a1688548f50a435727d9b23f0789ff01f4a9608b5861f61f0dc2fa50233e978b30812db4d8f0f30388c700a663eaf61fcc8b3812849f0aa777e436e964fc7b0d58c4842ae887d3984db2d757feda6a6f27655c4b8d54ab0f7af17eb1b945dc4b0474320cf7de1cd15b619aacabbb764e7218156b84b93b549fa1235ba465f54bS_n_prototypes_p2_n1.csv	indautor/descripcion-funcional.md indautor/evidencia-corridas/01_quickstart_run.log indautor/evidencia-corridas/04_test_suite_output.txt indautor/evidencia-corridas/quickstart_p2/
ddebaa9edd2a5b0ba0046cf4d717fda3079d7dabdd0a9f6c652140364934de5c5S_n_prototypes_p2_n2.csv	indautor/evidencia-corridas/quickstart_p2/
d6c27da8f84665c8e2ca88f22804e298d53792394f9907bba12bf4c8cb40d7feS_n_prototypes_p2_n3.csv	indautor/evidencia-corridas/quickstart_p2/
9226ac308e7dc4aaff35ebb4a44ff35a7f30764dca3e29287c1953dee34f63cS_n_prototypes_p2_n4.csv	indautor/evidencia-corridas/quickstart_p2/
6504319260d94a1f9fd1cd9412ddd3c63f0cdfcdf4a3de52e94f4649cf4e6f52S_n_prototypes_p2_n5.csv	indautor/evidencia-corridas/quickstart_p2/
c6aad984061bfd84f073bfb3c16c57c55a0cc68d761f7e6a19cab92713883a3bS_n_prototypes_p2_n6.csv	indautor/evidencia-corridas/quickstart_p2/
4244112a70f1bdafabfdfaedd8b32914fd2046b6243c60a37139abdf2e607da8c73f944bea8a36dbee16cf8940ac6bdd6c84991695482656723da4b0e14695ba	indautor/evidencia-corridas/quickstart_p2/audit_p2.csv indautor/evidencia-corridas/quickstart_p2/
coherence_hier_p2.csv	
a438816f46a0ab9a666c3270cbfe802965afa7d0c5acd6091c17c5036355dd94e282a2da1b1dbab38b1238de5c795a59bfead307dcee3d8c20d7c3ac717bf7130df5c6b5cc5b9b0790d11ce58c38c3e71789da693707c19296dd710a66de2de2c2d8544473132d3aa7698bfee8263b7d699d87ce55d107832f41d61ae31c5ef0fe4b00bad65abaea775c9899a6007d15f402b9937c19108de182aca808cd9b6cf106b97fc79fa92f099917d499929acb988ef56bb12838427cc7107543b839e3ba00e4295725b47e1114e12fb9fa204b89ddd9d6f7a69fb3081d361f7dafc83da12d2f48a46ae615a9c267f8588223c7c6a6169708c0a8d3003fa73877d39b651424d793fd39e7f8ebd7f11e5ebcea92aca61c9e3fe4534b844359906bb752d	indautor/evidencia-corridas/quickstart_p2/f_p2_n1.csv indautor/evidencia-corridas/quickstart_p2/f_p2_n2.csv indautor/evidencia-corridas/quickstart_p2/f_p2_n3.csv indautor/evidencia-corridas/quickstart_p2/f_p2_n4.csv indautor/evidencia-corridas/quickstart_p2/f_p2_n5.csv indautor/evidencia-corridas/quickstart_p2/f_p2_n6.csv indautor/evidencia-corridas/quickstart_p2/params.json indautor/evidencia-corridas/quickstart_p2/params.txt indautor/evidencia-corridas/quickstart_p2/
pi_p2_n2_to_n1.csv	
35ee53a947b55e7ce5d9c7e1b8b5cbbee046e5b947b6e300c9c82ad38a13a5612pi_p2_n3_to_n2.csv	indautor/evidencia-corridas/quickstart_p2/
d2503331b21c60dcfded94b088654368f1d9cf6f235154bae600638c1b72e27fpi_p2_n4_to_n3.csv	indautor/evidencia-corridas/quickstart_p2/
0b1a288f5766269a3ab3a3302bf6a396d937df0aca7c29a5b801df884eb9e0d9pi_p2_n5_to_n4.csv	indautor/evidencia-corridas/quickstart_p2/
57a08b02e2fc9fe667e748826f538f87334a7f7b91bbe16afb771d3c849cc502pi_p2_n6_to_n5.csv	indautor/evidencia-corridas/quickstart_p2/
1f0d5d63d3fab2bea5ae986c788651bc0983fb85d0a0d278912ad749a5a35fc53cbd91f099f1cf093823031da427cf99922590ca960face4c33dbfc857519fda	indautor/manifiesto-archivos.md indautor/manual-tecnico.md indautor/manual-usuario.md indautor/padicmidi-v1.0.0-source.zip
d5ce0916b68f4b11e69c706d170b0403e2266bce7d7883a22982539667c6a824b5ea901eba0686f5a6a887a060a31f61767a16567b4a54657a0d970f732c92b1	

C Cómo subir el software a la página personal

El paquete está preparado para ser publicado en <http://personal.cimat.mx:8181/~rogelio.perez/desarrollo-tecnologico/> (URL canónica futura: <https://www.cimat.mx/~rogelio.perez/desarrollo-tecnologico/>). La carpeta web-site/ contiene exactamente cuatro archivos HTML autocontenidos listos para subir tal cual:

```
web-site/|—
index.html      Landing en español (página principal)|—
index.en.html   Landing en inglés|—
applet.html     Applet de análisis (inglés)|—
applet-es.html  Applet de análisis (español)
```

Vía SSH / SCP.

```
# Subir los cuatro archivos HTML al servidor:
scp web-site/*.html rogelio@personal.cimat.mx:~/public_html/desarrollo-tecnologico/padicmidi/

# O bien por rsync:
rsync -av web-site/ rogelio@personal.cimat.mx:~/public_html/desarrollo-tecnologico/padicmidi/
```

Vía sitio Hugo (Hugo Blox). Si la página personal del autor está construida con Hugo, copiar los applets a static/:

```
cp web-demo/applet.html web-demo/applet-es.html \
~/sitio-hugo/static/desarrollo-tecnologico/padicmidi/
```

y enlazar desde la página de *Desarrollo de software* con un `<iframe>` o un botón “Abrir la demo web”, en línea con el patrón ya empleado para *p-adic GRN Suite*.

Aviso legal sugerido para la página de descarga.

***Derechos de autor (obra software).** © 2026 Jesús Rogelio Pérez Buendía. Derechos de autor en trámite ante el INDAUTOR (formato RPDA-03), con base en los artículos 13 fracción XI y 24 a 26 de la Ley Federal del Derecho de Autor. El código fuente se distribuye bajo licencia MIT; la documentación y los resultados verificados, bajo Creative Commons Attribution 4.0 Internacional (CC-BY 4.0). Se reservan los derechos morales reconocidos por los artículos 18 a 23 de la LFDA.*

D Resumen de archivos del dossier INDAUTOR

Archivo	Contenido
indautor/RPDA-03-prefilled.md	Pre-llenado del formato oficial
indautor/carta-titularidad.md	Carta de titularidad firmada
indautor/descripcion-funcional.md	Descripción funcional ampliada
indautor/VERSION-REGISTRADA.md	Identificación de la versión y hash global
indautor/manifiesto-archivos.md	Hashes SHA-256 individuales
indautor/manual-usuario.md	Manual de usuario (copia)
indautor/manual-tecnico.md	Manual técnico (copia)
indautor/codigo-fuente/	Copia del paquete src/padicmidi/
indautor/evidencia-corridas/	Logs de tests y quickstart con CSV resultantes
indautor/padicmidi-v1.0.0-source.zip	Paquete fuente firmado por hash
indautor/REPORTE-TECNICO.pdf	Este documento

Referencias

- [1] Pérez-Buendía, J.~R., *Prime-power indexed multiscale graph diagnostics for symbolic temporal data: methodological exploration and delimitation via BWV~1007*, sometido a *Journal of Mathematics and Music* (Taylor & Francis), 2026.
- [2] Pérez-Buendía, J.~R., *Profinite hierarchical patterns and prime-indexed multiscale invariants in symbolic music*, sometido, 2026.